

RÉPUBLIQUE TUNISIENNE
MINISTÈRE DE L'ENSEIGNEMENT SUPÉRIEUR
ET DE LA RECHERCHE SCIENTIFIQUE
UNIVERSITÉ DE SFAX
FACULTÉ DES SCIENCES ÉCONOMIQUES ET DE GESTION DE SFAX



PROJET DE FIN D'ÉTUDES

Présenté en vue de l'obtention du diplôme de Licence en
INFORMATIQUE DE GESTION

CONCEPTION ET RÉALISATION D'UNE PLATEFORME DE FINANCEMENT COLLABORATIF

Élaboré par : Gdoura Karim

Les membres de jury

M. Iskandar Keskes

M. Nabil Baklouti

M. Mahdi Ellouze

Année universitaire 2025–2026

Remerciements

Je tiens à exprimer toute ma reconnaissance aux personnes qui ont contribué, de près ou de loin, à la réalisation de ce mémoire et à l'aboutissement de mon parcours de licence.

J'adresse mes remerciements les plus sincères à mon encadreur académique, M. Iskandar Keskes, pour la confiance qu'il m'a accordée, la rigueur de ses remarques et la disponibilité dont il a fait preuve tout au long de l'élaboration de ce travail.

Je remercie également les membres du jury, pour l'honneur qu'ils me font en acceptant d'examiner ce mémoire et pour l'attention qu'ils porteront à sa lecture.

Enfin, j'associe à ce travail ma famille et mes proches : leur patience, leurs encouragements et leur soutien moral ont été, à chaque étape, une source d'énergie et de persévérance.

Table des matières

Remerciements

Liste des abréviations

Introduction générale	1
1 Définition du projet et analyse des solutions existantes	3
1.1 Introduction	3
1.2 Définition de la mission	3
1.2.1 Présentation de l'organisme d'accueil	3
1.2.2 Présentation de l'application	4
1.2.3 Objectifs à atteindre	5
1.3 Contexte du sujet	6
1.4 Analyse d'applications existantes	7
1.4.1 Plateformes internationales de référence	7
1.4.2 Plateformes tunisiennes	8
1.5 Problématique et solution proposée	9
1.6 Conclusion	11
2 Capture des besoins	12
2.1 Introduction	12
2.2 Identification des acteurs du système informatisé	12
2.3 Contexte du système informatisé (boîte noire)	13
2.4 Identification des cas d'utilisation	15
2.5 Description textuelle des cas d'utilisation	16

2.5.1	Inventaire exhaustif des cas d'utilisation	16
2.5.2	Fiches détaillées (format annexe)	19
2.6	Conclusion	24
3	Analyse et conception	25
3.1	Introduction	25
3.2	Réalisation conceptuelle des cas d'utilisation	25
3.2.1	Annuler une contribution	26
3.2.2	Créer et soumettre un projet	29
3.2.3	Investir dans un projet (paiement + <i>webhook</i>)	32
3.2.4	Valider un projet (Expert & Admin)	36
3.2.5	Fournir les coordonnées bancaires	38
3.2.6	Approuver un retrait (Admin)	40
3.2.7	Mise à jour de l'état d'un projet (Admin)	43
3.3	Modèle du domaine	45
3.4	Architecture du futur logiciel	47
3.4.1	Vue d'ensemble en couches	47
3.4.2	Diagramme de paquetages	47
3.4.3	Décisions de conception structurantes	50
3.5	Modèle de données MongoDB	50
3.5.1	Collections et structure des documents	51
3.5.2	Justification du modèle NoSQL	51
3.5.3	Indexation et relations	52
3.5.4	Dictionnaire des données	53
3.6	Conclusion	56
4	Implémentation	57

4.1	Introduction	57
4.2	Environnement de réalisation	57
4.2.1	Environnement logiciel	57
4.2.2	Conteneurisation (Docker) et expérimentation n8n	63
4.2.3	Orchestration et traitements asynchrones	63
4.2.4	Gestion de version et collaboration (GitHub)	64
4.2.5	Synthèse des choix technologiques	64
4.2.6	Validation et tests	65
4.3	Réalisation technique des cas d'utilisation	66
4.3.1	Annuler une contribution	66
4.3.2	Créer et soumettre un projet à l'analyse IA	68
4.3.3	Investir dans un projet (paiement simulé)	71
4.3.4	Valider ou rejeter un projet (Expert et Administrateur)	76
4.3.5	Coordonnées bancaires et demande de retrait	79
4.3.6	Approuver un retrait (Administrateur)	81
4.3.7	Suspendre ou réactiver un projet (Administrateur)	84
4.4	Conclusion	89
	Conclusion générale	90
	Webographie	93
	Bibliographie	94
	Annexe	95
	Format type d'une description textuelle de cas d'utilisation	95

Table des figures

1	Vue de contexte du système (boîte noire) — diagramme de collaboration UML entre acteurs externes et le SI	14
2	Diagramme de cas d'utilisation	16
3	Diagramme de séquence – Annuler une contribution	26
4	Diagramme de classes participantes – Annuler une contribution	28
5	Diagramme de séquence – Créer et soumettre un projet	29
6	Diagramme de classes participantes – Créer et soumettre un projet	31
7	Diagramme de séquence – Investir (paiement + <i>webhook</i>)	32
8	Diagramme de classes participantes – Investir dans un projet (paiement démo + <i>webhook</i>)	35
9	Diagramme de séquence – Valider ou rejeter (expert) et publier (administrateur)	36
10	Diagramme de classes participantes – Valider ou rejeter (expert) et publier (administrateur)	37
11	Diagramme de séquence – Fournir les coordonnées bancaires	38
12	Diagramme de classes participantes – Fournir les coordonnées bancaires	39
13	Diagramme de séquence – Approuver un retrait (Admin)	40
14	Diagramme de classes participantes – Approuver un retrait (Admin)	42
15	Diagramme de séquence – Mise à jour de l'état d'un projet (suspension / réactivation)	43
16	Diagramme de classes participantes – Suspendre ou réactiver un projet (Admin)	44
17	Diagramme de classes du domaine (modèle du domaine)	46
18	Diagramme de paquetages — architecture logicielle (alignée sur le dépôt)	48
19	Mes contributions — statuts et actions	67
20	Annulation : contrôle d'éligibilité et passage en CANCELLING	68

21	Création de projet — étape 1	69
22	Création de projet — étape 2 (compléments du formulaire)	69
23	Workflow n8n — analyse de risque	70
24	Soumission d'un projet DRAFT vers la file d'analyse IA	70
25	Soutien — saisie du montant et du pourboire	71
26	Soutien — confirmation du mode de paiement	72
27	Confirmation OTP (démonstration paiement)	72
28	Création d'investissement : contrôles métier	73
29	Création d'investissement : seuil consultation 30 %	74
30	Création d'investissement : enregistrement INITIATED / transaction PENDING .	74
31	Webhook : statut PENDING_CONSULTATION et dossier expert	75
32	Webhook : mise à jour atomique de currentFunding (anti-sur-financement) .	75
33	Expert — liste des projets en revue	76
34	Expert — détail de l'analyse IA avant décision	77
35	Expert — décision de validation ou de rejet	77
36	Administrateur — publication finale	78
37	Validation expert : contrôles et décision APPROVED	78
38	Validation expert : notifications et branche REJECTED	79
39	Saisie des coordonnées bancaires	80
40	Chiffrement du RIB (seal) et passage du retrait en READY	80
41	Console administrateur — retrait en statut READY (approbation possible)	82
42	Console administrateur — retrait après approbation (PROCESSING / COMPLETED)	82
43	Approbation admin : contrôles et initiation du transfert simulé	83
44	Approbation admin : statut PROCESSING et notification	83
45	Console administrateur — suspension d'un projet (SUSPENDED)	85
46	Suspension : contrôles et transition vers SUSPENDED (1/3)	86

47	Suspension : persistance et journal d’audit (2/3)	87
48	Suspension : notification PROJECT_SUSPENDED (3/3)	87
49	Réactivation : contrôles préalables (1/3)	88
50	Réactivation : transition UNDER_REVIEW ou AWAITING_AI (2/3)	88
51	Réactivation : notification PROJECT_REACTIVATED (3/3)	89

Liste des tableaux

1	Comparaison ciblée des plateformes au regard de la problématique retenue . . .	10
2	Inventaire des cas d'utilisation — paquetage Comptes	17
3	Inventaire des cas d'utilisation — paquetage Projets	17
4	Inventaire des cas d'utilisation — paquetage Contributions & finance	18
5	Inventaire des cas d'utilisation — paquetage Échanges & confiance	18
6	Inventaire des cas d'utilisation — paquetage Administration	19
7	Description textuelle du cas « Créer et soumettre un projet »	20
8	Description textuelle du cas « Soutenir un projet »	21
9	Description textuelle du cas « Valider ou rejeter (expert) et publier (admin) » . .	22
10	Description textuelle du cas « Voir les projets recommandés »	23
11	Description textuelle du cas « Signaler un contenu »	23
12	Services internes du serveur applicatif	47
13	Dictionnaire des données	53
14	Technologies utilisées — socle applicatif, données et sécurité	64
15	Technologies utilisées — IA, orchestration et exploitation (suite)	65
16	Traçabilité — sept cas alignés sur les diagrammes UML du chapitre 3	66
17	Modèle de description textuelle d'un cas d'utilisation	95

Liste des abréviations

Abrév.	Signification
AES-GCM	Advanced Encryption Standard - Galois/Counter Mode (chiffrement authentifié)
API	Application Programming Interface (interface de programmation)
BCT	Banque Centrale de Tunisie
BullMQ	File de jobs Node.js bâtie sur Redis
CDN	Content Delivery Network
CRUD	Create, Read, Update, Delete
DLQ	Dead-Letter Queue (file des échecs définitifs)
HMAC	Hash-based Message Authentication Code
HTTPS	Hypertext Transfer Protocol Secure
IA	Intelligence Artificielle
JWT	JSON Web Token
KPI	Key Performance Indicator
LLM	Large Language Model (modèle de langage de grande taille)
n8n	Outil d'orchestration de workflows
ODM	Object-Document Mapping (mapping objet-document, ex. Mongoose)
OTP	One-Time Password (code à usage unique)
PFE	Projet de Fin d'Études
RBAC	Role-Based Access Control (contrôle d'accès par rôle)
REST	Representational State Transfer
LPDP	Loi tunisienne relative à la protection des données à caractère personnel
RIB	Relevé d'Identité Bancaire
SPA	Single Page Application
SSL/TLS	Secure Sockets Layer / Transport Layer Security
TND	Dinar tunisien
UML	Unified Modeling Language

Introduction générale

Le financement participatif connaît un essor notable en Tunisie, porté par la loi n° 2020–37 qui encadre les opérations de crowdfunding et place leur supervision sous l'autorité de la Banque Centrale de Tunisie. Ce mode de financement constitue une alternative aux circuits bancaires classiques, en permettant à un porteur de projet de mobiliser une communauté d'investisseurs autour d'une idée entrepreneuriale, sociale ou innovante. Il introduit toutefois de nouvelles exigences : les investisseurs doivent disposer d'informations fiables, les porteurs doivent présenter leurs projets de manière structurée et l'opérateur de la plateforme doit garantir la traçabilité des opérations financières et des décisions administratives.

Dans ce contexte, la confiance devient un facteur déterminant. Une plateforme de financement collaboratif ne peut pas se limiter à publier des campagnes et à collecter des contributions ; elle doit aussi accompagner la décision d'investissement, expliquer les risques, assurer le suivi des statuts, gérer les notifications et contrôler les transitions sensibles telles que la validation d'un projet, le remboursement d'une contribution ou l'approbation d'un retrait. Or, plusieurs solutions existantes restent centrées sur la mise en relation et proposent peu de personnalisation, peu d'aide à l'analyse et peu d'automatisation autour des événements critiques.

La problématique traitée dans ce projet consiste donc à concevoir et réaliser une plateforme web capable de soutenir tout le cycle de vie d'un financement collaboratif, depuis la création d'un projet jusqu'au retrait des fonds, tout en améliorant l'aide à la décision des investisseurs. La solution proposée vise à combiner des fonctionnalités classiques de crowdfunding avec des mécanismes plus avancés : recommandations personnalisées, analyse de risque automatisée par modèle de langage, assistant conversationnel sur les fiches projet, consultation d'un expert en finance pour les investissements importants, notifications métier et supervision administrative.

La plateforme distingue plusieurs acteurs aux responsabilités complémentaires. Le porteur de projet crée un brouillon, complète les informations nécessaires, soumet son projet à l'analyse de risque et suit son parcours de validation. L'investisseur consulte les projets publiés, reçoit des recommandations adaptées à son profil, contribue à une campagne, peut demander un avis d'expert et suit l'état de ses investissements. L'expert en finance intervient dans l'évaluation des projets et dans le traitement des demandes de consultation. L'administrateur assure la gouvernance globale : publication des projets approuvés, modération des commentaires, traitement des signalements, supervision des remboursements, gestion des files d'échec et approbation des retraits.

Sur le plan technique, la réalisation repose sur une architecture web moderne séparant la couche présentation, la couche métier et la couche données. L'interface utilisateur est développée avec React, tandis que le serveur repose sur Node.js et Express afin d'exposer des API REST sécurisées. La persistance est assurée par MongoDB, avec une modélisation orientée documents et l'utilisation de transactions lorsque la cohérence financière l'exige. La sécurité applicative est prise en compte à travers l'authentification, l'autorisation par rôles, les cookies HttpOnly, le contrôle des accès aux routes sensibles et la protection des données critiques comme les coordonnées bancaires.

L'intégration de l'intelligence artificielle constitue un axe important du projet. Elle intervient d'abord dans l'analyse préliminaire du risque associé à un projet, sous forme d'un rapport structuré destiné à faciliter la revue par l'expert et l'administrateur. Elle intervient ensuite dans l'assistant conversationnel, qui permet à l'utilisateur de poser des questions contextualisées sur un projet avant de prendre une décision. L'objectif n'est pas de remplacer la validation humaine, mais de fournir un support d'analyse, d'explication et d'orientation, tout en conservant une gouvernance explicite par les rôles métier.

La démarche suivie dans ce mémoire part de l'étude du contexte et des solutions existantes, puis formalise les besoins à travers les acteurs, les cas d'utilisation et les scénarios représentatifs. Elle se poursuit par une conception UML et une modélisation des données, avant de présenter la réalisation technique, les choix d'environnement, les captures de l'application et les extraits de code les plus significatifs. Cette progression permet de relier les besoins fonctionnels aux décisions de conception, puis aux éléments réellement implémentés dans la solution.

Le mémoire est structuré en quatre chapitres. Le premier chapitre définit le projet, analyse les solutions existantes et pose la problématique. Le deuxième chapitre formalise la capture des besoins fonctionnels. Le troisième chapitre présente l'analyse et la conception technique, notamment le modèle du domaine, les diagrammes de séquence, l'architecture du futur logiciel et le modèle de données MongoDB. Le quatrième chapitre expose l'implémentation de la solution, l'environnement de réalisation, les cas d'utilisation développés, la stratégie de validation, les limites constatées et les perspectives d'évolution.

Chapitre 1

Définition du projet et analyse des solutions existantes

1.1 Introduction

Le présent chapitre dresse le cadre général du projet. Il expose la mission confiée par l'organisme d'accueil, l'application visée et les objectifs assignés. Il contextualise ensuite le sujet sur les plans économique et réglementaire, examine les principales solutions concurrentes sur le segment du financement participatif, puis formule la problématique et la solution retenue. Ce chapitre constitue ainsi le socle interprétatif des besoins qui seront développés au chapitre suivant.

1.2 Définition de la mission

1.2.1 Présentation de l'organisme d'accueil

Le stage du présent projet de fin d'études s'est effectué au sein d'Aziin Engineering Solution, une jeune société d'ingénierie basée à Sfax. L'organisme accompagne la conception et la réalisation de solutions informatiques ; sa taille modeste favorise une organisation souple, une communication directe entre les membres de l'équipe et une prise de décision rapide sur l'orientation du projet.

La mission réalisée en entreprise a consisté à porter, de l'expression du besoin jusqu'au produit opérationnel, la plateforme de financement collaboratif documentée dans ce mémoire. Le travail a couvert le cadrage fonctionnel, la production des livrables logiciels et les points de validation internes (pilotage de l'avancement, relectures techniques, arbitrages sur l'architecture retenue). Ce cadre professionnel constitue le socle du présent mémoire ; le détail des choix de mise en œuvre est développé au chapitre consacré à l'implémentation.

1.2.2 Présentation de l'application

L'application réalisée est une plateforme web de financement collaboratif (*crowdfunding*) qui met en relation quatre catégories d'acteurs métier :

- les **porteurs de projet** qui souhaitent financer une campagne (titre, description, catégorie, objectif financier, échéance, médias) ;
- les **investisseurs** qui souhaitent contribuer à des campagnes correspondant à leur profil (catégories préférées, tolérance au risque) ;
- les **experts en finance** qui interviennent dans l'évaluation des projets et dans l'accompagnement des investissements nécessitant un avis spécialisé ;
- les **administrateurs** qui assurent la gouvernance de la plateforme (publication des projets approuvés, modération des contenus, traitement des signalements, supervision des retraits).

Les concepts métier centraux portés par l'application sont les suivants :

- **Projet** : campagne de financement caractérisée par un cycle de vie explicite (DRAFT, AWAITING_AI, UNDER_REVIEW, APPROVED, ACTIVE, REJECTED, FUNDED, CLOSED, SUSPENDED).
- **Investissement** : engagement d'un investisseur pour un montant donné sur un projet, créé en INITIATED puis confirmé en SUCCESS (ou REFUNDED en cas de sur-financement) après réception du *webhook* de paiement.
- **Analyse de risque** : rapport structuré (résumé, avantages, inconvénients, améliorations, éléments à retirer, questions, score/riskLevel) produit par Gemini à la soumission ; le déclenchement est orchestré par n8n (file BullMQ), l'appel au modèle et la persistance passant par l'API Express (/internal/run-risk-analysis).
- **Recommandations personnalisées** : classement déterministe des projets ACTIVE selon les catégories préférées, le riskLevel issu de l'analyse IA, la progression de collecte et la récurrence (sans appel LLM dédié).
- **Notifications** : chaque message intégré à l'application est stocké en base avec un destinataire explicite ; l'interface expose une boîte personnelle commune à tous les rôles (y compris l'administrateur), avec pagination (30 entrées par page, tri antéchronologique), indicateur lu/non lu et compteur des messages non lus. Les courriels asynchrones ne concernent que les événements jugés critiques, afin de limiter les notifications superflues.
- **Retrait (payout)** : flux de versement au porteur de projet une fois la campagne FUNDED ou CLOSED, après saisie chiffrée des coordonnées bancaires, émission d'une facture numérique

séquentielle (exportable en PDF via l'interface) et approbation administrative.

- **Assistant conversationnel (IA)** : module interactif (alimenté par Gemini) intégré à chaque fiche projet, accessible aux utilisateurs authentifiés, capable de répondre aux questions à partir de la description du projet et d'un socle de connaissances générales.
- **Consultation d'expert en finance** : option offerte à l'investisseur lors de la contribution (si le montant est éligible) permettant de suspendre le paiement et de solliciter l'avis d'un expert avant confirmation de l'investissement.
- **Modèle économique hybride** : commission fixe de 5 % appliquée au versement au porteur (écart entre montant collecté et *realBudget*, ou estimation par défaut), complétée par un pourboire optionnel versé volontairement par l'investisseur lors du paiement.

1.2.3 Objectifs à atteindre

La plateforme doit répondre aux objectifs suivants, formulés à la fois sur le plan métier et sur le plan technique.

Objectifs métier :

- Permettre aux porteurs de projets de créer, soumettre, publier et suivre leurs campagnes de financement.
- Offrir aux investisseurs un parcours structuré de découverte, d'évaluation et de contribution aux projets.
- Automatiser l'évaluation préliminaire du risque via l'IA, et permettre à un expert en finance de valider cette analyse avant la publication du projet.
- Proposer une assistance interactive par assistant conversationnel pour répondre, de manière quasi immédiate, aux interrogations des investisseurs sur les projets.
- Offrir la possibilité aux investisseurs s'engageant sur des montants significatifs ($\geq 30\%$) de solliciter une boîte de consultation d'expert en finance avant de valider leur paiement.
- Personnaliser les recommandations selon le profil de l'investisseur (catégories, tolérance au risque).
- Assurer la traçabilité des transactions (statuts d'investissement, de transaction et de *payout*).
- Expérimenter un modèle économique hybride : commission de 5 % au retrait (sur la base de *realBudget*) et pourboire volontaire optionnel à l'investissement.
- Renforcer la gouvernance de la plateforme (validation, modération, signalements, retraits, factures numériques, statistiques).

Objectifs techniques :

- Concevoir une architecture modulaire en trois couches (présentation, métier, données) assurant une séparation claire des responsabilités.
- Garantir la sécurité des accès et des données grâce à des mécanismes d'authentification, d'autorisation et de protection des échanges.
- Intégrer des services d'intelligence artificielle afin d'automatiser certaines tâches d'analyse et d'assistance utilisateur.
- Assurer la maintenabilité et l'évolutivité de la plateforme grâce à une pile technologique contemporaine (Node.js, React, MongoDB).
- Mettre en place un système de paiement simulé permettant de reproduire le cycle complet des transactions financières.
- Automatiser certains traitements asynchrones tels que les notifications, les analyses et les mises à jour du système.

1.3 Contexte du sujet

Le financement participatif (*crowdfunding*) constitue une alternative aux modes de financement bancaires traditionnels. En Tunisie, le secteur connaît un essor important depuis la promulgation de la loi n° 2020-37 du 2 juillet 2020, qui encadre les opérations de financement participatif et place leur supervision sous la tutelle de la Banque Centrale de Tunisie (BCT). Ce cadre réglementaire ouvre la voie à des plateformes qui mobilisent l'épargne publique au profit de projets entrepreneuriaux, tout en imposant des exigences fortes en matière d'identification des acteurs, de traçabilité des opérations et de conservation des journaux financiers.

Sur le plan économique, ce mouvement répond à une difficulté concrète des jeunes entreprises et des PME tunisiennes pour accéder aux financements bancaires classiques, en particulier pour les projets à fort potentiel ou présentant un risque élevé. Sur le plan technologique, l'émergence des modèles de langage (LLM) et des outils d'orchestration de *workflows* (par exemple n8n) permet d'automatiser une partie de la pré-évaluation des projets et de fiabiliser les flux asynchrones (paiements, notifications, reprises) sans alourdir la base de code applicative. Le sujet du présent projet s'inscrit donc à l'intersection de la conformité réglementaire, de l'ingénierie des systèmes d'information et de l'IA appliquée à la décision métier.

1.4 Analyse d'applications existantes

Cette section présente une étude comparative des principales plateformes de financement participatif, en distinguant les acteurs internationaux qui ont structuré le marché et les acteurs locaux qui en illustrent la déclinaison tunisienne. L'objectif est d'identifier les fonctionnalités déjà offertes et, surtout, les manques que la solution proposée vise à combler.

1.4.1 Plateformes internationales de référence

Kickstarter

Kickstarter est la plateforme pionnière du crowdfunding, lancée en 2009. Elle se concentre sur le financement de projets créatifs (arts, design, technologie). Le modèle “tout ou rien” (all-or-nothing) impose que le projet atteigne 100% de son objectif pour que les fonds soient débloqués.

Atouts :

- Grande notoriété et trafic important
- Communauté engagée de créateurs
- Processus de validation qualité rigoureux

Limites :

- Absence d'analyse de risque automatisée
- Pas de personnalisation des recommandations
- Frais élevés (5% + 3-5% frais de paiement)

Indiegogo

Indiegogo offre plus de flexibilité que Kickstarter avec deux modèles : “tout ou rien” et “flexible funding” (les fonds sont conservés même si l'objectif n'est pas atteint).

Atouts :

- Flexibilité du modèle de financement
- Support de campagnes en continu (pas de deadline obligatoire)
- Partenariats avec des fabricants (InDemand)

Limites :

- Taux de succès des campagnes souvent plus faible que les leaders du marché (données publiées hétérogènes)
- Interface moins intuitive
- Absence de filtrage fondé sur le profil utilisateur

1.4.2 Plateformes tunisiennes

CoFundy

CoFundy est une plateforme tunisienne de crowdfunding lancée en 2016, spécialisée dans le financement de projets entrepreneuriaux.

Atouts :

- Intégration avec paiements tunisiens (cartes bancaires locales)
- Adaptation au contexte local et aux projets entrepreneuriaux
- Frais clairement identifiés : 5% du montant collecté

Limites :

- Absence de recommandations personnalisées
- Interface utilisateur datée
- Pas d'évaluation de risque visible pour les investisseurs
- Validation essentiellement manuelle, avec un délai de traitement pouvant atteindre 7 à 10 jours
- Intégration paiement difficile à tester sans environnement *sandbox* clairement documenté

Kaoun

Plateforme tunisienne plus récente (2019), focalisée sur les projets sociaux et environnementaux.

Atouts :

- Niche bien définie (impact social)
- Communauté engagée

Limites :

- Volume de projets limité
- Fonctionnalités limitées
- Pas d'IA ou d'automatisation

L'ensemble de ces limitations – absence d'analyse de risque automatisée, de recommandations personnalisées, de gestion structurée des paiements et de gouvernance complète – est synthétisé dans le tableau comparatif présenté en section 1.5 (tableau 1).

1.5 Problématique et solution proposée

L'analyse comparative menée en section précédente ne vise pas à reproduire une place de marché généraliste, mais à identifier les **écarts fonctionnels** qui freinent, dans un contexte tunisien encadré par la BCT, la confiance des investisseurs et l'efficacité des porteurs de projet. À partir de cette lecture, la problématique est formulée en trois dimensions : *aide à la décision*, *fiabilité des flux financiers* et *gouvernance du cycle de vie des campagnes*.

Constats et enjeux.

1. **Opacité du risque.** Sur la plupart des plateformes étudiées, l'investisseur s'appuie sur une description textuelle et une validation humaine tardive, sans synthèse standardisée ni score explicite. Le temps de revue administratif reste élevé et peu homogène.
2. **Découverte des projets peu guidée.** Les mécanismes de recommandation, lorsqu'ils existent, restent génériques : ils ne croisent pas systématiquement préférences de l'investisseur, catégorie de projet et niveau de risque issu d'une analyse formalisée.
3. **Fragilité des parcours financiers.** Paiement, annulation, sur-financement, remboursement et retrait au porteur imposent des enchaînements sensibles (notifications HTTP de rappel, idempotence, états métier). Les solutions observées offrent rarement une chaîne complète testable en environnement de démonstration, ni une reprise structurée des échecs asynchrones.
4. **Accompagnement limité à fort enjeu.** Lorsque le montant engagé devient significatif, l'investisseur ne dispose pas toujours d'un canal structuré (assistance contextuelle ou avis d'expert) avant de confirmer son engagement.
5. **Traçabilité et modération incomplètes.** Signalements, notifications personnelles, journalisation des décisions et supervision des retraits sont des prérequis de confiance, encore inégalement couverts par les acteurs locaux.

Ces constats débouchent sur la **question de recherche** suivante :

Comment concevoir une plateforme de financement collaboratif qui combine une pré-évaluation automatisée des projets, une personnalisation des recommandations et une gouvernance explicite du cycle de vie (validation, contribution, retrait), tout en respectant les exigences de sécurité et de traçabilité attendues dans un cadre réglementaire tunisien ?

Positionnement comparatif. Le tableau 1 compare, sur des critères directement liés à la problématique, les plateformes internationales de référence, deux acteurs tunisiens et la solution développée dans ce mémoire. Les critères retenus traduisent des capacités observées sur le marché ou démontrées dans le prototype implémenté ; ils ne prétendent pas à l'exhaustivité de toutes les fonctionnalités possibles d'une place de marché mondiale.

TABEAU 1 – Comparaison ciblée des plateformes au regard de la problématique retenue

Critère	Kickstarter	Indiegogo	CoFundy	Kaoun	Solution proposée
Analyse de risque automatisée (rapport structuré)	Non	Non	Non	Non	Oui (IA + heuristiques)
Recommandations personnalisées (profil + risque)	Non	Limité	Non	Non	Oui (score déterministe)
Assistance investisseur (IA / expert)	Limitée	Limitée	Non	Non	Assistant + consultation ≥ 30 %
Païement & retraits (contexte TN)	Non / US	Non / US	Partiel	Oui	Oui
Résilience des échecs (remboursements, relances)	Manuelle	Manuelle	Manuelle	Manuelle	Files + DLQ + tâches planifiées
Gouvernance (modération, signalements, audit)	Partielle	Partielle	Limitée	Limitée	Notifications, signalements, journal
Modèle économique plateforme	8–10 %	8–13 %	5 %	5 %	5 % + pourboire optionnel

Solution proposée. La réponse apportée n'est pas une juxtaposition de modules techniques, mais une **chaîne de valeur** articulée autour du cycle de vie d'une campagne :

1. **Préparer la décision.** À la soumission, une analyse IA produit un rapport lisible (forces, faiblesses, score, niveau de risque) ; une heuristique budgétaire peut écarter immédiatement les dossiers incohérents. L'expert en finance valide ou rejette ce dossier avant publication ; l'administrateur ouvre ensuite la campagne aux investisseurs.
2. **Guider l'investisseur.** Un moteur de recommandations classe les projets actifs selon le profil de l'investisseur ; un assistant conversationnel répond aux questions sur la fiche projet ; au-delà d'un seuil d'engagement (30 % de l'objectif), une consultation d'expert peut différer la confirmation du paiement.

3. **Sécuriser les flux.** Un parcours de paiement simulé, extensible vers un prestataire réel, s'appuie sur des rappels HTTP (*webhooks*) signés et idempotents, une mise à jour atomique du financement collecté et des mécanismes de remboursement en cas de sur-financement ou d'expiration.
4. **Clôturer et tracer.** Après collecte, le porteur saisit des coordonnées bancaires chiffrées, déclenche un retrait supervisé et reçoit une facture numérique (export PDF via le navigateur) ; les administrateurs modèrent les contenus, traitent les signalements et peuvent relancer les traitements en échec.
5. **Pérenniser l'exploitation.** Un modèle économique transparent (commission 5 % au retrait + pourboire volontaire à l'investissement) et une orchestration résiliente (files de messages, reprises planifiées, relance manuelle ou automatique des analyses bloquées) visent la continuité de service malgré les indisponibilités ponctuelles des services externes.

La conception détaillée, la modélisation UML et l'implémentation de cette solution sont développées respectivement aux chapitres 2, 3 et 4.

1.6 Conclusion

Ce chapitre a permis de préciser la mission (objectifs fonctionnels et techniques), de replacer le sujet dans son environnement économique et réglementaire, d'analyser les plateformes de référence retenues et d'en extraire la problématique centrale. La solution défendue repose sur une analyse par IA, sur des mécanismes de recommandation personnalisés, sur un parcours de paiement démonstratif et extensible, et sur une orchestration résiliente des traitements. Le chapitre suivant formalise les besoins métier à travers la description des acteurs et des cas d'utilisation.

Chapitre 2

Capture des besoins

2.1 Introduction

Ce chapitre formalise les besoins métier associés à la plateforme, du point de vue des acteurs qui en sont les utilisateurs ou les partenaires techniques. Il explicite le périmètre du système (représentation en « boîte noire »), recense les acteurs et leurs responsabilités respectives, structure les cas d'utilisation en paquetages métier cohérents, établit un inventaire exhaustif de ces cas (tableaux 2 à 6), puis présente, pour un sous-ensemble représentatif, des descriptions détaillées (scénario nominal, variantes, exceptions) mises en correspondance avec les points d'accès REST /api/* et /internal/* du serveur applicatif développé.

2.2 Identification des acteurs du système informatisé

La plateforme distingue des acteurs humains et des acteurs systèmes externes. Un même utilisateur peut cumuler les rôles d'*investisseur* et de *porteur de projet* ; la distinction de ces deux rôles est portée dans la description des cas d'utilisation.

- **Visiteur** : utilisateur non authentifié ; consulte et recherche les projets publics, peut créer un compte et lancer une procédure de récupération de mot de passe.
- **Utilisateur authentifié** (porteur de projet et / ou investisseur) : gère son compte, crée et soumet des projets, contribue à des campagnes, commente et signale des contenus, suit ses investissements, ses paiements et ses retraits.
- **Expert en finance** : acteur spécialisé assumant un double rôle : il valide les rapports d'analyse IA des projets avant leur publication, et il est sollicité par les investisseurs (lorsque le montant investi atteint un seuil de 30%) pour émettre un avis consultatif avant la validation finale du paiement.
- **Administrateur** : publie les projets approuvés (APPROVED → ACTIVE), peut révoquer une approbation, suspend ou réactive les projets, modère les commentaires, traite les signalements, supervise les retraits, désactive ou réactive les comptes, et

- relance les traitements échoués (notifications, retraits, remboursements). Une route `/api/admin/projects/:id/validate` permet aussi une validation directe.
- **Passerelle de paiement** : partenaire externe simulé qui confirme ou infirme les paiements via un *webhook* signé. Les messages de ce tiers déclenchent des transitions métier (statut de transaction, statut d'investissement, mise à jour du montant collecté, remboursements en cas de sur-financement).
 - **Service d'analyse de risque (orchestrateur, n8n)** : acteur externe qui consomme la file BullMQ `risk-analysis`, puis sollicite `POST /internal/run-risk-analysis` sur l'API Express (retentatives, normalisation). Il ne porte **pas** l'assistant conversationnel de la fiche projet.
 - **Fournisseur LLM (API Gemini)** : service externe invoqué par Express (`geminiRiskService`, `geminiChatService`). L'orchestrateur n8n déclenche l'analyse de risque; l'assistant (`POST /api/projects/:id/chat`) est appelé de façon **synchrone**, sans n8n.
 - **Service de messagerie (SMTP)** : partenaire externe utilisé pour les e-mails critiques (vérification, réinitialisation, décisions admin, événements de paiement / retrait).

2.3 Contexte du système informatisé (boîte noire)

Du point de vue externe, la plateforme échange des messages avec quatre grandes familles d'interlocuteurs (les canaux web humains, la passerelle de paiement, le service d'analyse de risque, la messagerie). La figure 1 en restitue la lecture : le système apparaît comme une « boîte noire » centrale, les acteurs périphériques étant reliés par des associations libellées qui matérialisent les échanges fonctionnels attendus.

- Les **acteurs humains web** (visiteur, utilisateur authentifié, expert, administrateur) interagissent avec le système via l'application web (HTTPS, REST, cookies `HttpOnly`).
- La **passerelle de paiement** envoie des confirmations signées (*webhook*) ; le système répond de façon idempotente pour absorber les retentatives.
- Le **service d'analyse de risque (orchestrateur n8n)** consomme la file BullMQ/Redis, puis appelle `/internal/run-risk-analysis` ; l'API Express invoque Gemini et met à jour le projet (succès ou échec tracé).
- Le **service de messagerie (SMTP)** est sollicité pour les e-mails critiques (vérification, réinitialisation, décisions admin, paiement/retrait).

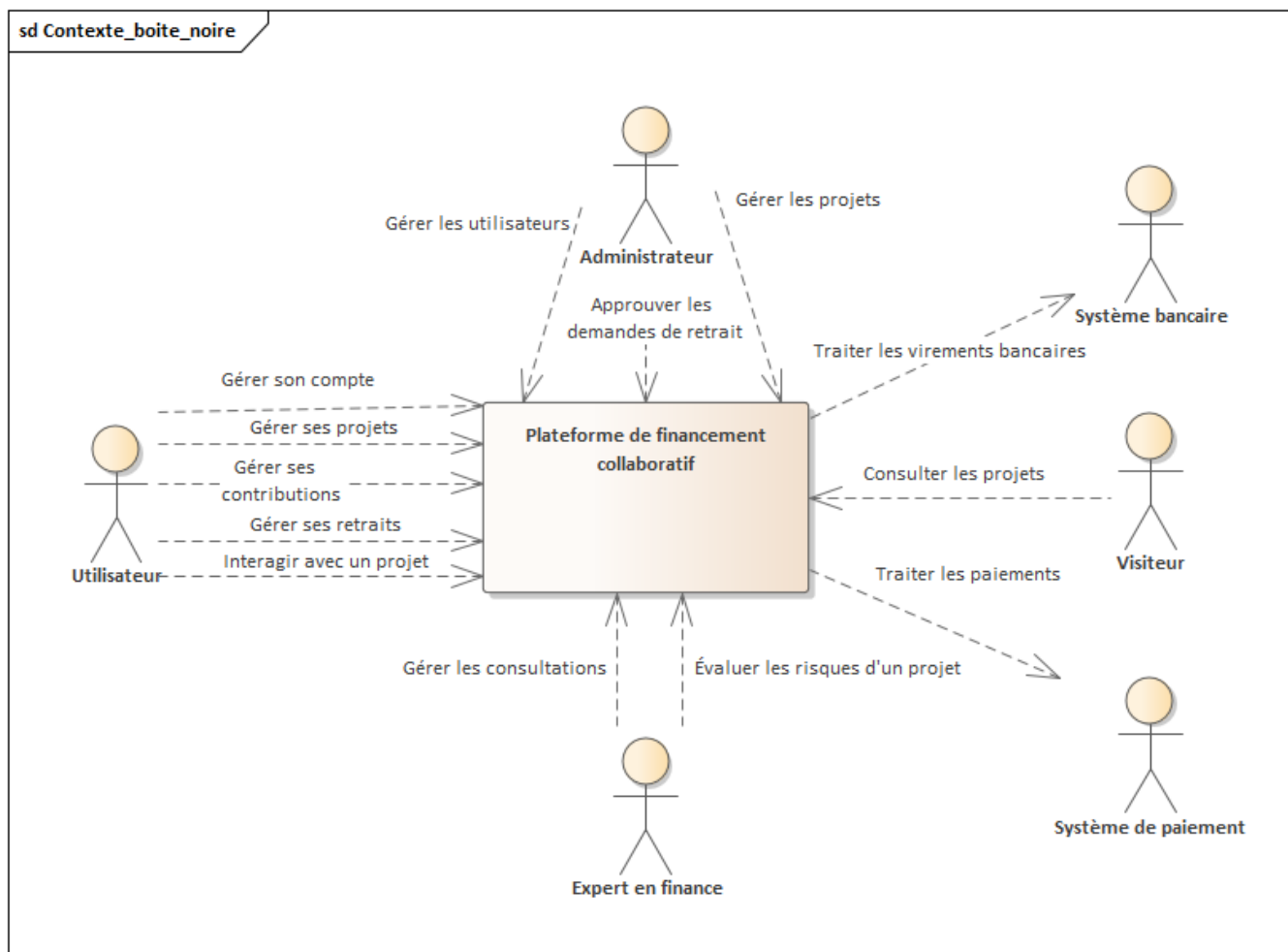


FIGURE 1 – Vue de contexte du système (boîte noire) — diagramme de collaboration UML entre acteurs externes et le SI

Le contour du logiciel se déduit de cette lecture : hors du noyau applicatif (persistance métier, API et interface utilisateur propre à la plateforme), chaque composant relève d'un partenaire externe et s'appuie sur un contrat d'interface explicite (REST, *webhook* signé, appels internes documentés).

Précision sur l'IA (lecture du schéma de contexte). La figure 1 ne fait pas apparaître l'assistant conversationnel comme acteur externe : il s'agit d'une **fonctionnalité interne** du SI (widget sur la fiche projet, route `/api/projects/:id/chat`). Seule l'**analyse de risque** est modélisée comme échange avec un orchestrateur externe (*n8n*), car elle est asynchrone, relançable et rappelle le serveur via `/internal/*`. Les deux usages s'appuient sur le même fournisseur LLM (Gemini), mais avec des chaînes d'intégration différentes : *n8n* → analyse batch à la soumission ; *Express* → question/réponse interactive à la demande de l'utilisateur authentifié.

2.4 Identification des cas d'utilisation

Les cas d'utilisation sont regroupés en thèmes métier qui structurent la plateforme (présentation synthétique). Les détails d'enchaînement et les variantes sont explicités dans les descriptions textuelles (section 2.5) et dans les diagrammes du chapitre 3.

- **Comptes** : inscription (avec OTP / lien de vérification), connexion, déconnexion, rafraîchissement de session, mot de passe oublié, mise à jour du profil, suppression de compte.
- **Projets** : exploration / consultation, recherche, création (DRAFT), soumission à l'analyse IA (AWAITING_AI), revue (UNDER_REVIEW), validation ou rejet par l'**expert** (APPROVED/REJECTED), publication par l'**administrateur** (ACTIVE), édition / re-soumission, archivage.
- **Contributions & finance** : contribution (paiement simulé avec pourboire), suivi des investissements, relance de paiement, annulation dans la fenêtre de grâce, remboursement en cas de sur-financement, retraits (*payouts*) et saisie chiffrée des coordonnées bancaires avec émission de factures numériques (export PDF via l'interface).
- **Échanges & confiance** : commentaires sur les projets publics (publication/suppression), notifications intégrées à l'application, signalement (projet ou commentaire), consultation de ses signalements, recommandations personnalisées, assistant conversationnel (LLM), demande de consultation d'expert en finance.
- **Administration** : gestion des projets (publication des projets APPROVED, révocation d'approbation, suspension/réactivation, relance analyse IA), gestion des utilisateurs (désactivation/réactivation), modération des commentaires (masquer/rétablir), traitement des signalements (résolution), supervision des retraits (approbation), relance des jobs en échec (envois SMTP, payouts, remboursements, annulations ; files DLQ / routes /api/admin/*).

La représentation graphique de ces cas d'utilisation est donnée par la figure 2.

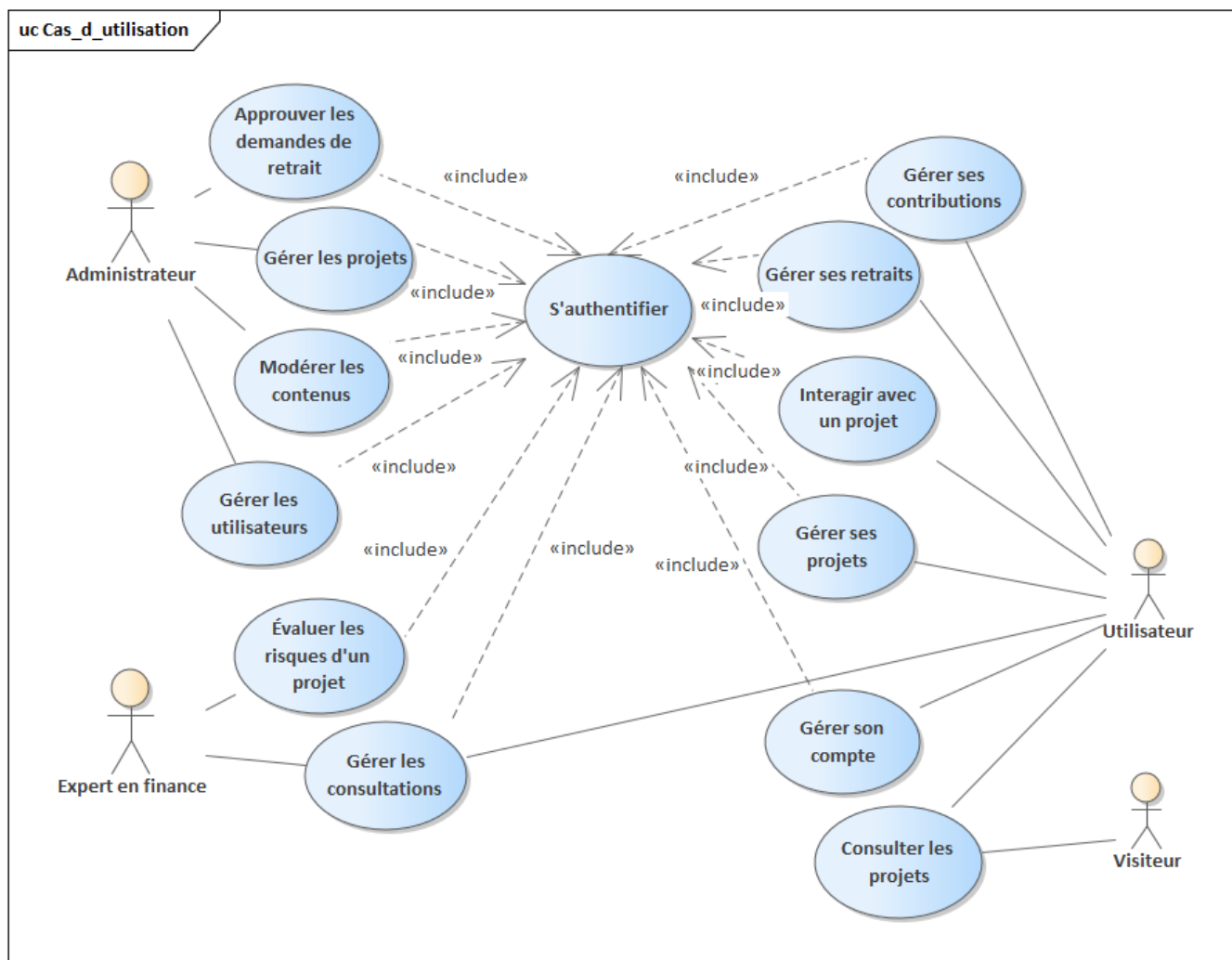


FIGURE 2 – Diagramme de cas d'utilisation

2.5 Description textuelle des cas d'utilisation

2.5.1 Inventaire exhaustif des cas d'utilisation

Les tableaux 2 à 6 recensent, par paquetage métier, les principales capacités exposées par l'application (front-end React, API REST /api/*, webhooks /api/webhooks, orchestration /internal/* protégée, files BullMQ/Redis). Le §2.5.2 approfondit un sous-ensemble de cas transverses au format annexe (tableaux 7–11), tandis que le chapitre 3 illustre sept enchaînements critiques par couples diagramme de séquence / classes participantes.

TABLEAU 2 – Inventaire des cas d'utilisation — paquetage Comptes

Cas d'utilisation	Acteurs / surfaces logicielles
Inscription et vérification du compte (OTP et/ou lien e-mail)	Visiteur → Utilisateur; /api/auth; SMTP
Connexion, déconnexion, rotation des jetons (<i>refresh</i>)	Utilisateur, Administrateur; /api/auth; cookies HttpOnly
Mot de passe oublié / réinitialisation sécurisée	Visiteur/Utilisateur; /api/auth; SMTP
Consultation et mise à jour du profil (préférences de risque, catégories)	Utilisateur; /api/users
Suppression de compte (contraintes si investissement actif)	Utilisateur; /api/users

TABLEAU 3 – Inventaire des cas d'utilisation — paquetage Projets

Cas d'utilisation	Acteurs / surfaces logicielles
Exploration, recherche et consultation d'un projet public	Visiteur, Utilisateur; /api/projects/public, /api/projects/search, optionalAuth
Création, modification, archivage d'un brouillon (DRAFT)	Porteur de projet; /api/projects
Soumission à l'analyse IA (AWAITING_AI) et suivi des relances	Porteur; n8n/BullMQ; /internal/run-risk-analysis (routes internes authentifiées)
Validation expert (UNDER_REVIEW → APPROVED/REJECTED)	Expert; POST /api/expert/projects/:id/validate
Publication (APPROVED → ACTIVE)	Administrateur; POST /api/admin/projects/:id/publish
Validation admin de secours (même transition, tests de bout en bout)	Administrateur; POST /api/admin/projects/:id/validate
Suspension / réactivation, relance d'analyse IA	Administrateur; /api/admin/projects
Révoquer une approbation (APPROVED → REJECTED) pour demander des corrections	Administrateur; /api/admin/projects/.../revoke-approval

TABLEAU 4 – Inventaire des cas d'utilisation — paquetage Contributions & finance

Cas d'utilisation	Acteurs / surfaces logicielles
Initier une contribution (avec pourboire optionnel), paiement simulé, <i>webhook</i> signé et idempotent	Investisseur, passerelle; /api/investments, /api/webhooks
Annulation pendant la fenêtre de grâce (CANCELLING/CANCELLED)	Investisseur; /api/investments/.../cancel
Liste « mes investissements », relance de paiement, suivi des transactions	Investisseur; collections investments, transactions
Remboursements (REFUNDED), anti-sur-financement, relances admin d'échecs	Système, administrateur; services métier + files
Saisie chiffrée des coordonnées bancaires (AES-256-GCM)	Porteur de projet; PUT /api/payouts/:id/bank-details
Demande de retrait (<i>payout</i>) et validation administrative	Porteur, administrateur; /api/payouts, /api/admin

TABLEAU 5 – Inventaire des cas d'utilisation — paquetage Échanges & confiance

Cas d'utilisation	Acteurs / surfaces logicielles
Commentaires sur projet public (publication, suppression auteur)	Utilisateur; /api/projects/.../comments
Signalement projet ou commentaire; consultation « mes signalements »	Utilisateur; /api/reports
Traitement des signalements (RESOLVED/DISMISSED)	Administrateur; /api/admin/reports
Notifications intégrées (GET/PUT /api/notifications, pagination ≤ 30 , tri antéchronologique); courriels critiques via BullMQ	Tous rôles; workers SMTP
Supervision des envois SMTP en échec et relance manuelle	Administrateur; /api/admin/failed-notifications; IHM /admin/email-failures

Cas d'utilisation	Acteurs / surfaces logicielles
Recommandations (score déterministe : catégories, riskLevel, progression, récence)	Utilisateur; /api/recommendations
Assistant conversationnel contextualisé (LLM)	Utilisateur; POST /api/projects/:id/chat
Demande de consultation d'expert en finance (mise en attente du paiement) et gestion de l'avis	Investisseur, Expert/Administrateur; /api/expert

TABLEAU 6 – Inventaire des cas d'utilisation — paquetage Administration

Cas d'utilisation	Acteurs / surfaces logicielles
Désactivation / réactivation d'utilisateurs	Administrateur; /api/admin/users
Modération des commentaires (masquage / rétablissement)	Administrateur; /api/admin/comments
Supervision des files d'échec (e-mails, payouts, remboursements, annulations)	Administrateur; collections failed* + actions de relance

2.5.2 Fiches détaillées (format annexe)

Les tableaux 7 à 11 détaillent cinq cas transverses (création de projet, contribution, validation expert / publication admin, recommandations, signalement). Les autres lignes des tableaux 2 à 6 suivent le même canevas en annexe ou en fiches séparées si le jury l'exige.

TABLEAU 7 – Description textuelle du cas « Créer et soumettre un projet »

Cas d'utilisation	Créer et soumettre un projet
Acteur	Utilisateur (porteur de projet)
Précondition	L'utilisateur est authentifié
Postcondition	Projet persisté ; statut DRAFT puis AWAITING_AI après soumission ; analyse IA planifiée
Scénario nominal	1. L'utilisateur renseigne titre, description, catégorie, objectif (<code>fundingGoal</code> $\geq$ 1 000 TND), besoin réel (<code>realBudget</code>) et fenêtre temporelle (<code>startAt</code> , <code>deadline</code>).
	2. Le système enregistre un projet en DRAFT.
	3. L'utilisateur déclenche la soumission : passage en AWAITING_AI, création d'un job (trace <code>aiJobId</code> , compteurs de relance).
	4. Le workflow n8n/LLM produit ou enrichit <code>project.aiAnalysis</code> (rapport + <code>score/riskLevel</code>).
	5. Si aucune règle bloquante n'est déclenchée, le projet passe en UNDER_REVIEW et l'administration est notifiée.
Scénarios alternatifs	1. Auto-rejet budgétaire : heuristique <code>riskHeuristicService</code> — si l'écart entre l'objectif et l'estimation des besoins extraits de la description atteint ou dépasse 30 % (sévérité BLOCK), passage immédiat en REJECTED avec motif structuré (<code>/internal/workflow</code>).
	2. Échec ou indisponibilité du pipeline IA : le projet reste en AWAITING_AI ; relances automatiques (<code>aiAutoRetryCount</code> , <code>aiAnalysisRetries</code> bornées).
Exceptions	1. Validation serveur (champs obligatoires, dates, médias).
	2. <code>fundingGoal</code> ou <code>realBudget</code> hors bornes ($< 1\,000$ TND) : rejet 400.

TABLEAU 8 – Description textuelle du cas « Soutenir un projet »

Cas d'utilisation	Soutenir un projet
Acteur	Utilisateur (investisseur), Passerelle de paiement
Précondition	Utilisateur authentifié ; projet ACTIVE ; montant $\geq$ minimum applicatif (100 TND côté API)
Postcondition	Investissement INITIATED puis SUCCESS si <i>webhook</i> SUCCEDED ; incrément atomique de currentFunding
Scénario nominal	1. L'utilisateur saisit un montant $\geq$ 100 TND et sélectionne ou saisit éventuellement un pourboire pour la plateforme.
	2. Le système crée un investissement (INITIATED) en incluant le pourboire optionnel, et une transaction (PENDING , couple provider+providerPaymentId unique).
	3. Le système renvoie une URL de paiement simulée.
	4. La passerelle appelle le <i>webhook</i> signé.
	5. Le serveur applicatif vérifie signature et idempotence.
	6. Transaction SUCCEDED , investissement SUCCESS , mise à jour du total collecté (garde-fou anti-sur-financement).
Scénarios alternatifs	1. Paiement refusé : transaction FAILED / CANCELLED , investissement FAILED .
	2. Sur-financement : statuts REFUNDED , remontées admin pour relance éventuelle.
	3. Annulation utilisateur dans la fenêtre de grâce (CANCELLING $\rightarrow$ CANCELLED).
	4. Demande d'expert : Si le montant $\geq$ 30% de l'objectif, l'investisseur sollicite un expert ; l'investissement est mis en attente (PENDING_CONSULTATION) le temps de la consultation.
Exceptions	1. Projet indisponible ou investisseur administrateur (règle requireNotAdmin).
	2. Montant hors bornes (<100 TND).

TABLEAU 9 – Description textuelle du cas « Valider ou rejeter (expert) et publier (admin) »

Cas d'utilisation	Valider ou rejeter un projet (expert) et le publier (administrateur)
Acteur	Expert en finance (validation); Administrateur (publication)
Précondition	Projet en UNDER_REVIEW; analyse IA terminée (aiStatus=COMPLETED, aiAnalysis présent)
Postcondition	Après expert : APPROVED ou REJECTED; après admin : ACTIVE pour les projets APPROVED; journalisation et notifications
Scénario nominal	1. L'expert consulte la file des projets en revue (/expert/projects).
	2. Il analyse le rapport IA (aiAnalysis).
	3. Il émet APPROVED ou REJECTED via POST /api/expert/projects/:id/validate (expertService.validateProjectAnalysis).
	4. Audit EXPERT_VALIDATE_PROJECT ou EXPERT_REJECT_PROJECT; notifications au créateur.
	5. L'administrateur publie un projet APPROVED : POST /api/admin/projects/:id/publish → ACTIVE.
	6. Le projet devient visible pour les investissements (ACTIVE).
Scénarios alternatifs	1. REJECTED : le créateur corrige et resoumet (DRAFT → cycle IA).
	2. Révocation : /api/admin/projects/:id/revoke-approval (APPROVED → REJECTED).
	3. Voie admin (tests de bout en bout) : POST /api/admin/projects/:id/validate (adminProjectService.validateProject) applique la même transition depuis UNDER_REVIEW sans passer par l'IHM expert.
Exceptions	1. Analyse IA absente ou expirée; projet hors UNDER_REVIEW.
	2. Publication sans statut APPROVED; droits insuffisants.

TABLEAU 10 – Description textuelle du cas « Voir les projets recommandés »

Cas d'utilisation	Voir les projets recommandés
Acteur	Utilisateur (investisseur)
Précondition	Utilisateur authentifié
Postcondition	Liste de projets ACTIVE non archivés, triée par score interne
Scénario nominal	1. L'utilisateur ouvre l'écran « recommandations ».
	2. Le serveur charge <code>profile.preferredCategories</code> et <code>profile.riskPreference</code> .
	3. <code>recommendationService</code> calcule un score déterministe par projet (adéquation catégorie, cohérence avec <code>aiAnalysis.riskLevel</code> , progression, récence).
	4. Les projets sont triés par score décroissant (limite paramétrable, défaut 8).
Scénarios alternatifs	1. Aucun projet ACTIVE : liste vide + message.
	2. Risque ou catégorie manquants : pénalités neutres dans le score (implémentation actuelle).
Exceptions	1. Session expirée (401).

TABLEAU 11 – Description textuelle du cas « Signaler un contenu »

Cas d'utilisation	Signaler un contenu (projet ou commentaire)
Acteur	Utilisateur
Précondition	Utilisateur authentifié ; cible existante
Postcondition	Rapport stocké avec statut PENDING (énumération FRAUD, INAPPROPRIATE_CONTENT, SPAM, OTHER)
Scénario nominal	1. L'utilisateur ouvre le formulaire de signalement.
	2. Il choisit le type, rédige la description, lie éventuellement un commentaire.
	3. Le serveur enregistre le signalement et notifie l'administration.
Scénarios alternatifs	1. Consultation de l'historique « mes signalements ».
	2. Administrateur : résolution (RESOLVED) ou classement sans suite (DISMISSED).
Exceptions	1. Champs requis manquants.
	2. Signalement redondant (contraintes d'unicité <code>reports</code> côté base).

2.6 Conclusion

Ce chapitre a précisé la délimitation du système en tant que « boîte noire », recensé **sept** acteurs pertinents dans le contexte (Visiteur, Utilisateur authentifié, Expert en finance, Administrateur, passerelle de paiement de démonstration, service d'analyse de risque orchestré, service SMTP) et ordonné les besoins en **cinq** paquetages métier (Comptes, Projets, Contributions & finance, Échanges & confiance, Administration). Les tableaux 2 à 6 consolident l'inventaire exhaustif des cas d'utilisation, en stricte continuité avec l'implémentation ; les fiches 7–11 en offrent, pour des cas transverses majeurs, une illustration au format annexe. Le chapitre 3 poursuit la démarche par la traduction de ces besoins en conception UML et par la description du **modèle de données MongoDB**.

Chapitre 3

Analyse et conception

3.1 Introduction

Ce chapitre transpose les besoins recueillis au chapitre 2 en une conception UML structurée. Il expose d'abord la réalisation conceptuelle des cas d'utilisation les plus structurants au travers de diagrammes de séquence qui décrivent l'interaction entre acteurs, IHM, contrôleur et classes métier. Chaque séquence est complétée par un diagramme de classes participantes qui en définit la structure statique. Sont ensuite présentés le modèle du domaine (diagramme de classes global), l'architecture du futur logiciel, puis le modèle de données MongoDB. Une conclusion récapitule les choix de conception et annonce l'implémentation (chapitre 4).

3.2 Réalisation conceptuelle des cas d'utilisation

Les diagrammes de séquence suivants décrivent le flux d'interactions entre les acteurs et les objets du système pour un ensemble de cas d'utilisation représentatifs identifiés au chapitre 2. Chaque séquence est complétée par un diagramme de classes participantes afin d'assurer la cohérence entre la vue dynamique (interactions) et la vue statique (classes mobilisées). Ces diagrammes sont alignés sur les règles métier et sur l'implémentation développée.

Choix des scénarisations UML. Le référentiel académique peut exiger une interaction par cas d'utilisation ; compte tenu du nombre de cas listés aux tableaux 2–6, cette section présente en priorité les enchaînements les plus structurants (contributions et financement, cycle de projet, conformité administrative, retrait). Elle illustre la démarche de conception et garantit la mise en cohérence avec le modèle du domaine (figure 17) ; les autres comportements reposent sur les mêmes classes et contrats d'API documentés au chapitre 2.

Cas retenus. Annulation de contribution, création et soumission d'un projet, investissement (paiement démo + *webhook*), validation ou rejet par l'expert puis publication par l'administrateur, fourniture de coordonnées bancaires, approbation d'un retrait par l'administrateur, suspension ou réactivation d'un projet.

Structure de chaque sous-section : diagramme de séquence puis son commentaire ; diagramme de classes participantes puis son commentaire.

3.2.1 Annuler une contribution

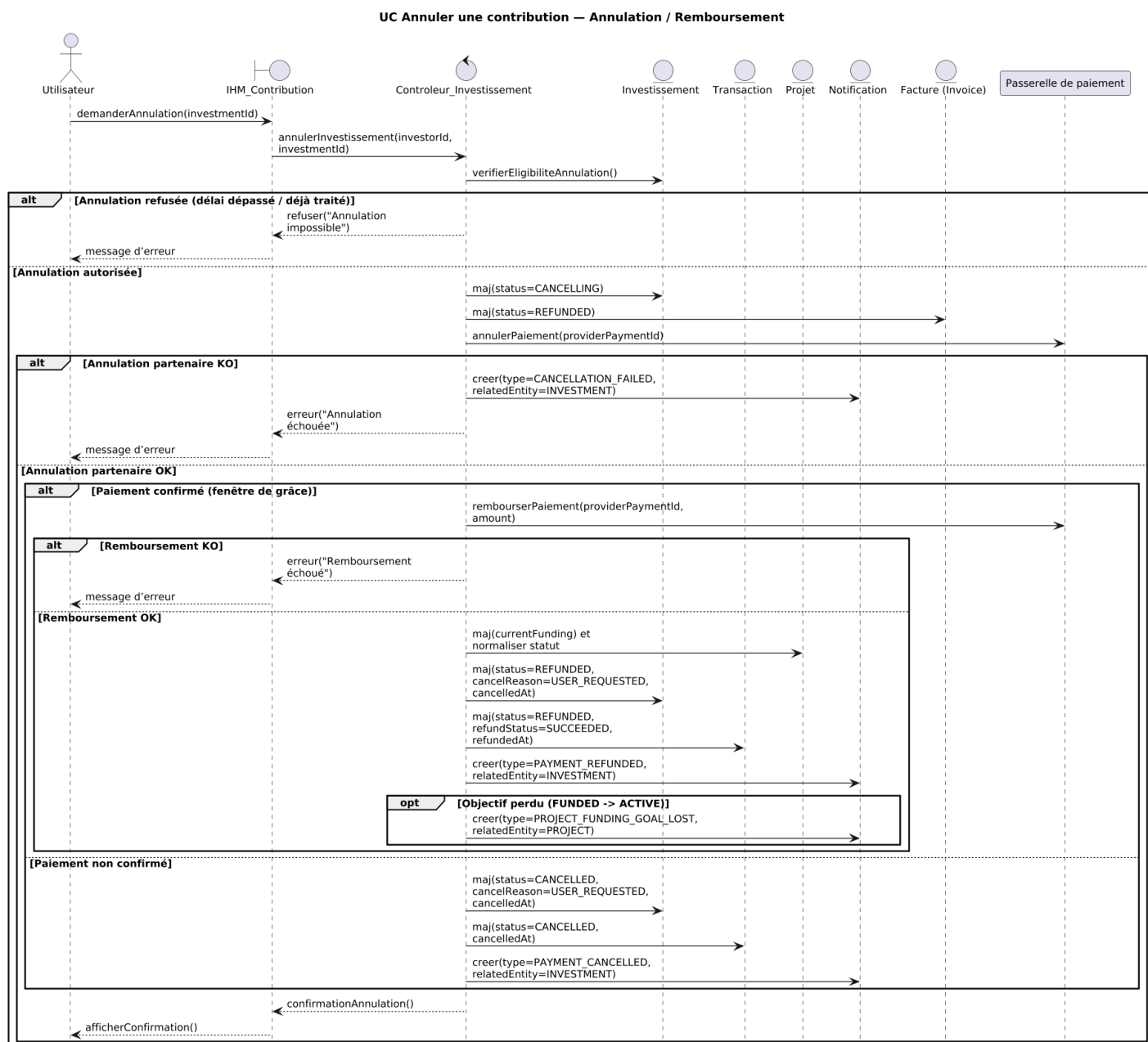


FIGURE 3 – Diagramme de séquence – Annuler une contribution

Diagramme de séquence (figure 3) : il illustre l'ordre chronologique des interactions : l'investisseur initie l'annulation via l'interface, le contrôleur vérifie les règles métier (délai de grâce, état de l'investissement et de la transaction), sollicite la passerelle de paiement de démonstration si nécessaire, met à jour l'investissement et la transaction, puis notifie l'utilisateur du succès ou de l'échec.

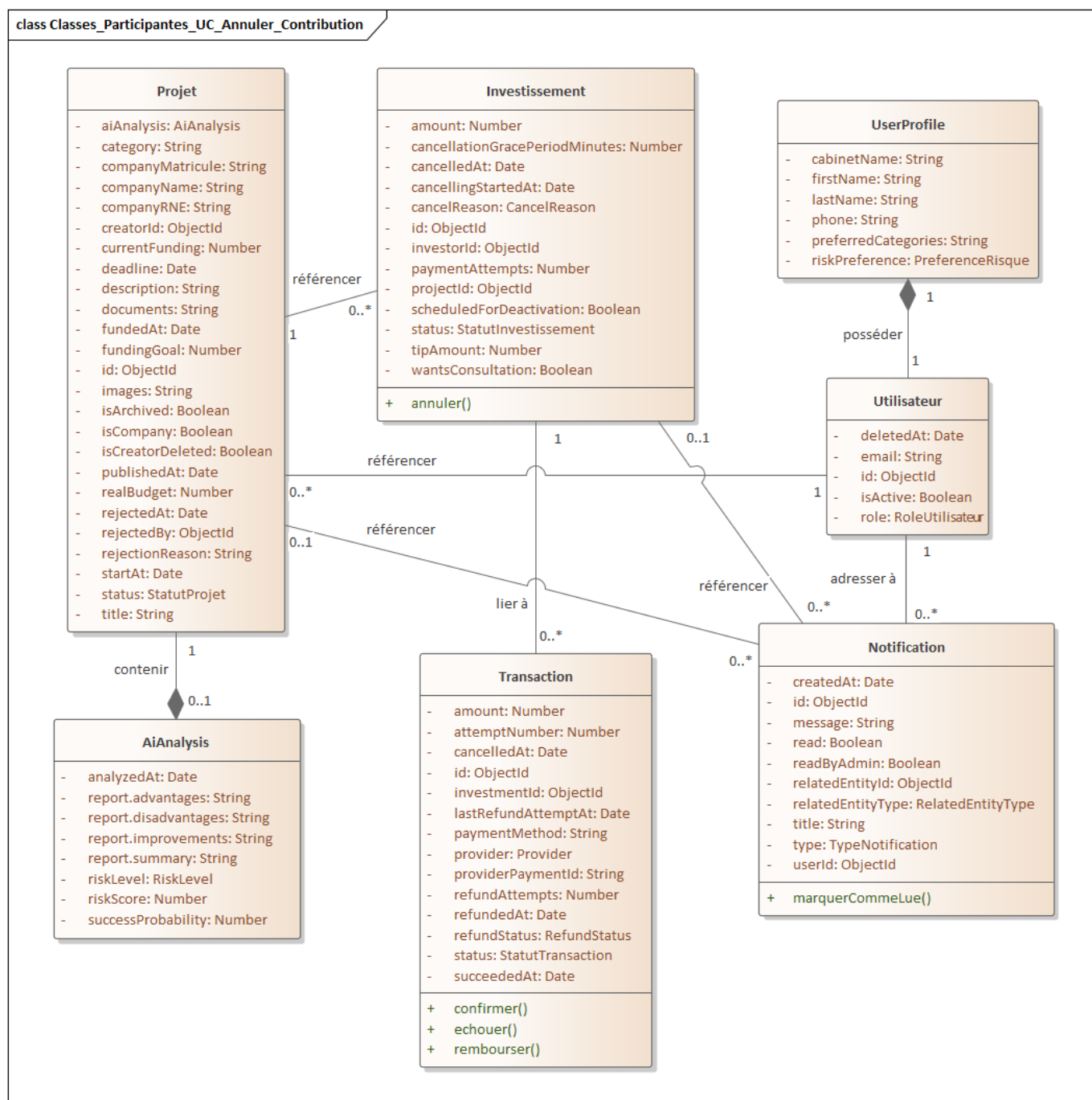


FIGURE 4 – Diagramme de classes participantes – Annuler une contribution

Diagramme de classes participantes (figure 4) : il fixe la vue statique associée à la séquence ci-dessus ; il recense les entités métier mobilisées (Utilisateur, Projet, Investissement, Transaction, Notification) ainsi que la passerelle externe, avec les attributs et énumérations utiles au cas (statuts d'investissement, de transaction, motifs d'annulation, etc.).

3.2.2 Créer et soumettre un projet

UC Créer et soumettre un projet – Brouillon + analyse IA

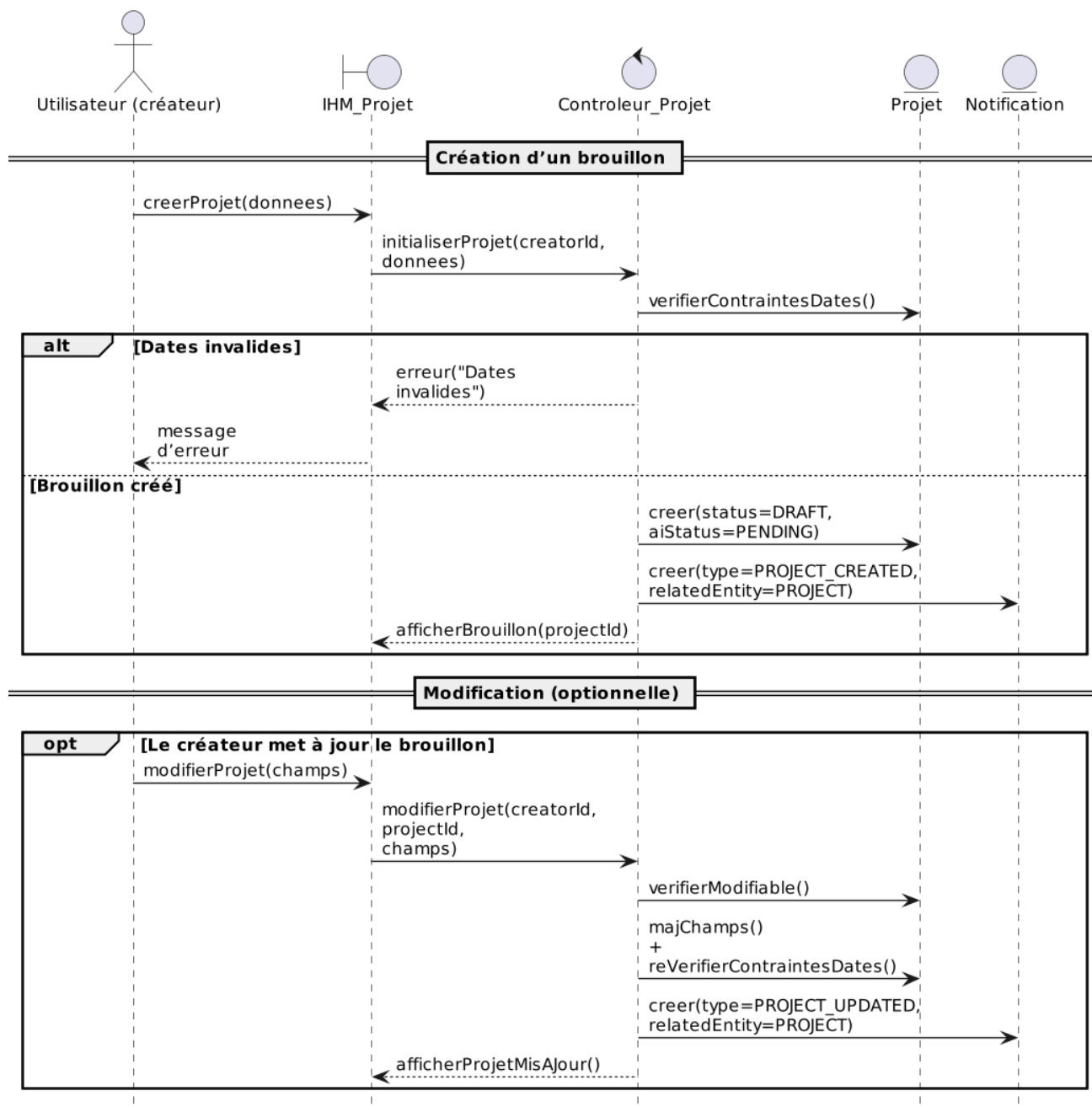


FIGURE 5 – Diagramme de séquence – Créer et soumettre un projet

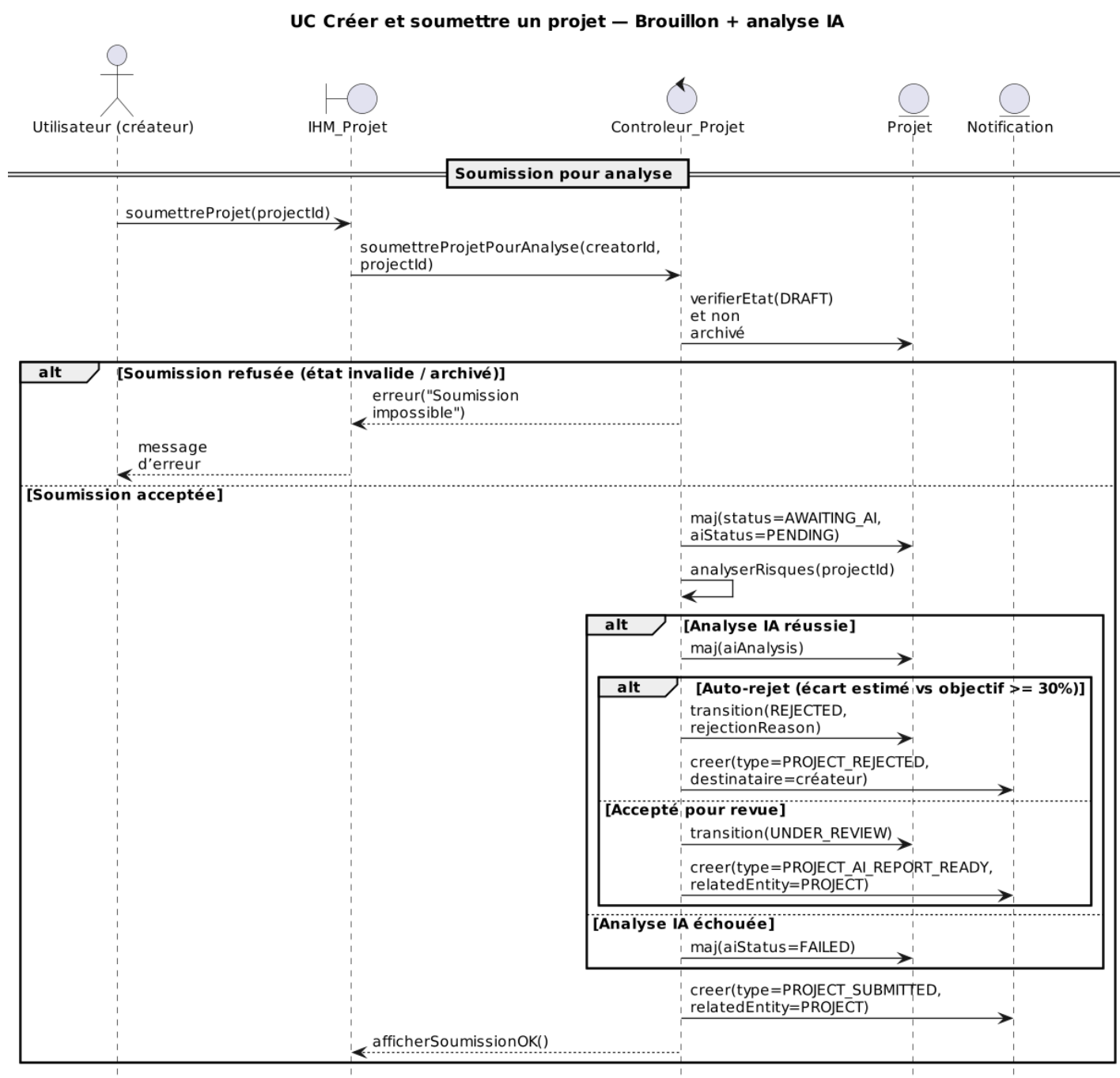


Figure 5 — Diagramme de séquence – Créer et soumettre un projet — suite

Diagramme de séquence (figure 5) : il détaille le scénario de création puis de soumission : le porteur renseigne le projet via l’IHM, le contrôleur persiste le brouillon, déclenche l’analyse de risque (orchestration externe), met à jour les états du projet (DRAFT, AWAITING_AI, UNDER_REVIEW, etc.) et notifie le créateur des transitions importantes.

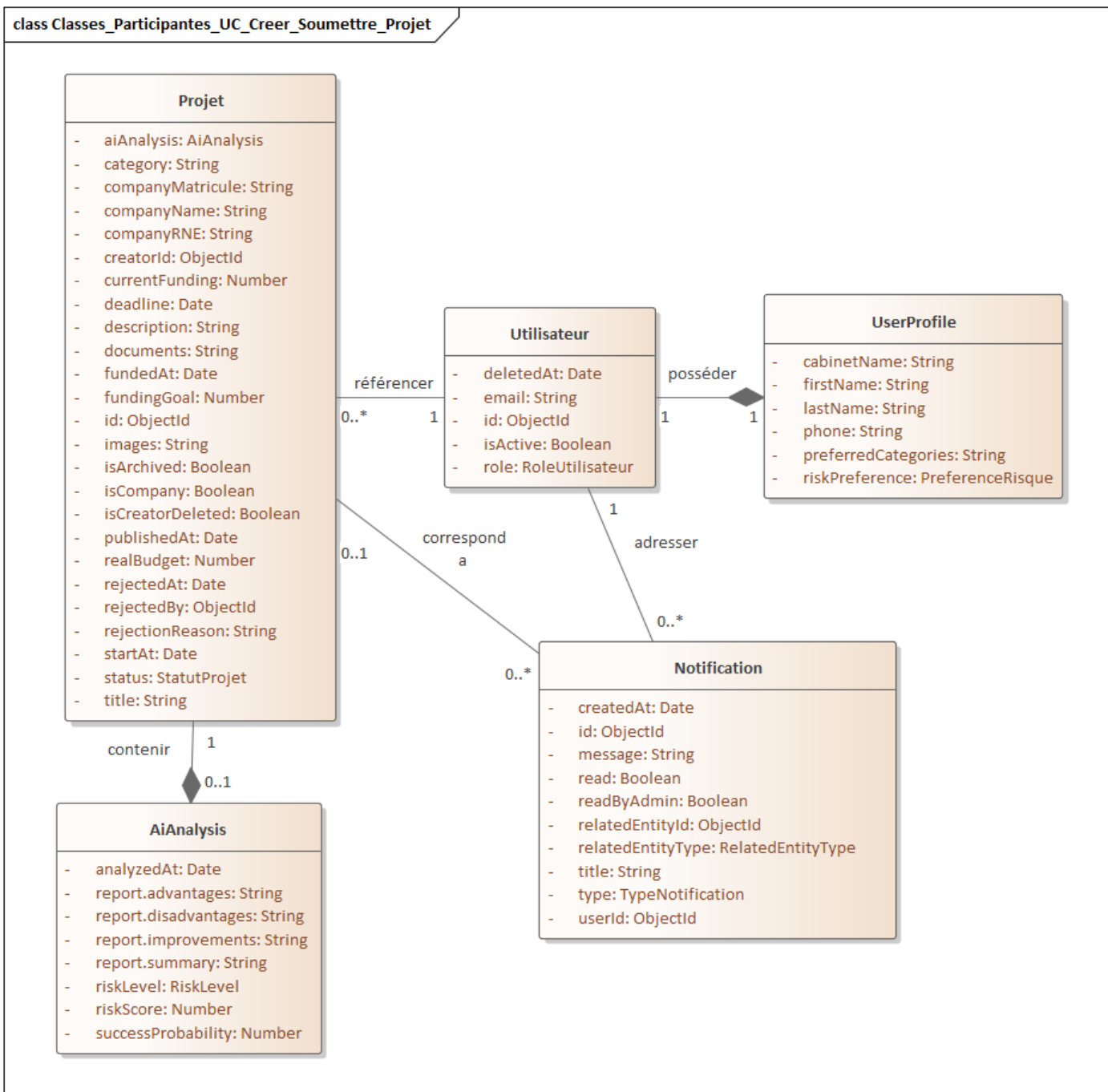


FIGURE 6 – Diagramme de classes participantes – Créer et soumettre un projet

Diagramme de classes participantes (figure 6) : il montre les classes du domaine concernées par ce flux (Utilisateur, profil, Projet, sous-objet d’analyse IA, Notification) et leurs attributs alignés sur le schéma applicatif; il fait le lien entre les messages de la séquence et la structure de données persistée en base.

3.2.3 Investir dans un projet (paiement + webhook)

UC Investir — Demande et confirmation d'investissement

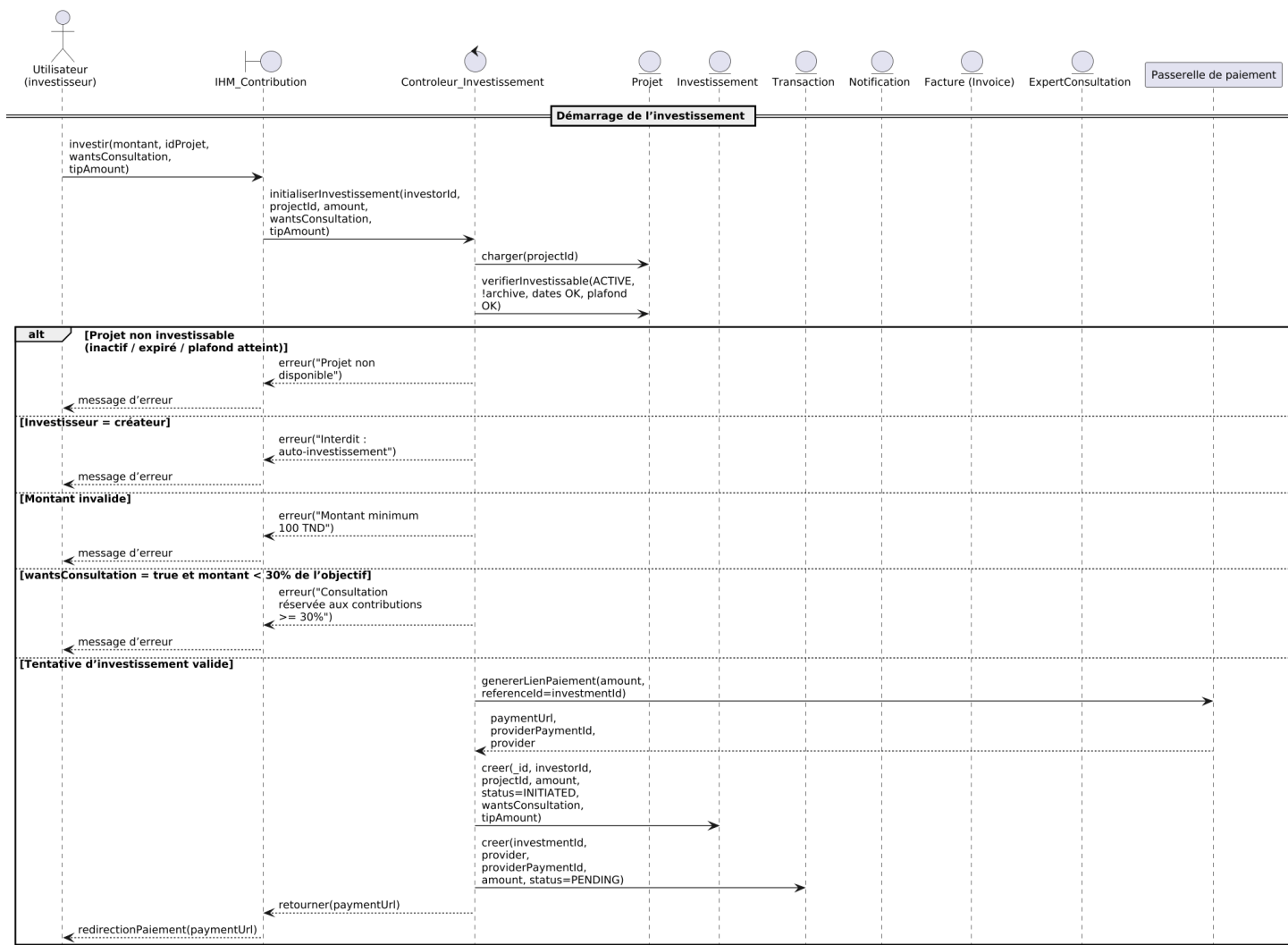


FIGURE 7 – Diagramme de séquence – Investir (paiement + webhook)

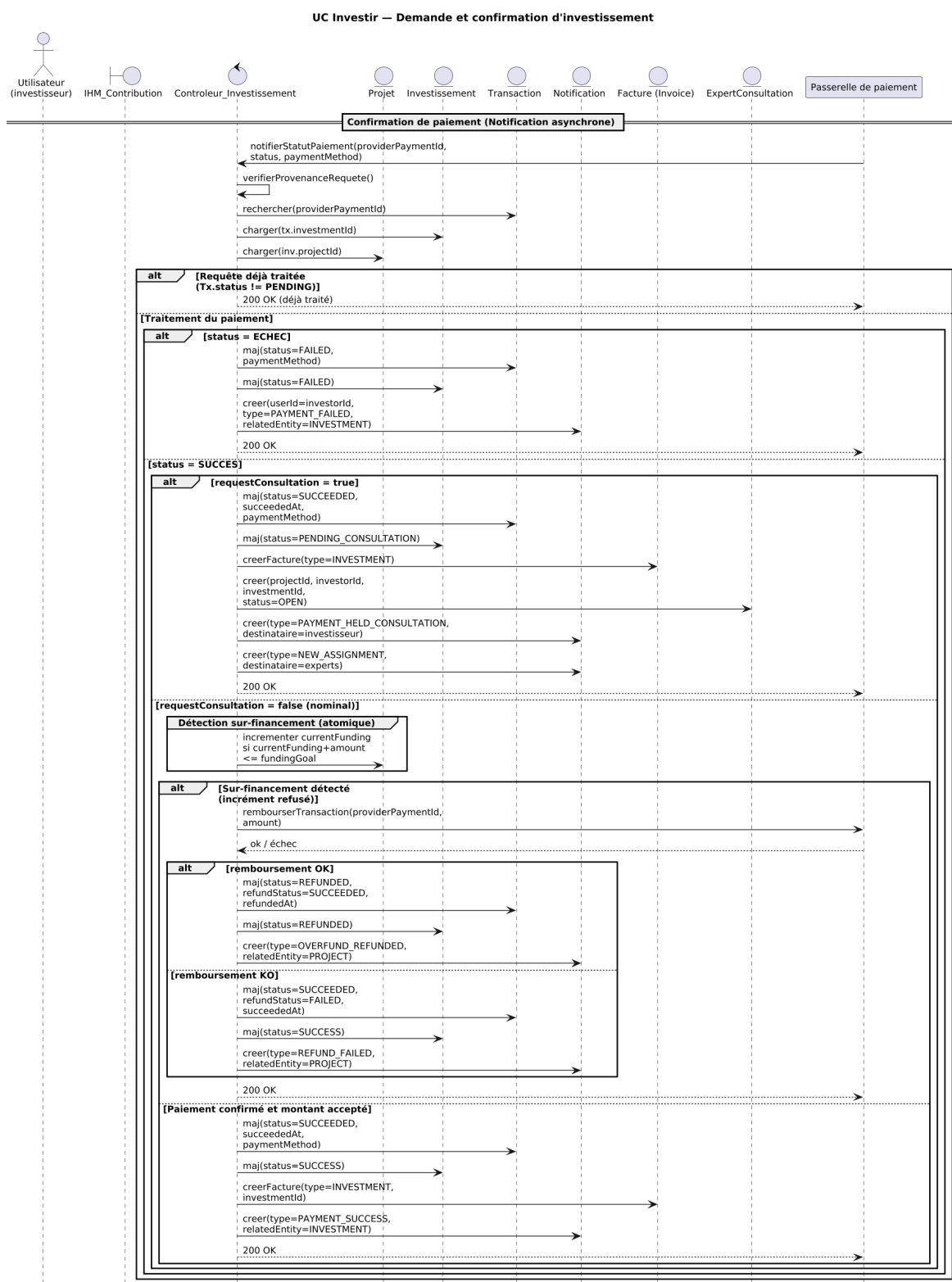


Figure 7 — Diagramme de séquence – Investir (paiement démo + *webhook*) — suite

Diagramme de séquence (figure 7) : il met en évidence le caractère asynchrone du paiement démo : l'investisseur initie le flux côté application en incluant éventuellement un pourboire, la passerelle confirme ou refuse le paiement, puis le *webhook* rappelle le serveur applicatif pour finaliser la transaction, promouvoir l'investissement au statut réussi et déclencher les notifications associées.

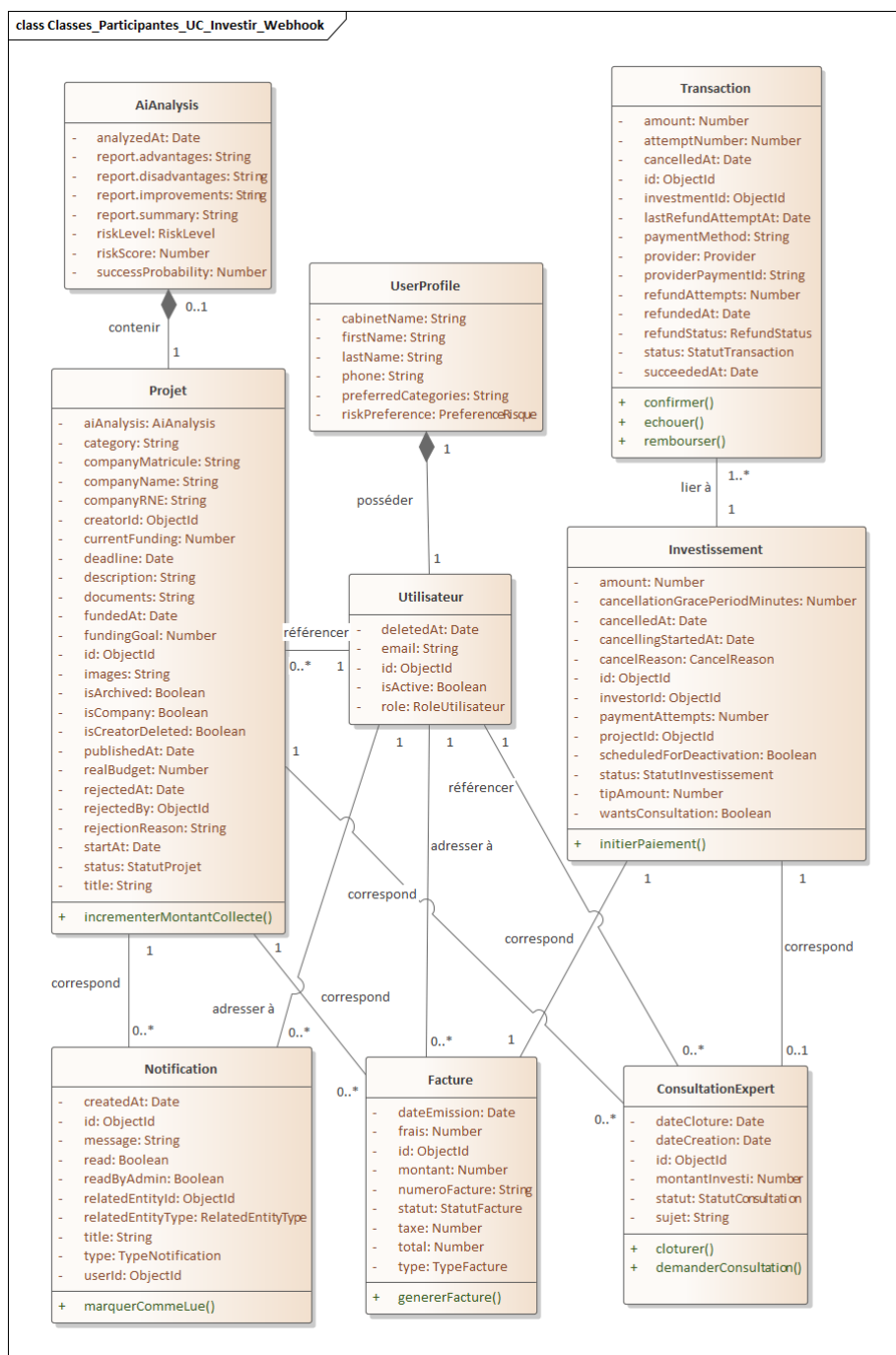


FIGURE 8 – Diagramme de classes participantes – Investir dans un projet (paiement démo + webhook)

Diagramme de classes participantes (figure 8) : il explicite les entités et attributs impliqués (Utilisateur investisseur, Projet, Investissement avec son champ `tipAmount`, Transaction, Notification, passerelle) et permet de relier chaque message de la séquence à une structure de données concrète (montants, statuts, identifiants fournisseur, liens vers le projet).

3.2.4 Valider un projet (Expert & Admin)

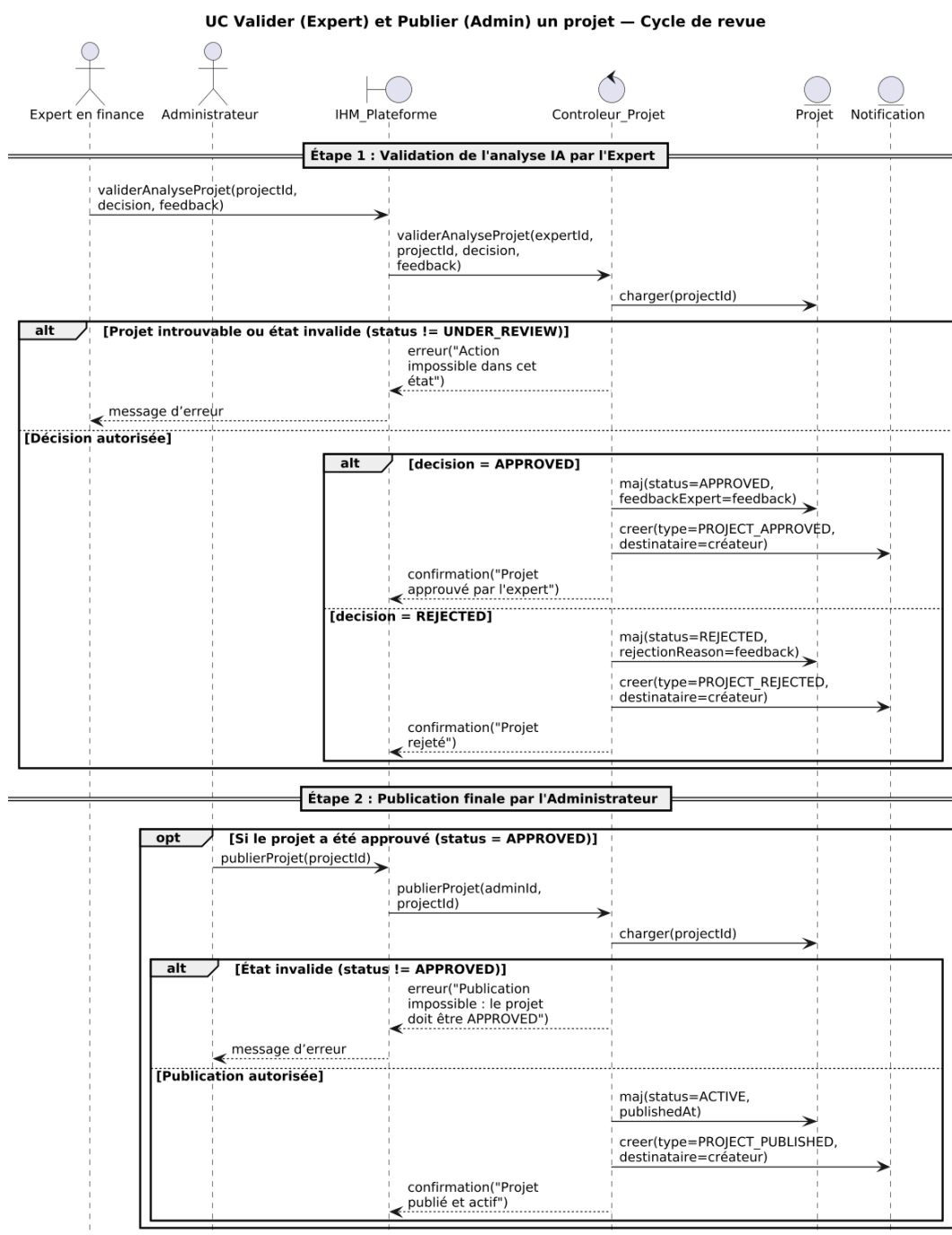


FIGURE 9 – Diagramme de séquence – Valider ou rejeter (expert) et publier (administrateur)

Diagramme de séquence (figure 9) : il décrit le parcours de validation : chargement d'un projet en UNDER_REVIEW, examen par l'expert en finance de la synthèse d'analyse IA, enregis-

trement du verdict (APPROVED ou REJECTED avec motif) via EXPERT_VALIDATE_PROJECT ou EXPERT_REJECT_PROJECT, puis publication par l'administrateur des projets APPROVED vers l'état ACTIVE, avec notifications au créateur.

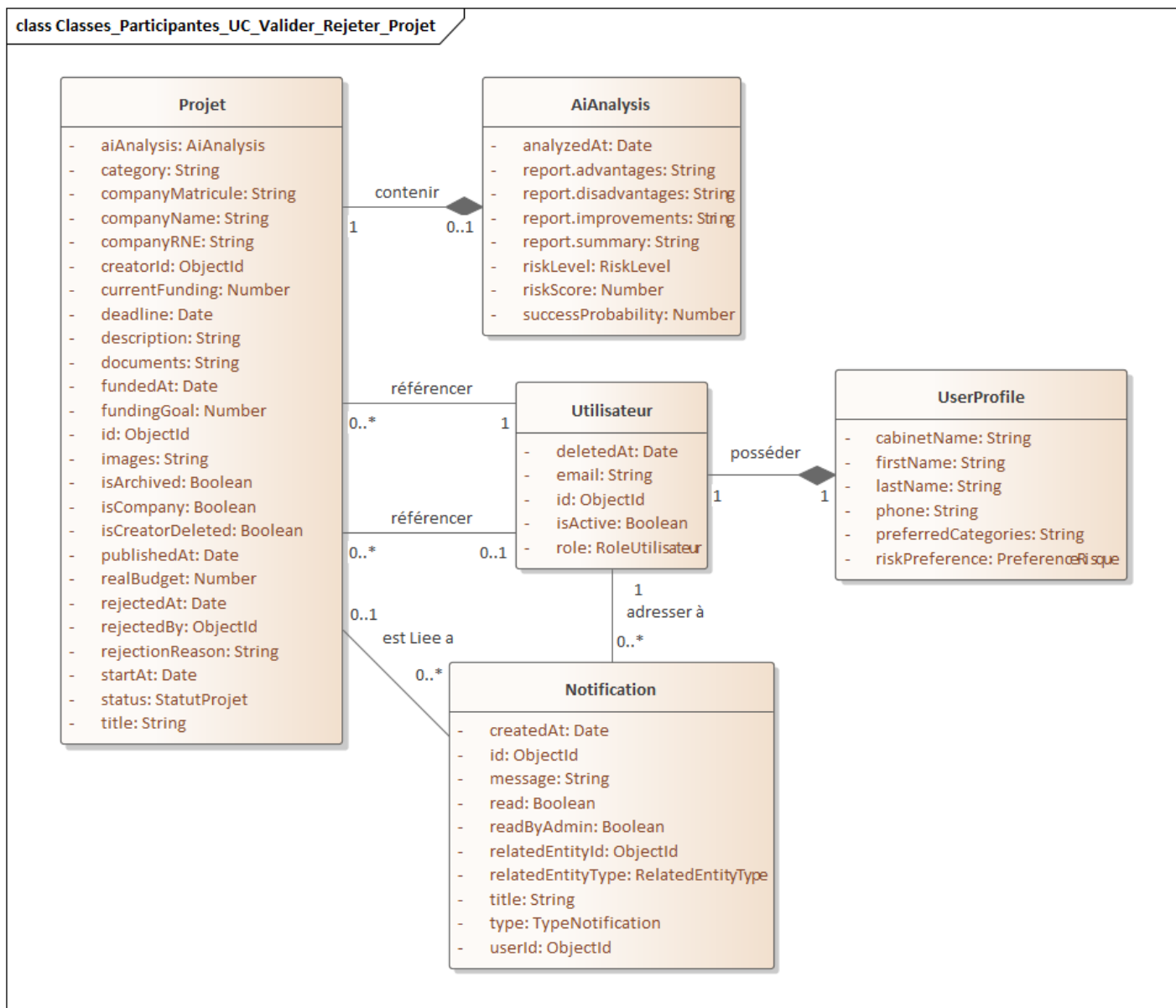


FIGURE 10 – Diagramme de classes participantes – Valider ou rejeter (expert) et publier (administrateur)

Diagramme de classes participantes (figure 10) : il complète la vue en montrant les classes métier mobilisées (**Utilisateur**, **Projet**, bloc d'analyse IA, **Notification**) pour la validation par l'expert et la publication par l'administrateur ; il matérialise statut du projet, motif de rejet et notifications associées.

3.2.5 Fournir les coordonnées bancaires

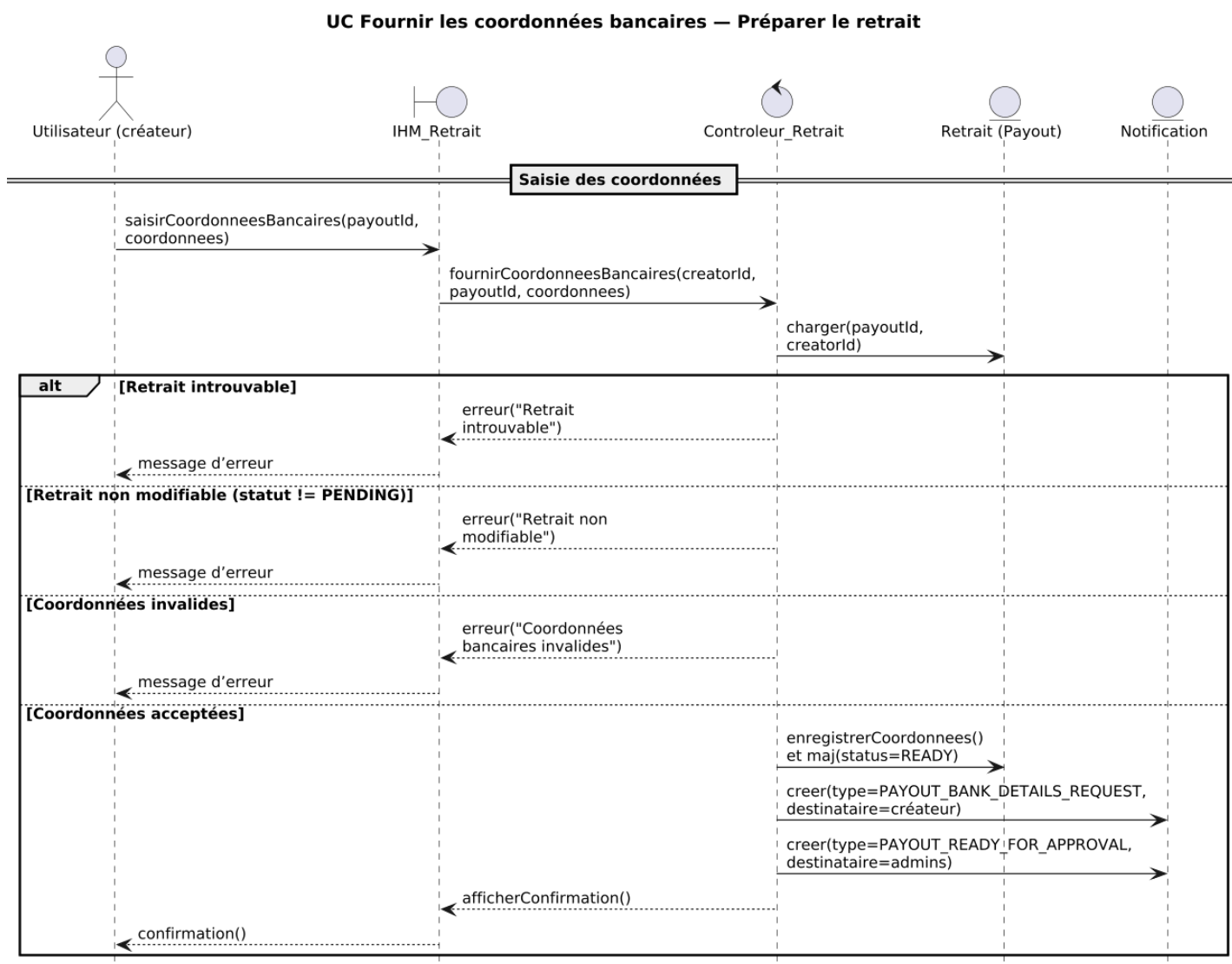


FIGURE 11 – Diagramme de séquence – Fournir les coordonnées bancaires

Diagramme de séquence (figure 11) : il montre comment le créateur fournit les informations bancaires encadrées par l'application, comment le service métier met à jour l'objet Payout (coordonnées, date de saisie, passage à l'état prêt pour validation) et comment des notifications sont émises vers le créateur et/ou l'administration.

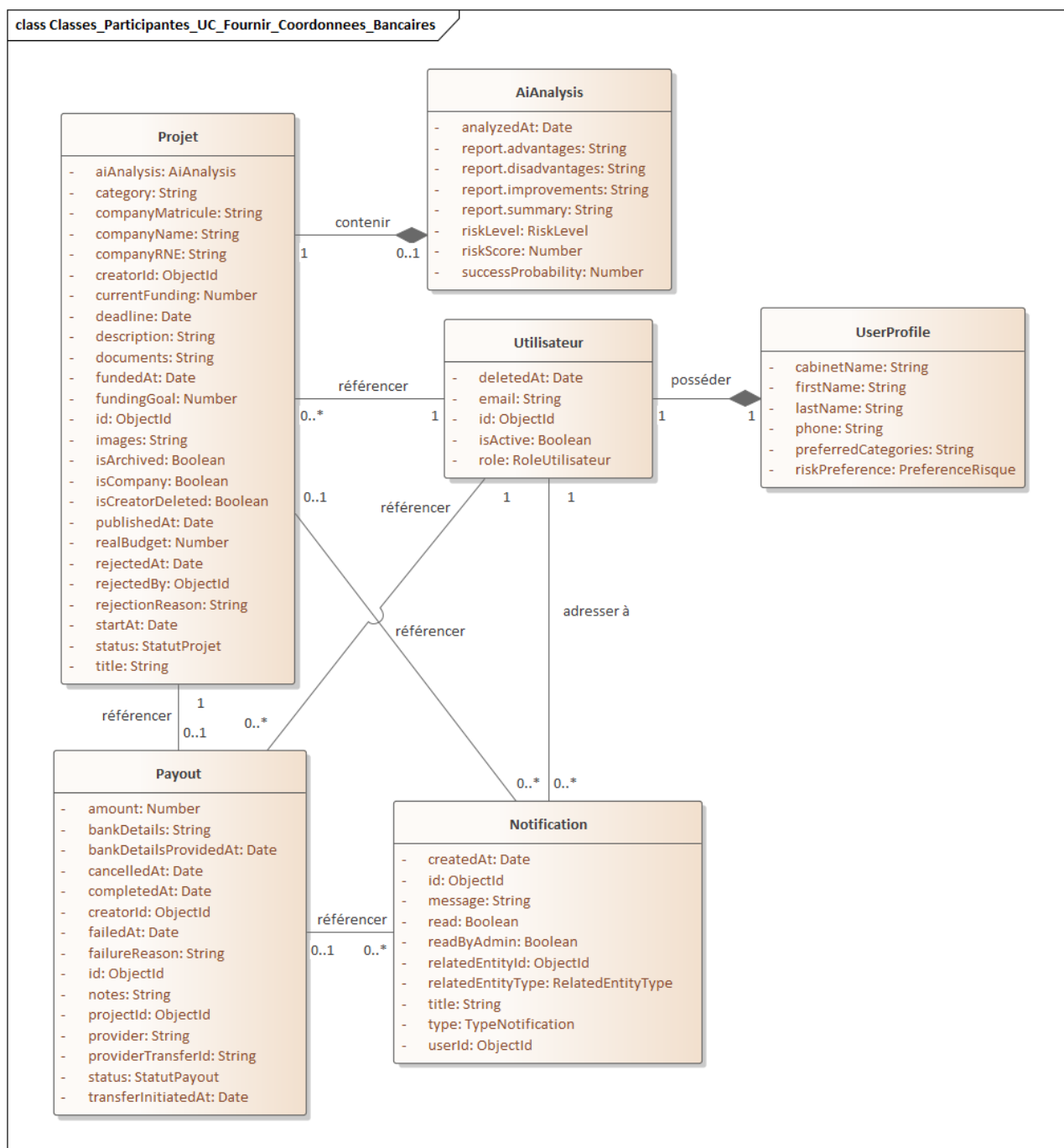


FIGURE 12 – Diagramme de classes participantes – Fournir les coordonnées bancaires

Diagramme de classes participantes (figure 12) : il présente les entités en jeu (Utilisateur, Projet, Payout, Notification) avec les champs pertinents pour ce cas et relie visuellement les messages de la séquence aux agrégats persistés en MongoDB.

3.2.6 Approuver un retrait (Admin)

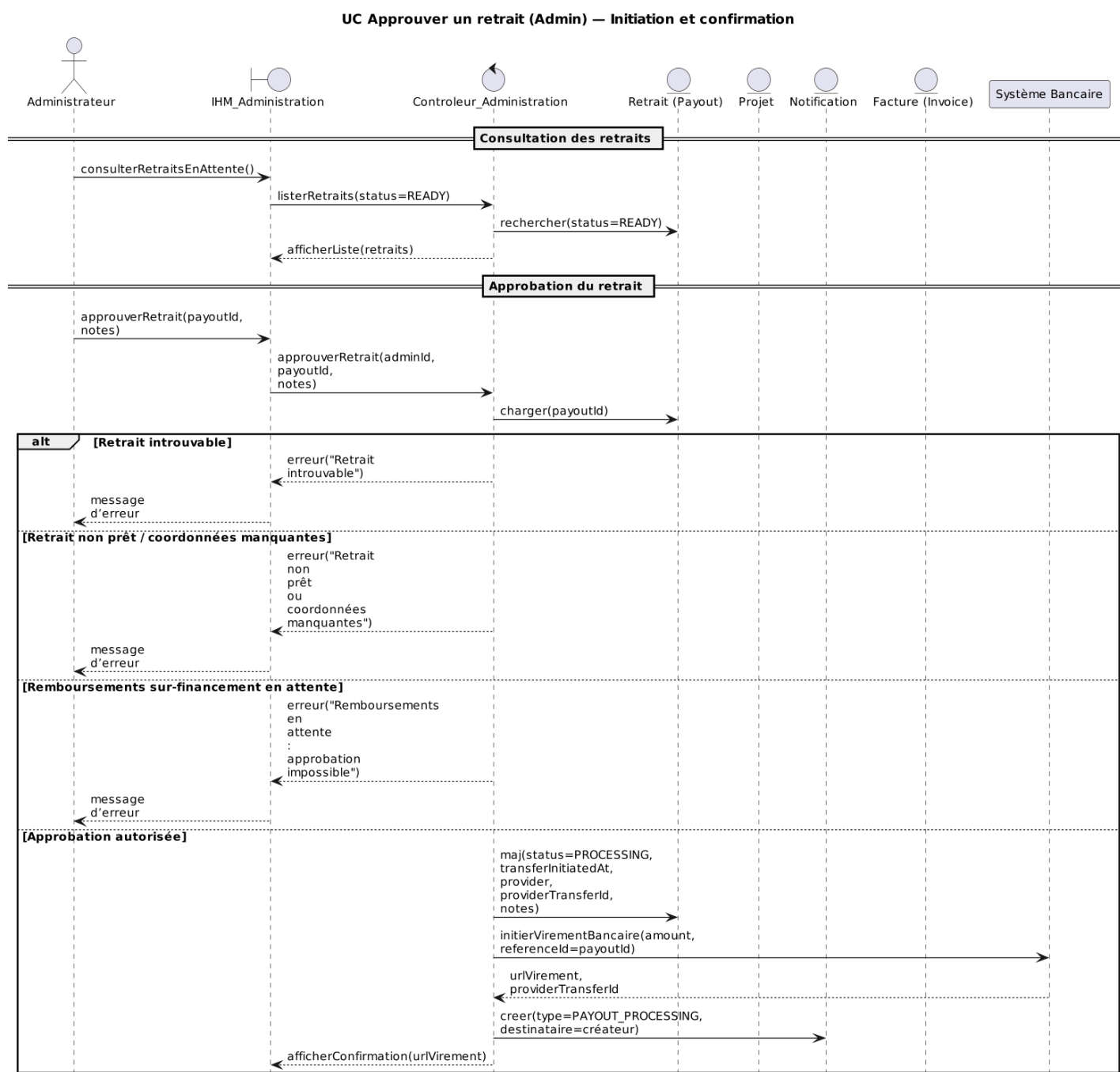


FIGURE 13 – Diagramme de séquence – Approuver un retrait (Admin)

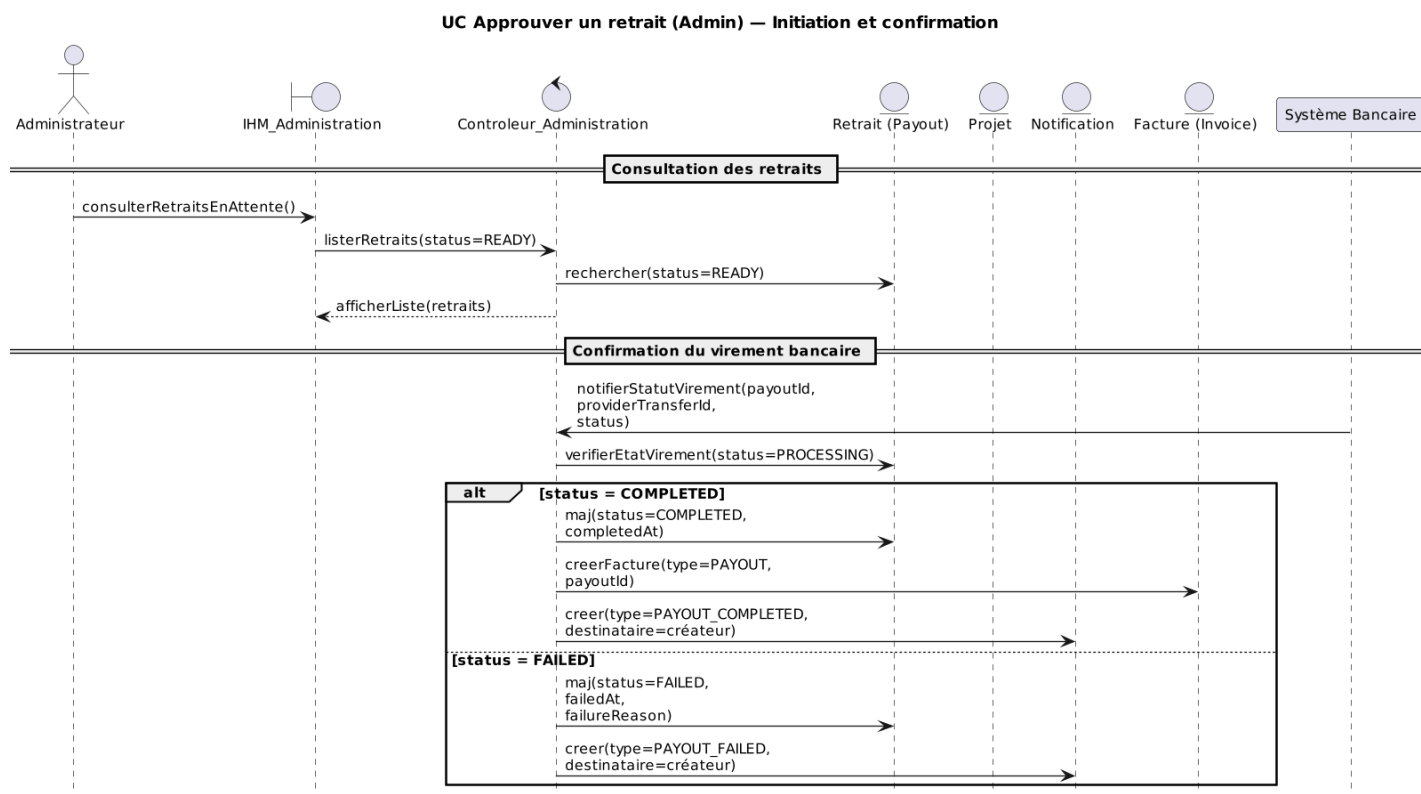


Figure 13 — Diagramme de séquence – Approuver un retrait (Admin) — suite

Diagramme de séquence (figure 13) : il ordonne les actions d’approbation : contrôle des préconditions (retrait READY, coordonnées présentes), appel au prestataire de virement de démonstration, mise à jour des statuts temporels du Payout (PROCESSING, COMPLETED ou échec), création de la facture numérique (Invoice, exportable en PDF) et notification au créateur.

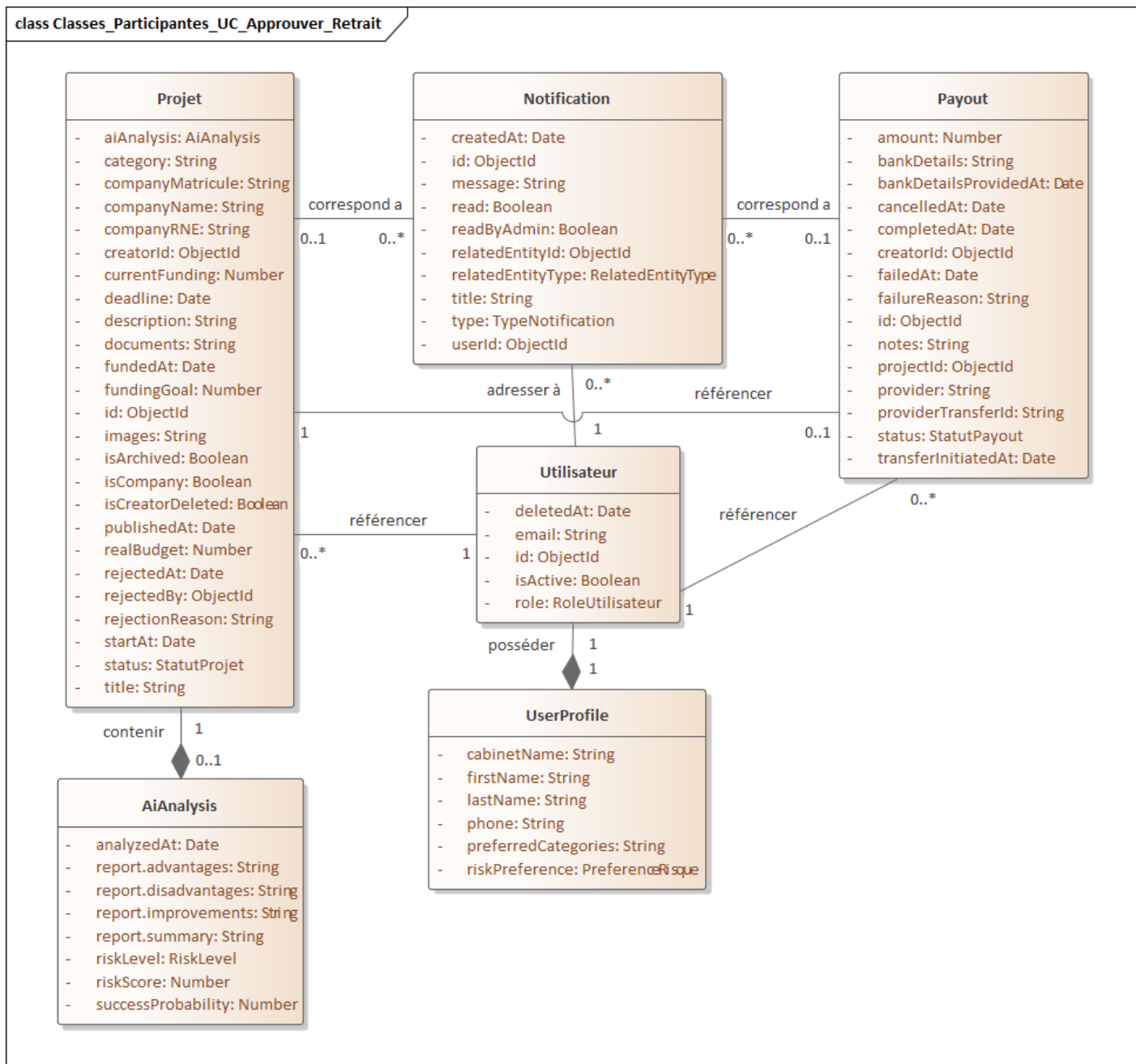


FIGURE 14 – Diagramme de classes participantes – Approuver un retrait (Admin)

Diagramme de classes participantes (figure 14) : il identifie les classes du domaine et l'acteur externe (Projet, Payout, Invoice, Utilisateur, Notification, prestataire) et précise quelles données sont lues ou écrites à chaque étape visible sur la séquence.

3.2.7 Mise à jour de l'état d'un projet (Admin)

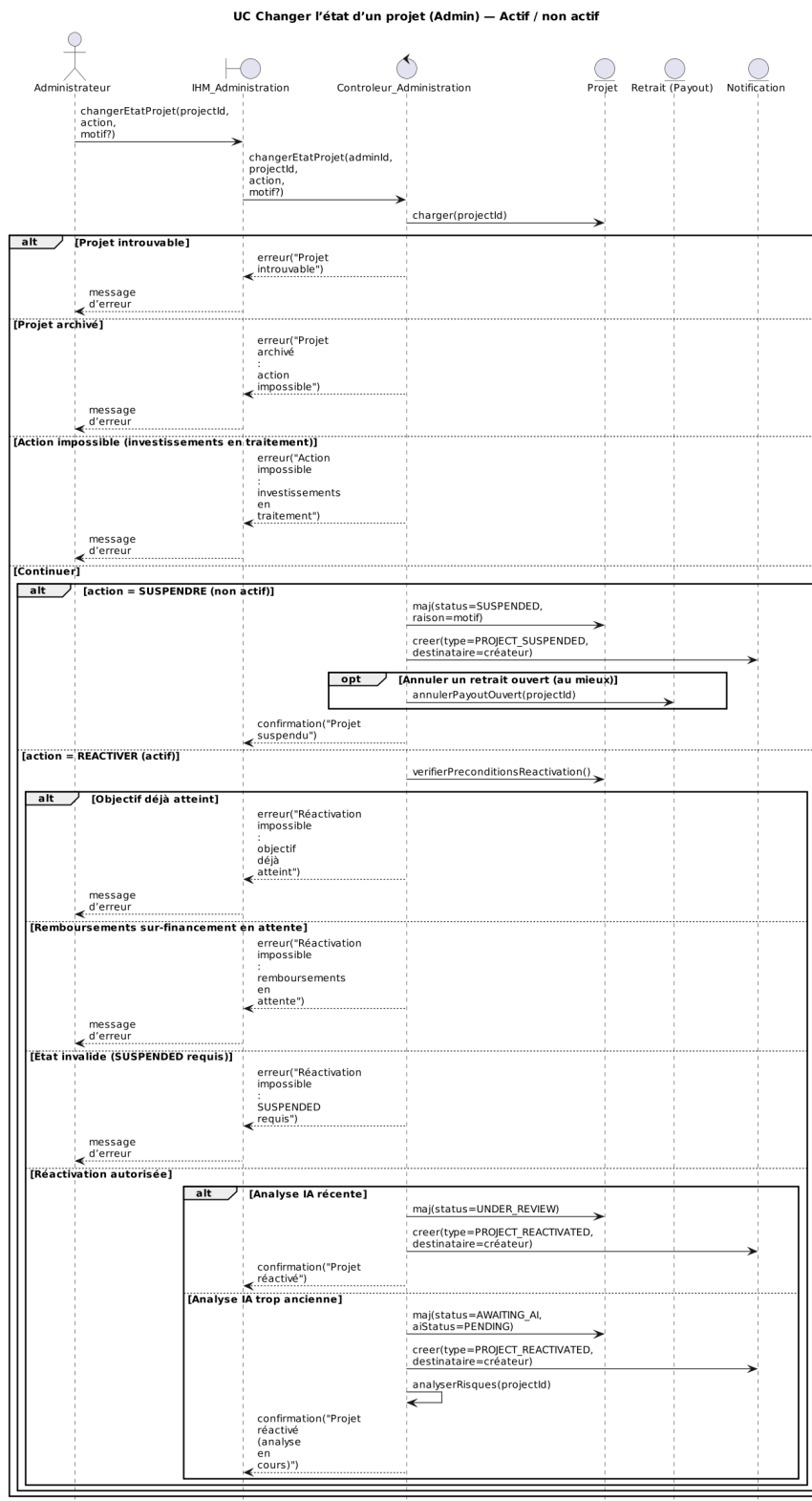


FIGURE 15 – Diagramme de séquence – Mise à jour de l'état d'un projet (suspension / réactivation)

Diagramme de séquence (figure 15) : il décrit la suspension ou la réactivation d'un projet par l'administrateur : sélection du projet, vérification des transitions autorisées depuis l'état courant, persistance du nouveau statut (SUSPENDED ou retour vers un état actif) et envoi de notifications explicites (PROJECT_SUSPENDED, PROJECT_REACTIVATED, etc.).

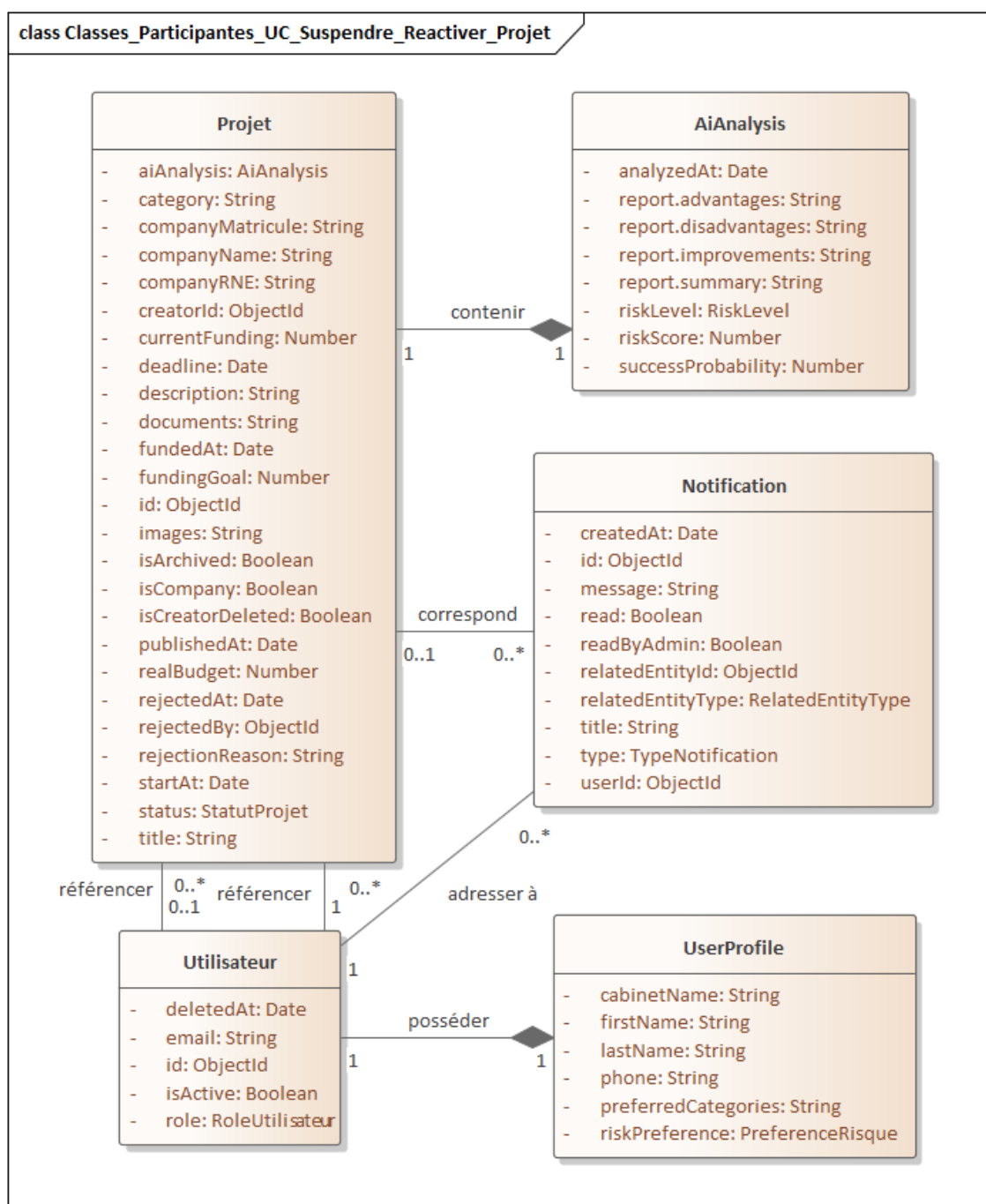


FIGURE 16 – Diagramme de classes participantes – Suspendre ou réactiver un projet (Admin)

Diagramme de classes participantes (figure 16) : il synthétise les entités impactées (Utilisateur, Projet, analyse IA éventuelle, Notification) et clarifie quels attributs de projet et de notification portent la trace métier visible dans la séquence.

3.3 Modèle du domaine

Lecture du diagramme. Le diagramme de classes présenté à la figure 17 matérialise le modèle du domaine de la plateforme : il explicite les entités métier centrales (Utilisateur, Projet, Investissement, Transaction, Retrait, Facture, Consultation d'expert, Notification, Commentaire, Signalement) avec leurs attributs, leurs opérations et leurs relations assorties de multiplicités. Les énumérations portant sur le rôle, les statuts ou les types de notification rendent lisibles les cycles de vie et les règles de gestion correspondantes. Trois invariants y sont rappelés de manière synthétique : (i) unicité de l'adresse électronique ; (ii) plafonnement de la collecte par l'objectif de financement ; (iii) limitation à un retrait par projet. Ce modèle fait office de référence pour les développements ultérieurs relatifs à l'architecture et au modèle de données MongoDB.

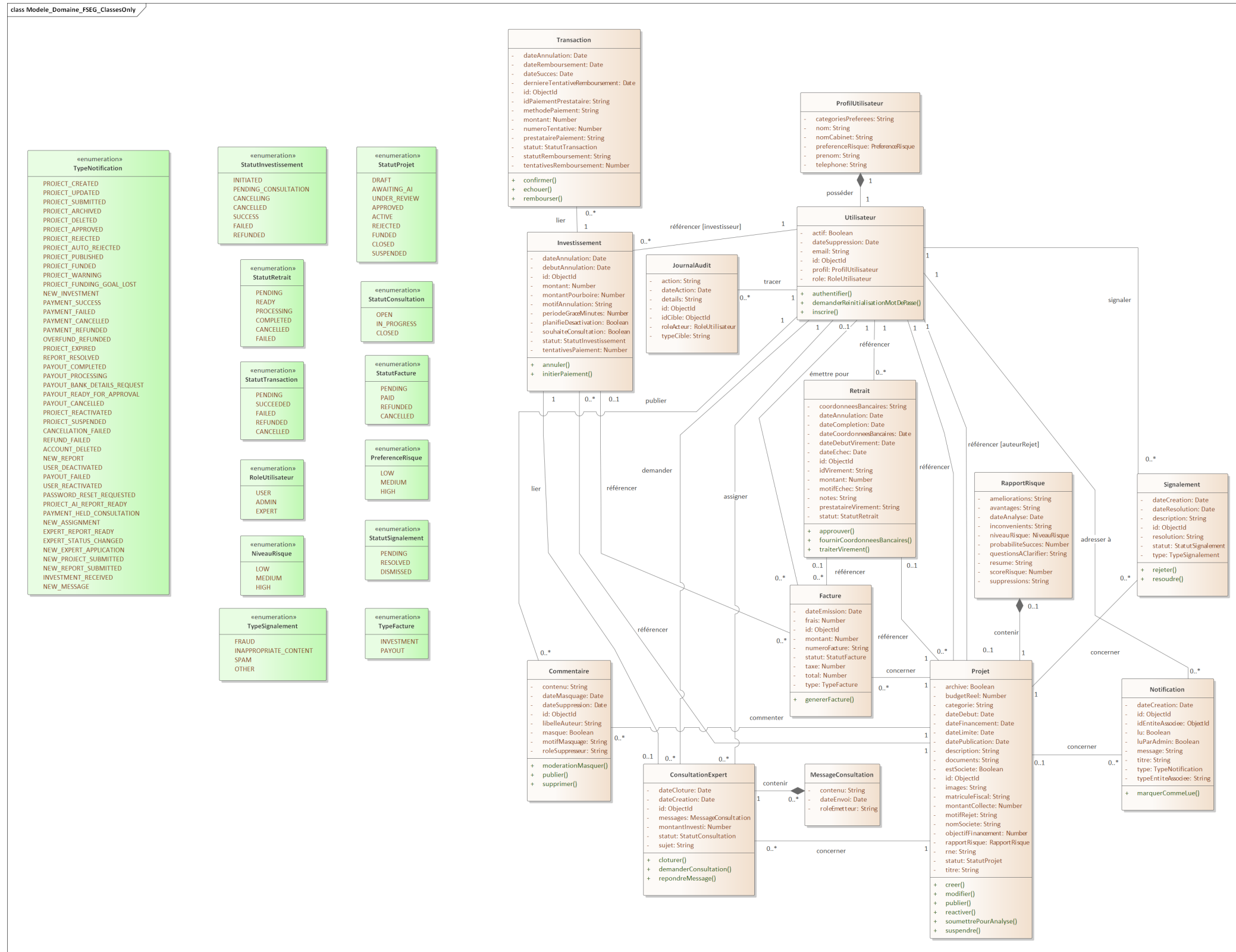


FIGURE 17 – Diagramme de classes du domaine (modèle du domaine)

3.4 Architecture du futur logiciel

3.4.1 Vue d'ensemble en couches

L'application suit une architecture client-serveur à trois couches, organisée ainsi :

- **Couche présentation** : application React (SPA) servie via Vite, communiquant avec la couche métier par appels REST sécurisés (cookies HttpOnly).
- **Couche métier** : Node.js + Express, qui expose les points d'accès API, applique les règles de validation (Zod), orchestre les services internes et gère les transactions MongoDB.
- **Couche données** : MongoDB (transactions multi-documents possibles si replica set; environnement local du PFE sans replica set).

Les services internes, détaillés dans le tableau 12, sont organisés par domaine : projets, investissements, retraits, administration, authentification, notifications, recommandations, expertise, IA, facturation et flux asynchrones. Ils sont assemblés par le routeur Express et leurs traitements asynchrones sont délégués à des files BullMQ/Redis (courriels, nouvelles tentatives) ou à des flux n8n (analyse de risque); l'assistant conversationnel est traité par l'API Express (POST /api/projects/:id/chat) avec appel LLM côté serveur.

TABLEAU 12 – Services internes du serveur applicatif

Domaine	Services internes concernés
Projets	projectService, adminProjectService
Investissements	investmentService
Retraits et facturation	payoutService, invoiceService
Administration et comptes	adminUserService, accountService
Authentification	authService
Notifications et signalements	notificationService, reportService
Recommandations	recommendationService
Expertise financière	expertService
Intelligence artificielle	geminiRiskService, geminiChatService
Workflows asynchrones	workflowInternalService

3.4.2 Diagramme de paquetages

La figure 18 formalise l'architecture logicielle sous forme de **diagramme de paquetages UML**. Contrairement à une organisation Spring Boot (*controller / repository / entités JPA*),

la plateforme suit la **structure des dossiers du dépôt** : client React, middleware, routeurs Express, services métier, validateurs Zod, configuration partagée, utilitaires, modèles Mongoose et intégrations. Les flèches orientées matérialisent les dépendances de compilation ou d'appel entre paquetages (lecture de haut en bas : du client vers la persistance).

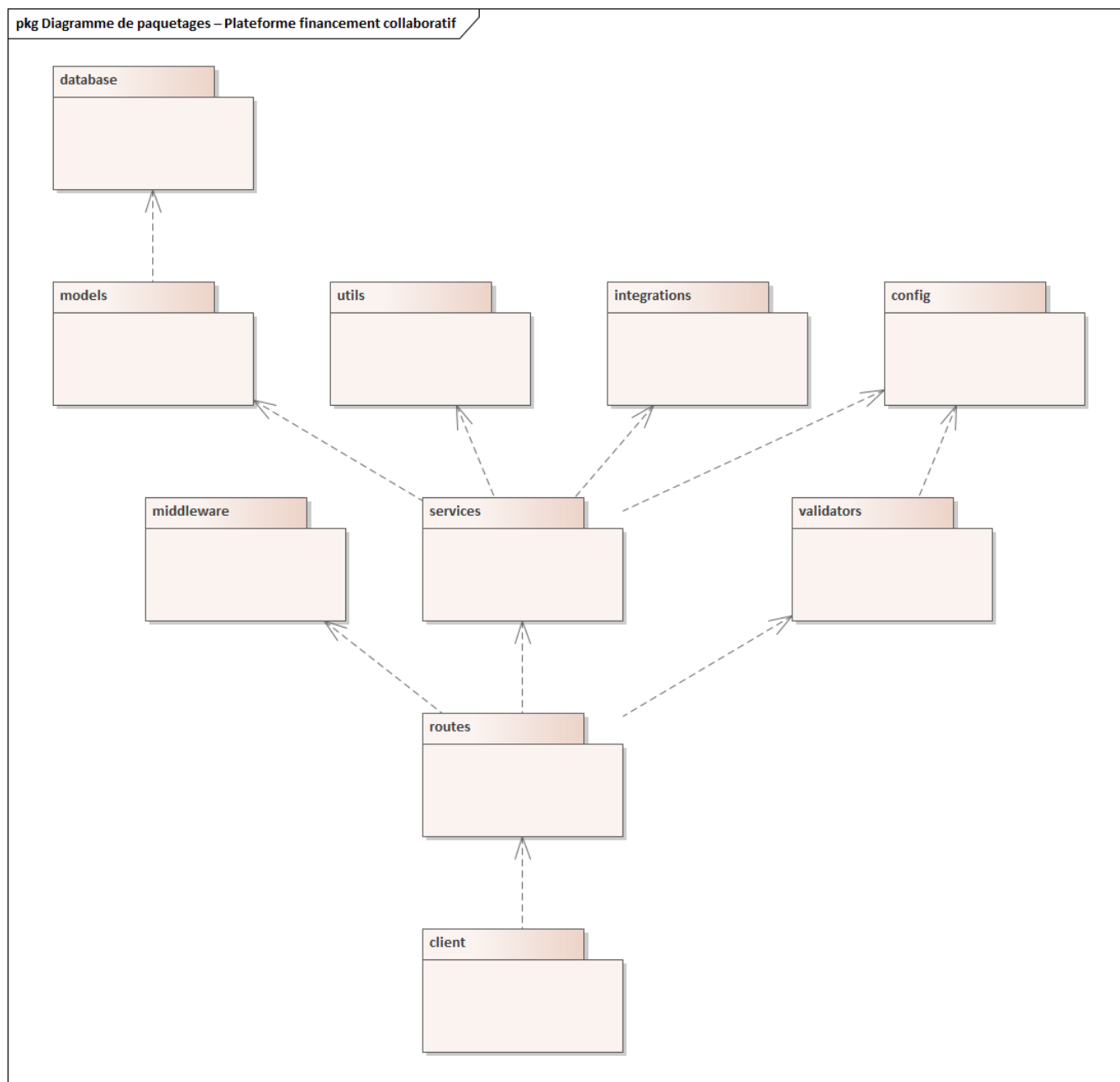


FIGURE 18 – Diagramme de paquetages — architecture logicielle (alignée sur le dépôt)

Lecture du diagramme. Chaque paquet UML correspond à un répertoire (ou ressource) du code :

- **client** (frontend/src/) : `pages/`, `components/`, `routes/AppRoutes.jsx`, `api/` (client Axios, `withCredentials`), `context/`, `payments/mock/` pour la démo de paiement; appels HTTPS vers `/api/*`.
- **middleware** (backend/middleware/) : JWT (`auth.js`), rôles (`requireAdmin`, `requireExpert`), `asyncHandler`, `errorHandler`, téléversement; appliqué dans les routeurs ou via `app.js` (CORS, *rate limiting* sur `/api`).
- **routes** (backend/routes/api/, backend/routes/internal/) : points d'entrée HTTP (`/api/*`, `/internal/*`, `webhooks`); valide les corps avec Zod puis délègue aux services.
- **validators** (backend/validators/) : schémas Zod (`authSchemas`, `projectSchemas`, `internalSchemas`, etc.); dépendent de `config/` pour les règles métier (plafonds, catégories).
- **services** (backend/services/) : logique métier (`*Service.js`, tableau 12), dont `geminiRiskService` et `geminiChatService`; s'appuient sur `config/projectLifecycle.js` pour les transitions de statut.
- **config** (backend/config/) : constantes et règles partagées (`businessRules`, `projectLifecycle`, `categories`); importé par les services et les validateurs (présent sur le diagramme source; régénérer la figure si le PNG du rapport est antérieur à cette mise à jour).
- **utils** (backend/utils/) : `HttpError`, envoi SMTP, chiffrement (`cryptoSeal`), transactions MongoDB optionnelles (`withOptionalTransaction`).
- **models** (backend/models/) : schémas Mongoose (`User`, `Project`, `Investment`, `Transaction`, `Payout`, `Notification`, `collections failed*`, etc.).
- **integrations** (backend/integrations/, backend/workers/) : prestataires simulés (`mockPaymentProvider`, `mockPayoutProvider`), files BullMQ (`emailQueue`, `riskAnalysisQueue`) consommant **Redis**, worker SMTP (`emailWorker.js`).
- **database** : MongoDB (collections documentaires); Redis n'apparaît pas comme paquet distinct mais est sollicité via `integrations`.

Enchaînement et acteurs externes. Flux nominal : *client* → *routes*; *routes* → *middleware* (import des garde-fous JWT, rôles, `asyncHandler`); *routes* → *validators* puis *services*; *services* → *models*, *integrations*, *utils* et *config*; *models* → *database*. Les appels à **Gemini** (analyse

et assistant) partent des services, hors paquet *integrations*. L'orchestrateur **n8n** (externe au dépôt) consomme la file `risk-analysis` et appelle `POST /internal/run-risk-analysis`; le détail est donné au chapitre 4.

Point de vigilance. En code, la validation Zod est toujours déclenchée depuis les **routeurs** (routes → validators), jamais depuis les `*Service.js`. Le diagramme source (`diagramme_paquetage.txt`) a été aligné en ce sens (suppression de la dépendance *services* → *validators*); vérifier que le PNG inséré dans le rapport correspond bien à cette version.

3.4.3 Décisions de conception structurantes

Les décisions suivantes relèvent de la conception globale du système et répondent aux exigences non fonctionnelles de sécurité, de cohérence et de traçabilité. Les détails d'implémentation relatifs à n8n, BullMQ/Redis et aux traitements planifiés sont présentés au chapitre 4.

- **Authentification par JWT (cookies HttpOnly)** : access token court + refresh token à plus longue durée, tous deux servis en cookies `HttpOnly`; `Secure`; `SameSite`, afin d'éviter l'exposition aux scripts client et au CSRF.
- **Base de données NoSQL document** : MongoDB choisi pour la flexibilité du document `Project` (champs variables, sous-document `aiAnalysis`) et la simplicité d'évolution du schéma.
- **Paiement asynchrone idempotent** : la confirmation arrive par *webhook* signé; la combinaison (`provider`, `providerPaymentId`) est unique pour absorber les tentatives du fournisseur.
- **Mise à jour conditionnelle atomique** du financement : `updateOne` sur `projects` avec contrainte `currentFunding + amount ≤ fundingGoal` pour empêcher tout sur-financement, même en concurrence.
- **Chiffrement des coordonnées bancaires** : algorithme AES-256-GCM avec IV par enregistrement et clé maître chargée hors code, pour respecter les exigences de confidentialité des RIB.
- **Limitation du débit des requêtes** : en sortie API publique, plus stricte sur les surfaces sensibles (commentaires, signalements, authentification) pour limiter les abus.

3.5 Modèle de données MongoDB

3.5.1 Collections et structure des documents

Le modèle de données est dérivé du modèle du domaine et des schémas Mongoose. Comme la solution repose sur une base NoSQL (MongoDB), la structure est décrite sous forme de collections et de références (ObjectId) plutôt que sous forme de tables relationnelles.

- **users** : stocke les identifiants, le rôle, le statut actif, les jetons de réinitialisation, et le profil (préférences de risque, catégories).
- **projects** : contient les détails du projet ainsi que l'analyse IA embarquée (**aiAnalysis**).
- **investments** : lie un investisseur à un projet avec un montant et un statut.
- **transactions** : enregistre les informations de paiement (démon / extensible).
- **payouts** : stocke les retraits (statut, coordonnées bancaires chiffrées, suivi du virement).
- **invoices** : stocke les factures numériques à numérotation séquentielle (investissement et retraits), affichées dans l'interface et exportables en PDF via le navigateur.
- **expertconsultations** : trace le processus de consultation entre l'investisseur et l'expert (statut OPEN, IN_PROGRESS, CLOSED).
- **comments** : stocke les commentaires (modération, masquage, suppression).
- **reports** : gère les signalements.
- **notifications** : stocke les notifications intégrées à l'application.
- **auditLogs** : journalise les actions sensibles (traçabilité).
- **failedWorkflowEvents** : stocke les échecs des workflows n8n pour reprise manuelle.
- **failedCancellationEvents** : stocke les échecs d'annulation de paiement lors de la désactivation d'un utilisateur.
- **failedRefundEvents** : stocke les échecs de remboursement (sur-financement, expiration, etc.).
- **failedPayoutEvents** : stocke les échecs de traitement des retraits.

3.5.2 Justification du modèle NoSQL

Le choix de MongoDB repose sur la nature semi-structurée des données. Les projets contiennent des champs variables (images, documents, analyse IA) qui évoluent au fil du temps. Un modèle relationnel imposerait des tables supplémentaires ou des colonnes redondantes, tandis que MongoDB permet d'ajouter des champs sans migration. De plus, la possibilité d'utiliser des sous-documents (embedding) pour **profile** et **aiAnalysis** améliore les performances de

lecture.

3.5.3 Indexation et relations

Pour optimiser les performances, les index suivants sont créés :

- `users.email` : unique.
- `projects` : `{status, deadline}`, `{status, startAt}`, `{creatorId}`, `{status, isArchived}` et index texte sur `title/description`.
- `investments.investorId` et `investments.projectId` : pour retrouver rapidement les investissements d'un utilisateur ou d'un projet.
- `transactions.investmentId` : index simple (plusieurs transactions peuvent exister pour un même investissement, par exemple en cas de retentative).
- `transactions.{provider, providerPaymentId}` : index unique pour assurer l'idempotence des webhooks; `{status, createdAt}` pour les relances.
- `notifications.{userId, createdAt}` et `notifications.read` : listes paginées et compteur de non-lus.
- `payouts.projectId`, `payouts.creatorId` et `payouts.status` : suivis administratifs et file d'approbation.
- `comments.projectId`, `comments.userId`, `comments.isHidden` et `comments.deletedAt` : filtrage, modération et affichage public.
- `auditLogs.timestamp` et `auditLogs.action` : historique des actions sensibles.
- `failedRefundEvents.resolved`, `failedPayoutEvents.resolved` (et dates associées) : files d'échec exploitées par l'administration.
- `failedWorkflowEvents.resolved` et `failedWorkflowEvents.createdAt` : pour lister les échecs non résolus.
- `failedCancellationEvents.resolved` et `failedCancellationEvents.createdAt` : idem.

Les relations sont gérées par références (`ObjectId`). L'embedding est utilisé pour les profils utilisateur et l'analyse IA dans les projets. En MongoDB, les jointures sont effectuées au niveau de l'application en utilisant les méthodes `populate` (avec Mongoose) ou des requêtes manuelles.

3.5.4 Dictionnaire des données

Le tableau 13 détaille les principaux champs retenus au sein des collections décrites.

TABLEAU 13 – Dictionnaire des données

Collection	Champ	Description
users	email	Adresse email de l'utilisateur (unique).
	passwordHash	Mot de passe haché avec bcrypt.
	role	Rôle de l'utilisateur (USER, EXPERT ou ADMIN).
	isActive	Statut du compte (actif / désactivé).
	resetToken	Jeton de réinitialisation du mot de passe.
	resetTokenExpiry	Date d'expiration du jeton.
	profile.firstName	Prénom.
	profile.lastName	Nom de famille.
	profile.riskPreference	Tolérance au risque (LOW, MEDIUM, HIGH).
	profile.preferredCategories	Catégories de projets préférées.
projects	title	Titre du projet.
	description	Description détaillée.
	category	Catégorie (ex : Technologie, Art, etc.).
	fundingGoal	Objectif de collecte affiché (minimum 1 000 TND côté API).
	realBudget	Besoin réel net du porteur ; base du calcul de la commission plateforme (5 %) au retrait.
	currentFunding	Montant collecté (mise à jour atomique conditionnelle).
	status	État du projet (DRAFT, AWAITING_AI, UNDER_REVIEW, APPROVED, ACTIVE, REJECTED, FUNDED, CLOSED, SUSPENDED).
	aiStatus	État de l'analyse IA (PENDING, COMPLETED, FAILED).
	aiAnalysis.report	Rapport structuré (résumé, avantages, inconvénients, améliorations, questions, score).

Collection	Champ	Description
	aiAnalysis.riskLevel	Niveau de risque (LOW, MEDIUM, HIGH).
investments	investorId	Référence vers l'utilisateur investisseur.
	projectId	Référence vers le projet.
	amount	Montant investi (strictement positif).
	status	État de l'investissement (INITIATED, PENDING_CONSULTATION, CANCELLING, CANCELLED, SUCCESS, FAILED, REFUNDED).
	tipAmount	Pourboire optionnel ajouté par l'investisseur.
	updatedAt	Date de dernière mise à jour (utilisée pour repérer les contributions restées en INITIATED trop longtemps).
transactions	investmentId	Référence vers l'investissement.
	provider	Fournisseur de paiement (mock en démonstration).
	providerPaymentId	Identifiant du paiement chez le fournisseur (index unique avec provider pour l'idempotence).
	status	État de la transaction (PENDING, SUCCEEDED, FAILED, REFUNDED, CANCELLED).
payouts	projectId	Projet concerné ; contrainte d'unicité (au plus un retrait par projet).
	creatorId	Porteur de projet bénéficiaire.
	amount	Montant du versement demandé ou approuvé.
	status	Cycle de vie (PENDING, READY, PROCESSING, COMPLETED, CANCELLED, FAILED).
	bankDetails	Coordonnées bancaires chiffrées (AES-256-GCM) lorsque saisies.
	provider / providerTransferId	Fournisseur de virement et identifiant côté prestataire (démonstration / extensible).

Collection	Champ	Description
	failureReason / horodatages (bankDetailsProvidedAt, etc.)	Suivi des échecs et jalons métier du virement.
comments	projectId / userId	Projet commenté et auteur du message.
	content	Texte du commentaire (longueur plafonnée côté schéma).
	isHidden / hiddenReason / hiddenBy / hiddenAt	Modération administrative : masquage et traçabilité.
	deletedAt / deletedBy / deletedByRole	Suppression (« soft delete ») par l’auteur ou l’administrateur.
reports	reporterId	Référence vers l’utilisateur ayant signalé.
	projectId	Référence vers le projet signalé.
	type	Type de signalement (FRAUD, SPAM, etc.).
	status	État du signalement (PENDING, RESOLVED, DISMISSED).
notifications	userId	Référence vers l’utilisateur destinataire.
	type	Type de notification (ex. PROJECT_APPROVED).
	read	Indicateur de lecture (booléen read).
auditLogs	actorId / actorRole	Auteur de l’action (utilisateur référencé) et rôle (USER, ADMIN).
	action / targetType / targetId	Verbe métier (EXPIRE_PROJECT, etc.) et cible typée avec identifiant.
	details / timestamp	Charge utile libre (Mixed) pour le contexte, date de l’événement.
failedRefundEvents	investmentId / projectId	Investissement et projet concernés par l’échec de remboursement.
	reason	Motif (OVERFUNDING, EXPIRY, désactivation utilisateur, etc.).
	error / retryCount / resolved	Diagnostic, tentatives et indicateur de résolution manuelle.
	resolvedBy / resolvedAt	Traçabilité de la clôture côté administration.
failedPayoutEvents	payoutId	Retrait ayant échoué définitivement côté file / worker.

Collection	Champ	Description
	error / retryCount / resolved	Diagnostic, tentatives et reprise administrative.
	resolvedBy / resolved At	Audit de résolution lorsque traité par un administrateur.
failedWorkflowEvents	workflowType	Type de workflow (risk-analysis, notification).
	payload	Données initiales du workflow.
	error	Message d'erreur.
	retryCount	Nombre de tentatives déjà effectuées.
	resolved	Indique si l'échec a été résolu (manuellement).
failedCancellationEvents	investmentId	Investissement concerné.
	error	Message d'erreur.
	retryCount	Nombre de tentatives.
	resolved	Indique si l'échec a été traité.

3.6 Conclusion

Ce chapitre a opéré la transposition des besoins métier dans une conception UML homogène : les diagrammes de séquence en explicitent les enchaînements pour les cas critiques, le diagramme de classes en restitue la structure conceptuelle, et l'architecture en couches situe les principaux domaines applicatifs ainsi que les arbitrages structurants (JWT dans des cookies HttpOnly, idempotence des *webhooks*, atomicité des mises à jour de financement et chiffrement AES-256-GCM des RIB). Le modèle de données MongoDB et le dictionnaire des données constituent le référentiel directement mobilisable par l'implémentation exposée au chapitre suivant.

Chapitre 4

Implémentation

4.1 Introduction

Ce chapitre présente la mise en œuvre logicielle de la plateforme de financement collaboratif. Il décrit d'abord l'environnement de réalisation (langages, frameworks, outils de développement et système de gestion de base de données), puis illustre, pour les cas d'utilisation retenus, les éléments d'implémentation les plus significatifs : captures d'écran de l'interface, extraits de code commentés, bibliothèques mobilisées et composants techniques associés (files asynchrones, routes internes, intégrations externes). Les sept scénarios documentés en section 4.3 sont alignés sur les diagrammes UML du chapitre 3.

4.2 Environnement de réalisation

La plateforme a été développée selon une architecture client-serveur : une SPA React (Vite) consomme une API REST Node.js/Express, avec persistance MongoDB, files BullMQ sur Redis et orchestration n8n pour l'analyse de risque. Les sous-sections suivantes détaillent les langages et frameworks, les outils de développement, le SGBD et la validation de l'implémentation. La plateforme a été exécutée en **environnement local** (poste de développement) ; elle n'a pas fait l'objet d'une mise en ligne sur un serveur distant au cours du PFE.

4.2.1 Environnement logiciel

Les principaux outils et bibliothèques utilisés sont présentés ci-dessous selon le canevas suivant : rôle général, usage dans l'application développée, puis atouts retenus pour le développement. Les icônes reproduites proviennent des chartes graphiques des éditeurs concernés.

— Couche métier (serveur)

- **Node.js (runtime) et pile serveur** : Node.js est le moteur d'exécution JavaScript côté serveur. Express 5 structure l'API REST (/api/*, routes internes /internal/*,

webhooks). Mongoose 9 matérialise les collections et la validation au plus près des documents ; Zod 4 renforce le contrôle des entrées HTTP ; BullMQ 5 pilote les jobs asynchrones ; Nodemailer, jsonwebtoken et bcrypt couvrent respectivement l'e-mail transactionnel, les jetons d'authentification et le hachage des mots de passe (versions principales indiquées dans `backend/package.json`).

Dans l'application : la logique métier est découpée en services (`*Service.js`) raccordés aux routeurs Express ; les opérations sensibles au temps (envois SMTP, relances) sont confiées aux workers plutôt qu'exécutées de manière bloquante dans le cycle requête-réponse.

Avantages : même langage que la couche présentation sur la partie « données et API », modèle non bloquant adapté aux entrées/sorties réseau et à la base de données, écosystème Node.js fourni pour intégrer files, validation et sécurité.



- **Redis (files et état des jobs) :** Redis est une base clé-valeur en mémoire utilisée ici comme magasin de données pour les files d'attente.

Dans l'application : BullMQ s'appuie sur Redis (via ioredis) pour persister l'état des jobs (retardements, tentatives, verrous légers) avant toute remontée métier vers les collections d'échecs définitifs lorsque les reprises automatiques sont épuisées.

Avantages : latence très faible, primitives adaptées aux files et au scheduling, bonne tenue en charge lors des pics (notifications, relances administrateur).



— Couche présentation (client)

- **React.js :** librairie JavaScript pour construire une interface utilisateur à base de composants réutilisables et d'un rendu réactif sur l'état.

Dans l'application : les vues plein écran et pages métier sont regroupées dans `frontend/src/pages` (parcours investisseur, porteur de projet, administration, paiement démo, etc.) ; les éléments réutilisables (cartes projet, en-têtes, garde-fous d'accès)

sont placés dans `frontend/src/components` et `frontend/src/components/ui`. La navigation repose sur React Router : enveloppe `BrowserRouter` dans `App.jsx`, déclaration des routes dans `routes/AppRoutes.jsx`. Les appels au serveur passent par Axios, centralisé dans `frontend/src/api/client.js` (`withCredentials` pour les cookies d'authentification, intercepteur qui tente un rafraîchissement de session sur réponse 401 puis rejoue la requête initiale).

Avantages : forte modularité par composants, mise à jour efficace du DOM via le DOM virtuel, écosystème mature (hooks, outils de débogage, bibliothèques complémentaires).



- **Vite** : outil de build et serveur de développement orienté modules ES, chargé d'assembler le frontend pour le navigateur.

Dans l'application : le point d'entrée `main.jsx` monte l'arbre React sur le nœud `#root`; la configuration (`vite.config.*`) fixe le serveur de développement, les alias de chemins et la cible de production. Le rechargement à chaud accélère les itérations sur les pages et composants.

Avantages : démarrage rapide du serveur de développement, retour visuel quasi immédiat lors des modifications, configuration sobre comparée à des chaînes Webpack historiques plus lourdes.



- **Tailwind CSS v4 et Shaden UI** : framework utilitaire CSS combiné à une bibliothèque de composants accessibles et personnalisables (Radix UI sous le capot).

Dans l'application : le style global est géré via la directive `@import "tailwindcss"`; et le plugin Vite `@tailwindcss/vite`. Les composants réutilisables (boutons, modales, dialogues, toasts) sont issus de Shaden UI et configurés dans `frontend/src/components/ui/`, avec des micro-animations (`tw-animate-css`, `clsx`).

Avantages : compilation CSS efficace avec Tailwind v4, design system personnalisable, intégration avec React 19 et respect des bonnes pratiques d’accessibilité (WAI-ARIA).



- **MongoDB :** base de données NoSQL orientée documents, adaptée aux schémas évolutifs et aux regroupements logiques (sous-documents, tableaux embarqués).

Dans l’application : les collections `users`, `projects`, `investments`, `transactions`, `payouts`, `notifications`, etc., sont décrites avec Mongoose; le document `Project` embarque notamment `aiAnalysis`. Les transactions multi-documents (nécessitant un replica set MongoDB, non activé dans l’environnement local du PFE) sécurisent les opérations qui exigent une atomicité (ex. mise à jour du financement avec contrainte de plafond).

Avantages : flexibilité du modèle document, performances de lecture lorsque les données affichées ensemble sont stockées ensemble, alignement naturel avec les objets manipulés côté Node.js.



- **Orchestration et exécution conteneurisée**

- **n8n :** plateforme d’orchestration visuelle de workflows (nœuds HTTP, webhooks, branchements conditionnels, politiques de nouvelle tentative).

Dans l’application : à la soumission, un job BullMQ est consommé par n8n, qui appelle `POST /internal/run-risk-analysis`; Express exécute alors `geminiRiskService` et persiste le rapport (ou trace l’échec). L’assistant (`/api/projects/:id/chat`) appelle `geminiChatService` directement, sans n8n.

Avantages : découverte visuelle des enchaînements, évolutions d’intégration sans

redéployer tout le cœur métier, mécanismes de *retry* configurables.



- **Docker** : technologie de conteneurisation qui encapsule une application et ses dépendances dans une image reproductible.

Dans l'application : l'instance n8n utilisée pour concevoir et valider le workflow d'analyse a été exécutée dans un conteneur dédié, isolée de la pile Node/React locale (voir aussi la sous-section « Conteneurisation (Docker) »).

Avantages : environnement reproductible entre séances de travail (image + orchestrateur), isolation des ports et des versions sans pollution de la machine hôte.



— Outillage

- **Visual Studio Code** : éditeur de code extensible, utilisé comme environnement de développement intégré quotidien.

Dans l'application : édition conjointe du serveur Node/Express, du client React/Vite, des scripts de tests de bout en bout et des fichiers de configuration ; coloration et navigation dans les schémas Mongoose, les routeurs et les composants JSX.

Avantages : intégration Git, terminal et débogueur dans une seule fenêtre, marketplace d'extensions (ESLint, Prettier, clients REST légers), prise en charge native de JavaScript et JSX.



- **Git / GitHub** : Git assure le suivi de version ; le dépôt distant sur GitHub est partagé avec l'encadrant et l'architecte d'entreprise (voir § 4.2.4).

Dans l'application : historique des fonctionnalités par branches et *commits*, revue

des changements avant intégration, diffusion des livrables et des captures associées au rapport.

Avantages : traçabilité fine, sauvegarde centralisée, facilitation du travail collaboratif et des relectures.



- **Enterprise Architect 15 :** outil de modélisation UML et d'ingénierie dirigée par les modèles.

Dans l'application : production des diagrammes de cas d'utilisation, de séquence, de classes (y compris classes participantes) et de paquetages exportés dans ce rapport, en cohérence avec les fonctionnalités implémentées.

Avantages : formalisme UML reconnu, traçabilité des exigences vers la conception, supports graphiques exploitables en soutenance.



- **Postman :** client HTTP pour composer, rejouer et documenter des requêtes vers l'API.

Dans l'application : tests manuels des routes `/api/*` (authentification, projets, investissements, webhooks simulés) et validation des corps JSON / codes de statut pendant le développement.

Avantages : reproductibilité des scénarios, collections partageables, inspection rapide des en-têtes et cookies.



- **Tests automatisés et validation manuelle :** les scénarios de bout en bout s'appuient sur le même runtime Node.js que le serveur applicatif.

Dans l'application : les scripts `e2e-lifecycle-payout-api.js`, `e2e-lifecycle-scenarios.js`, `e2e-edgescases-api.js` et `e2e-interactions.js` enchaînent création de données, transitions de statut, paiements simulés et cas d'erreur ; ils complètent les tests manuels sur l'interface React.

Avantages : détection précoce des régressions sur les parcours critiques, exécution sans navigateur pour la couche API, automatisation rejouable avant chaque livraison.

4.2.2 Conteneurisation (Docker) et expérimentation n8n

Les flux d'analyse de risque (déclenchement BullMQ, consommation n8n, appel `/internal/run-risk-analysis`, persistance côté Express) ont été mis au point et testés dans un environnement Docker local. L'instance n8n y est exécutée dans un conteneur dédié, ce qui permet : (i) d'isoler l'orchestrateur du reste de la machine de développement ; (ii) d'obtenir une configuration reproductible (même image, mêmes ports, même version) entre séances de travail. En pratique, cette base a servi à valider les nœuds du graphe (HTTP, rappels HTTP, gestion des erreurs, nouvelles tentatives) avant de documenter le flux dans le rapport et de le corrélérer avec les appels `/internal/*` côté API.

4.2.3 Orchestration et traitements asynchrones

Dans l'implémentation développée, n8n isole le déclenchement asynchrone de l'analyse de risque. À la soumission, Express enfile un job BullMQ ; n8n le consomme et appelle `/internal/run-risk-analysis`, où Express invoque Gemini, applique les contrôles (dont l'auto-rejet si l'écart budgétaire est $\geq 30\%$) et persiste le rapport ou l'échec. L'assistant conversationnel sur projet suit un autre parcours : il est traité directement par l'API Express via `POST /api/projects/:id/chat`, avec authentification, limitation du débit des requêtes et appel au service LLM côté serveur.

Les traitements de reprise et les envois non bloquants sont confiés à BullMQ sur Redis. Les e-mails critiques, les relances de workflows et les reprises liées aux paiements ou aux retraits sont placés dans des files persistantes avec tentatives contrôlées ; lorsque les reprises automatiques sont épuisées, l'échec est conservé dans une collection dédiée afin de permettre une reprise ou une vérification administrative. Les tâches planifiées du serveur couvrent notamment l'expiration des projets, l'expiration des investissements, la reprise des analyses bloquées et le nettoyage des sessions.

4.2.4 Gestion de version et collaboration (GitHub)

Le dépôt privé de la plateforme (serveur Node.js/Express, client React, scripts de tests de bout en bout, éventuelles configurations) est versionné sur GitHub. Il assure la traçabilité des évolutions (*commits*, branches, éventuelles *pull requests*) et constitue la référence unique pour les livrables du PFE. L'accès au dépôt a été partagé avec l'encadrant académique afin de permettre le suivi et la relecture du travail, ainsi qu'avec l'équipe technique afin d'aligner les choix d'implémentation sur la vision d'architecture retenue en entreprise et de faciliter les revues de code (structure, conventions, points d'intégration).

4.2.5 Synthèse des choix technologiques

Les tableaux 14 et 15 synthétisent les technologies retenues et la justification de chaque choix (socle applicatif d'une part, IA/orchestration et exploitation d'autre part).

TABLEAU 14 – Technologies utilisées — socle applicatif, données et sécurité

Composant	Technologie	Justification
Couche présentation	React + Vite + Tailwind CSS v4 + Shadcn UI	Composants accessibles, charte graphique sur mesure, interface cohérente et responsive.
Couche métier	Node.js + Express	Environnement JavaScript unifié, non bloquant, adapté aux API REST et aux traitements asynchrones.
Validation	Zod	Schémas déclaratifs en JavaScript (zod), validation des entrées API et messages d'erreur exploitables côté contrôleur.
Base de données	MongoDB	Modèle document, sous-documents pour aiAnalysis, transactions multi-documents pour la cohérence.
Files asynchrones	BullMQ + Redis	Retentatives exponentielles, DLQ, supervision des jobs (e-mails, reprises).
Authentification	JWT (cookies HttpOnly)	Access token court ; refresh token persisté ; protection contre l'exposition côté client (XSS).
Chiffrement RIB	AES-256-GCM	Confidentialité des coordonnées bancaires, IV unique par enregistrement.

TABLEAU 15 – Technologies utilisées — IA, orchestration et exploitation (suite)

Composant	Technologie	Justification
IA (analyse / assistant)	Gemini + n8n (déclenchement); Express (appels LLM)	Analyse via <code>/internal/run-risk-analysis</code> , retentatives n8n, auto-rejet si écart $\geq 30\%$; assistant synchrone <code>/api/projects/:id/chat</code> .
Orchestration	n8n	Découplage des workflows d'analyse; évolutions sans toucher au cœur Express pour ces flux.
Conteneur local	Docker (instance n8n)	Isolation de n8n et reproductibilité des tests de workflow sur le poste de développement.
Versionnage	GitHub	Traçabilité du code; dépôt partagé avec l'encadrant académique et l'équipe technique pour suivi et revue.

4.2.6 Validation et tests

La validation a été effectuée au moyen de scripts de tests de bout en bout (E2E) en Node.js, qui simulent des parcours métier complets via l'API, en l'absence de *frameworks* de tests unitaires (le projet n'utilise ni Jest, ni Mocha, ni Cypress). Les fichiers concernés résident sous `backend/scripts/` :

- `e2e-lifecycle-payout-api.js` : inscription avec vérification OTP, création projet (DRAFT), soumission à l'analyse IA, validation via `/api/admin/projects/:id/validate` puis publication, contribution + *webhook* signé, création du *payout*, saisie des coordonnées bancaires chiffrées, approbation administrateur du retrait.
- `e2e-lifecycle-scenarios.js` et `e2e-edgescases-api.js` : scénarios limites (annulation d'investissement, relance de paiement, sur-financement et remboursement, transitions de statut invalides, erreurs de validation).
- `e2e-interactions.js` : interactions sociales (commentaires, signalements, assistant conversationnel).

Une vérification rapide après démarrage local du serveur peut consister en des requêtes GET anonymes (`/api/health`, `/api/projects/public`) contre l'API lancée sur la machine de développement (URL par défaut `http://localhost:5000`, modifiable via la variable `PORT`).

En complément, des tests manuels guidés ont été réalisés sur l'interface web (parcours porteur de projet, investisseur, administrateur) afin de vérifier l'ergonomie, la cohérence des messages et la fidélité de l'affichage des statuts.

4.3 Réalisation technique des cas d'utilisation

Conformément au canevas du mémoire, chaque cas d'utilisation retenu fait l'objet d'une description technique structurée autour des éléments suivants :

- une **capture d'écran** de l'interface utilisateur correspondante ;
- un **extrait de code** (sous Visual Studio Code) mettant en évidence la partie significative de l'implémentation ;
- les **bibliothèques** utilisées (Node.js, React, Mongoose, BullMQ, etc.) ;
- les **composants techniques** mobilisés (services métier, routes API, files Redis, workflow n8n, webhooks, chiffrement, etc.).

Les sept blocs ci-dessous sont mis en relation avec les diagrammes de séquence et les classes participantes du chapitre 3 (tableau 16). Les autres fonctionnalités du chapitre 2 (authentification, recommandations, assistant conversationnel, notifications, tâches planifiées) s'appuient sur les mêmes briques et sont couvertes par les scripts de validation décrits en § 4.2.6.

TABEAU 16 – Traçabilité — sept cas alignés sur les diagrammes UML du chapitre 3

Cas traité (ch. 4)	Références de conception (ch. 3)	Réalisation illustrée	Service principal
Annuler contribution	Séquence 3 ; classes 4	UI 19 ; code 20	investmentService.cancelInvestment
Créer / soumettre projet	Séquence 5 ; classes 6	UI 21 ; code 24	projectService.submitProjectForAi
Investir + webhook	Séquence 7 ; classes 8	UI 25 ; code 28	investmentService
Valider (expert) / publier (admin)	Séquence 9 ; classes 10	UI 35 ; code 37	expertService.validateProjectAnalysis ; adminProjectService.publishProject
Coordonnées bancaires	Séquence 11 ; classes 12	UI 39 ; code 40	payoutService.provideBankDetails
Approuver retrait	Séquence 13 ; classes 14	UI 41 ; code 43	payoutService.approvePayout
Suspendre / réactiver	Séquence 15 ; classes 16	UI 45 ; code 46	adminProjectService

4.3.1 Annuler une contribution

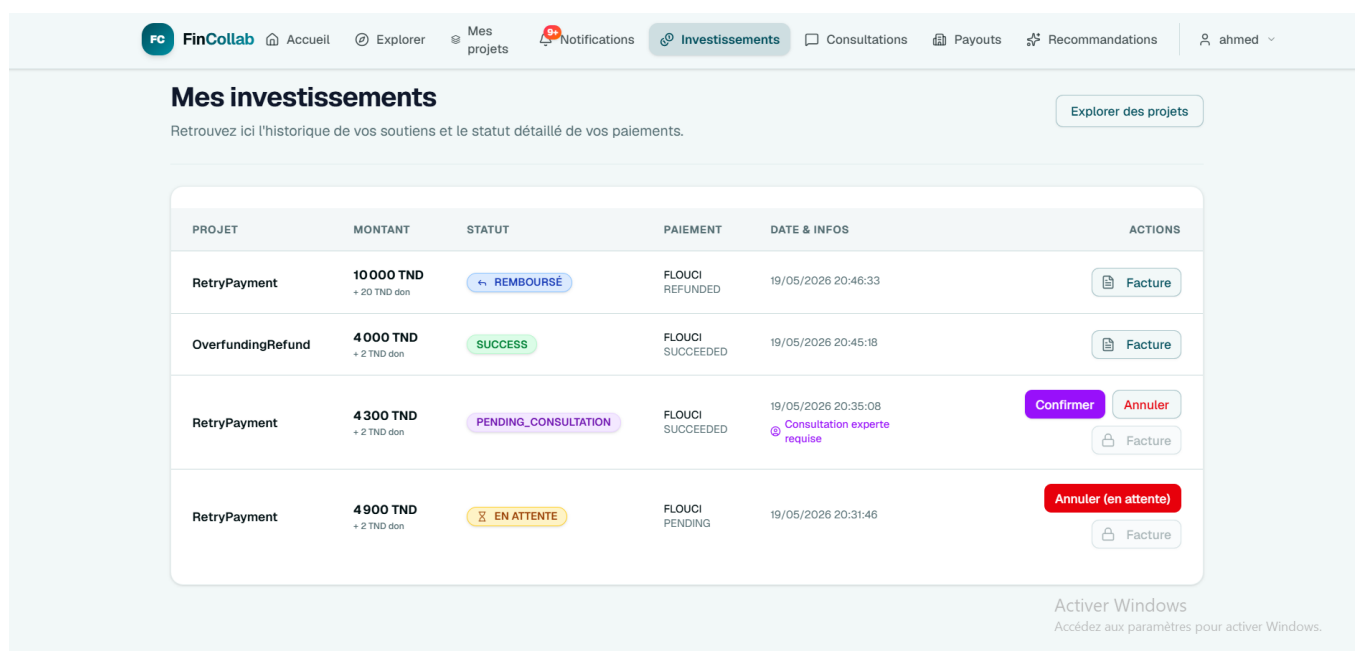
Lien avec la conception

Ce cas correspond aux figures 3 et 4 (chapitre 3).

Description

L'investisseur peut annuler tant que l'investissement est INITIATED (transaction PENDING), ou, après succès, pendant la fenêtre `cancellationGracePeriodMinutes` (ou tant que le dossier est en PENDING_CONSULTATION).

Capture d'écran



The screenshot shows the 'Mes investissements' (My Investments) page. The page header includes navigation links: Accueil, Explorer, Mes projets, Notifications, Investissements (active), Consultations, Payouts, and Recommendations. The user 'ahmed' is logged in.

The main section is titled 'Mes investissements' and includes a sub-header 'Retrouvez ici l'historique de vos soutiens et le statut détaillé de vos paiements.' and a button 'Explorer des projets'.

The table displays the following data:

PROJET	MONTANT	STATUT	PAIEMENT	DATE & INFOS	ACTIONS
RetryPayment	10 000 TND + 20 TND don	REMBOURSE	FLOUCI REFUNDED	19/05/2026 20:46:33	Facture
OverfundingRefund	4 000 TND + 2 TND don	SUCCESS	FLOUCI SUCCEEDED	19/05/2026 20:45:18	Facture
RetryPayment	4 300 TND + 2 TND don	PENDING_CONSULTATION	FLOUCI SUCCEEDED	19/05/2026 20:35:08 Consultation experte requis	Confirmer, Annuler, Facture
RetryPayment	4 900 TND + 2 TND don	EN ATTENTE	FLOUCI PENDING	19/05/2026 20:31:46	Annuler (en attente), Facture

At the bottom right, there is a Windows watermark: 'Activer Windows. Accédez aux paramètres pour activer Windows.'

FIGURE 19 – Mes contributions — statuts et actions

Extrait de code (VS Code — investmentService.js)

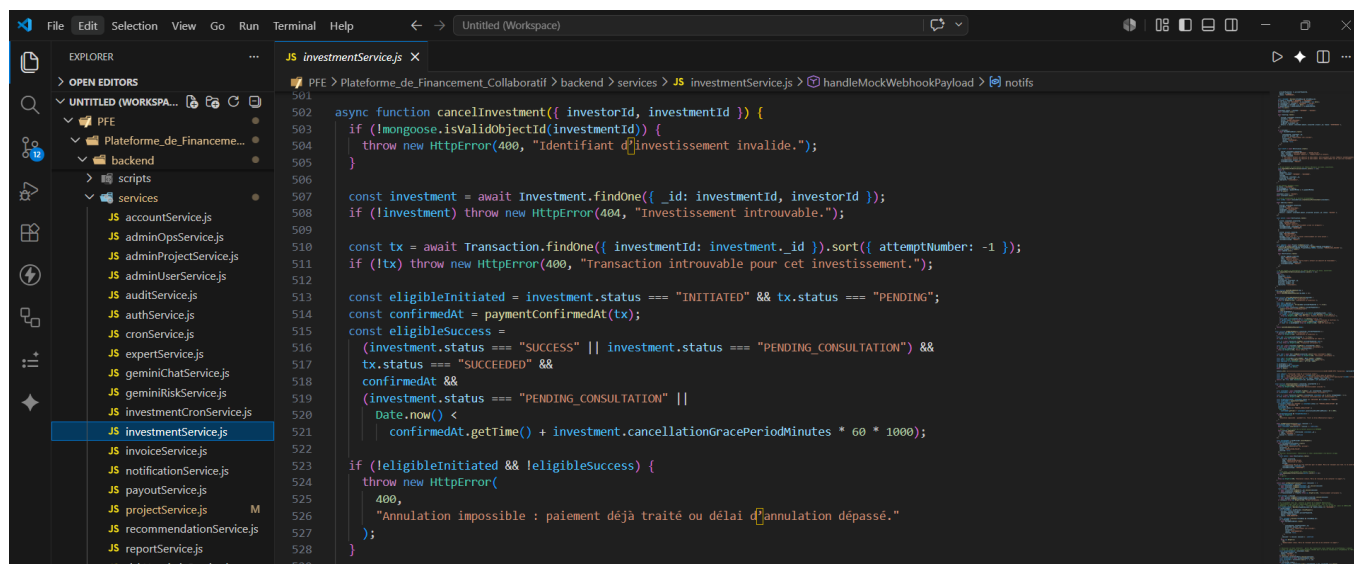


FIGURE 20 – Annulation : contrôle d'éligibilité et passage en CANCELLING

Bibliothèques et composants

— Transactions MongoDB, file remboursement BullMQ, Invoice, route POST/api/investments/:id/cancel.

4.3.2 Créer et soumettre un projet à l'analyse IA

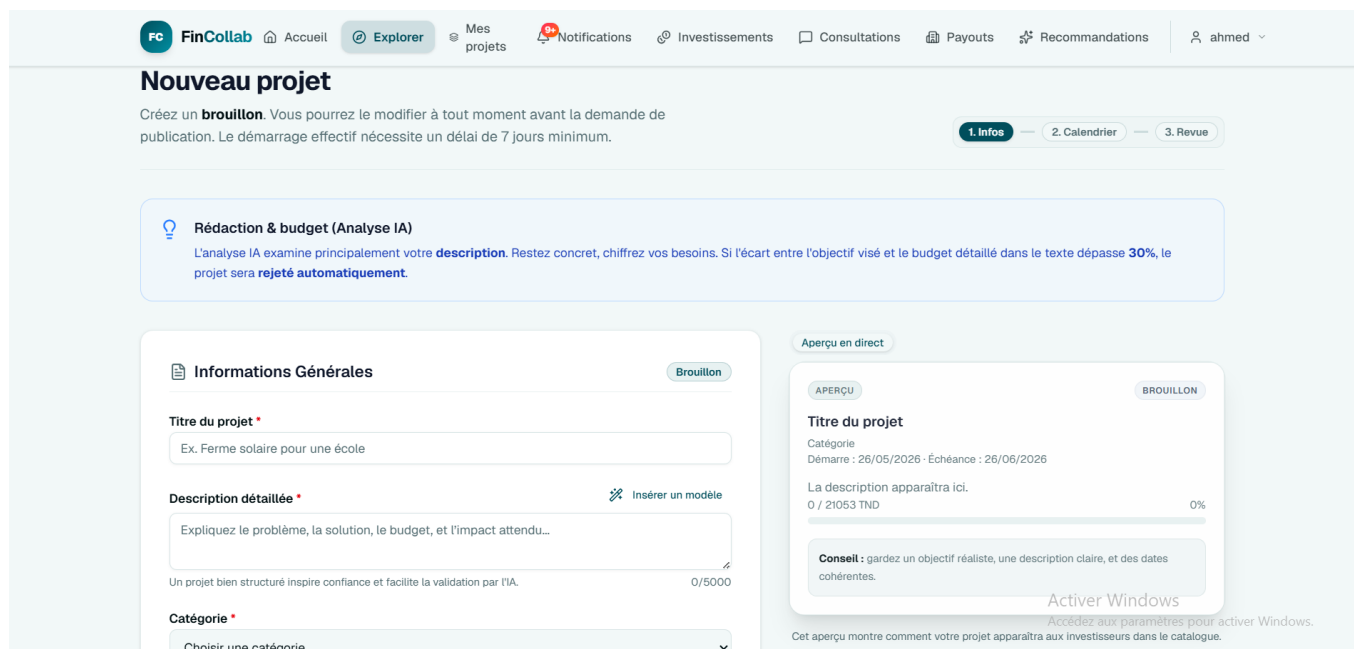
Lien avec la conception

Ce cas correspond aux figures 5 et 6 (chapitre 3).

Description

Le porteur enregistre un brouillon (DRAFT), puis le soumet : passage en AWAITING_AI, file BullMQ et workflow n8n (figure 23) pour l'analyse LLM, puis UNDER_REVIEW.

Captures d'écran



Nouveau projet

Créez un **brouillon**. Vous pourrez le modifier à tout moment avant la demande de publication. Le démarrage effectif nécessite un délai de 7 jours minimum.

1. Infos 2. Calendrier 3. Revue

Rédaction & budget (Analyse IA)

L'analyse IA examine principalement votre **description**. Restez concret, chiffrez vos besoins. Si l'écart entre l'objectif visé et le budget détaillé dans le texte dépasse **30%**, le projet sera **rejeté automatiquement**.

Informations Générales (Brouillon)

Titre du projet *

Ex. Ferme solaire pour une école

Description détaillée * [Insérer un modèle](#)

Expliquez le problème, la solution, le budget, et l'impact attendu...

Un projet bien structuré inspire confiance et facilite la validation par l'IA. 0/5000

Catégorie *

Choisir une catégorie...

Aperçu en direct

APERÇU (BROUILLON)

Titre du projet

Catégorie

Démarre : 26/05/2026 · Échéance : 26/06/2026

La description apparaîtra ici. 0 / 21053 TND 0%

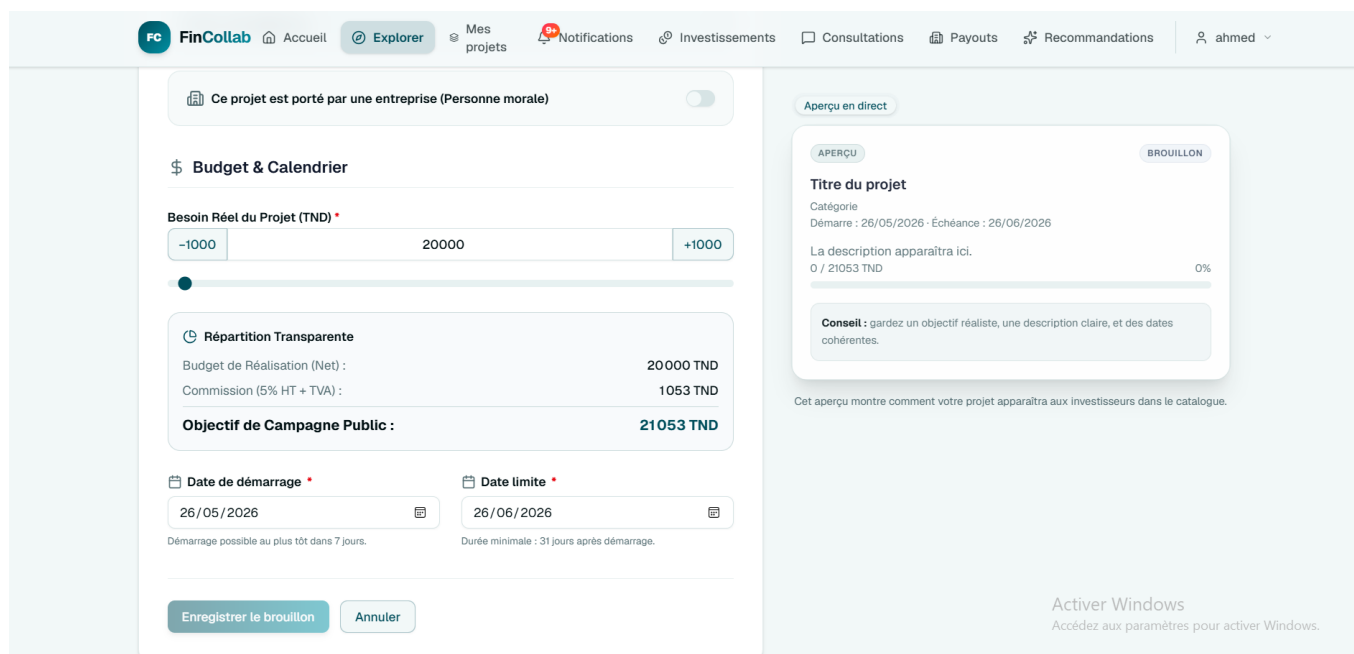
Conseil : gardez un objectif réaliste, une description claire, et des dates cohérentes.

Activater Windows

Accédez aux paramètres pour activer Windows.

Cet aperçu montre comment votre projet apparaîtra aux investisseurs dans le catalogue.

FIGURE 21 – Création de projet — étape 1



Budget & Calendrier

Ce projet est porté par une entreprise (Personne morale) ☐

Besoin Réel du Projet (TND) *

-1000 20000 +1000

Répartition Transparente

Budget de Réalisation (Net) : 20000 TND

Commission (5% HT + TVA) : 1053 TND

Objectif de Campagne Public : 21053 TND

Date de démarrage * 26/05/2026

Date limite * 26/06/2026

Démarrage possible au plus tôt dans 7 jours. Durée minimale : 31 jours après démarrage.

Enregistrer le brouillon Annuler

Aperçu en direct

APERÇU (BROUILLON)

Titre du projet

Catégorie

Démarre : 26/05/2026 · Échéance : 26/06/2026

La description apparaîtra ici. 0 / 21053 TND 0%

Conseil : gardez un objectif réaliste, une description claire, et des dates cohérentes.

Activater Windows

Accédez aux paramètres pour activer Windows.

Cet aperçu montre comment votre projet apparaîtra aux investisseurs dans le catalogue.

FIGURE 22 – Création de projet — étape 2 (compléments du formulaire)

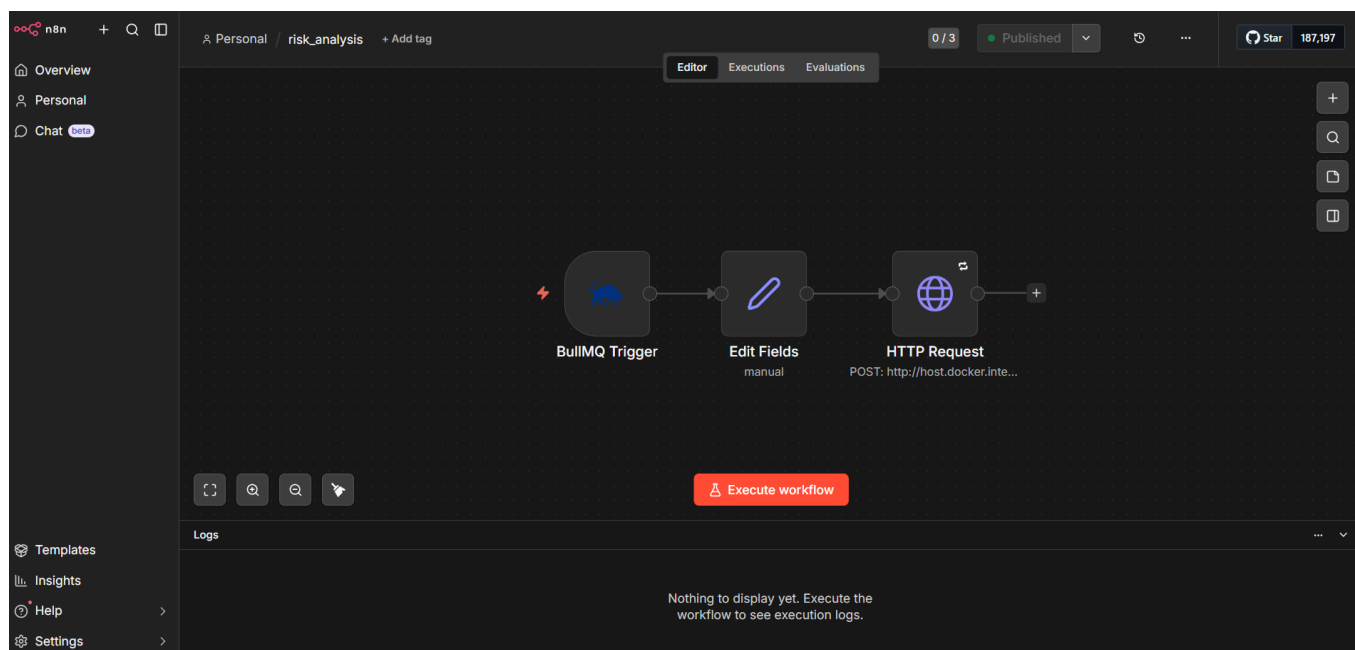


FIGURE 23 – Workflow n8n — analyse de risque

Extrait de code (VS Code — projectService.js)

```

345 async function submitProjectForAI(creatorId, projectId) {
346   const project = await Project.findOne({
347     _id: projectId,
348     creatorId,
349   });
350   if (!project) {
351     throw new HttpError(404, "Projet introuvable.");
352   }
353   if (project.status !== ProjectStatus.DRAFT) {
354     throw new HttpError(
355       409,
356       "Soumission impossible : seul un projet en brouillon (DRAFT) peut être soumis.",
357       { expected: ProjectStatus.DRAFT, actual: project.status },
358       "INVALID_PROJECT_STATE"
359     );
360   }
361   if (project.isArchived) {
362     throw new HttpError(400, "Soumission impossible : le projet est archivé.");
363   }
364   const deadline = new Date(project.deadline);
365   deadline.setHours(0, 0, 0, 0);
366   const today = new Date();
367   today.setHours(0, 0, 0, 0);
368   if (deadline < today) {
369     throw new HttpError(400, "La date limite doit être dans le futur.");
370   }
371   try { // Tenter d'enfiler le travail d'analyse d'abord (sans modifier le statut DRAFT en BD)
372     const enq = await enqueueRiskAnalysisJob(project); // Si l'enfilement réussit, on met à jour le statut en AWAITING_AI
373     transitionProjectStatus(project, ProjectStatus.AWAITING_AI, { action: "SUBMIT_FOR_AI" });
374     project.aiStatus = AiStatus.PENDING;
375     project.aiQueuedAt = new Date();
376     project.aiLastError = "";
377     if (enq && enq.queued && enq.jobId) {
378       project.aiJobId = String(enq.jobId);
379     } else {
380       project.aiJobId = "";
381     }

```

FIGURE 24 – Soumission d'un projet DRAFT vers la file d'analyse IA

Bibliothèques et composants

- mongoose, zod, file risk-analysis-queue (BullMQ/Redis), axios (appels internes signés HMAC), cron retryStuckAiAnalyses, DLQ failedWorkflowEvents.

4.3.3 Investir dans un projet (paiement simulé)

Lien avec la conception

Ce cas correspond aux figures 7 et 8 (chapitre 3).

Description

L'investisseur saisit montant et pourboire optionnel (figures 25–27) ; le serveur crée l'investissement INITIATED (figures 28–30). Après confirmation du paiement démo, le *webhook* signé finalise le dossier : branche PENDING_CONSULTATION si le seuil de 30 % est atteint (figure 31), sinon mise à jour atomique de *currentFunding* (figure 32).

Captures d'écran

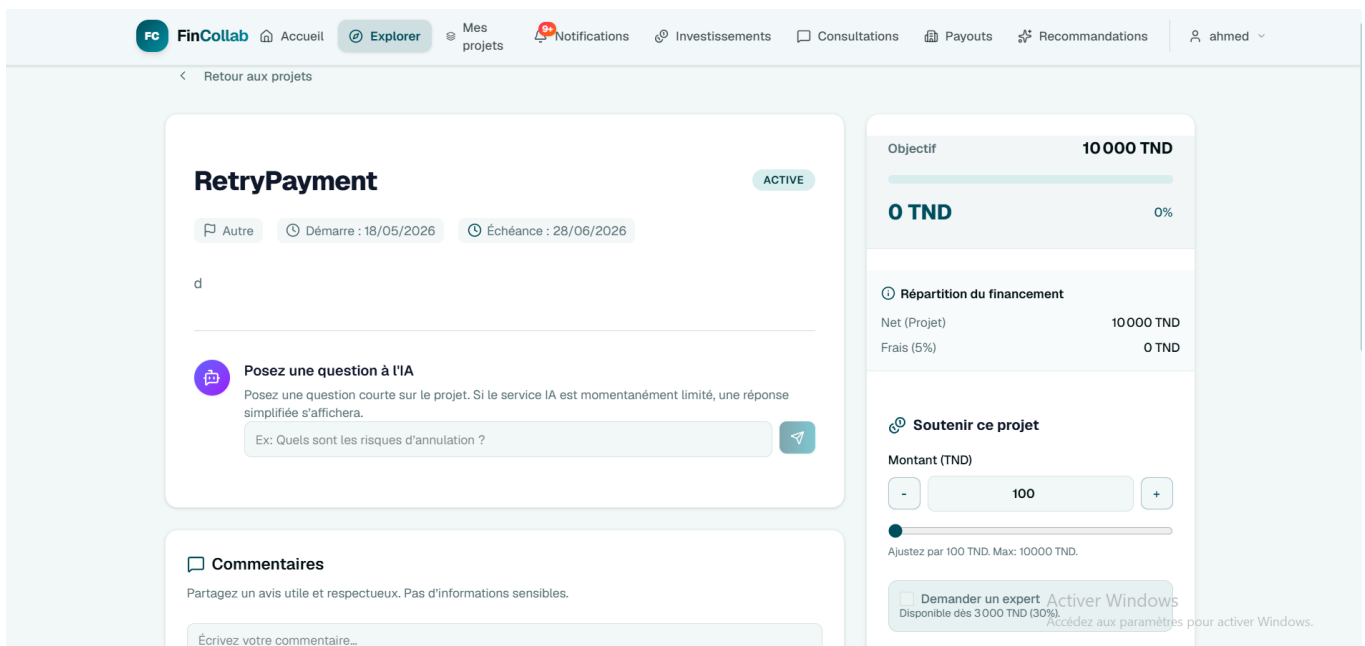
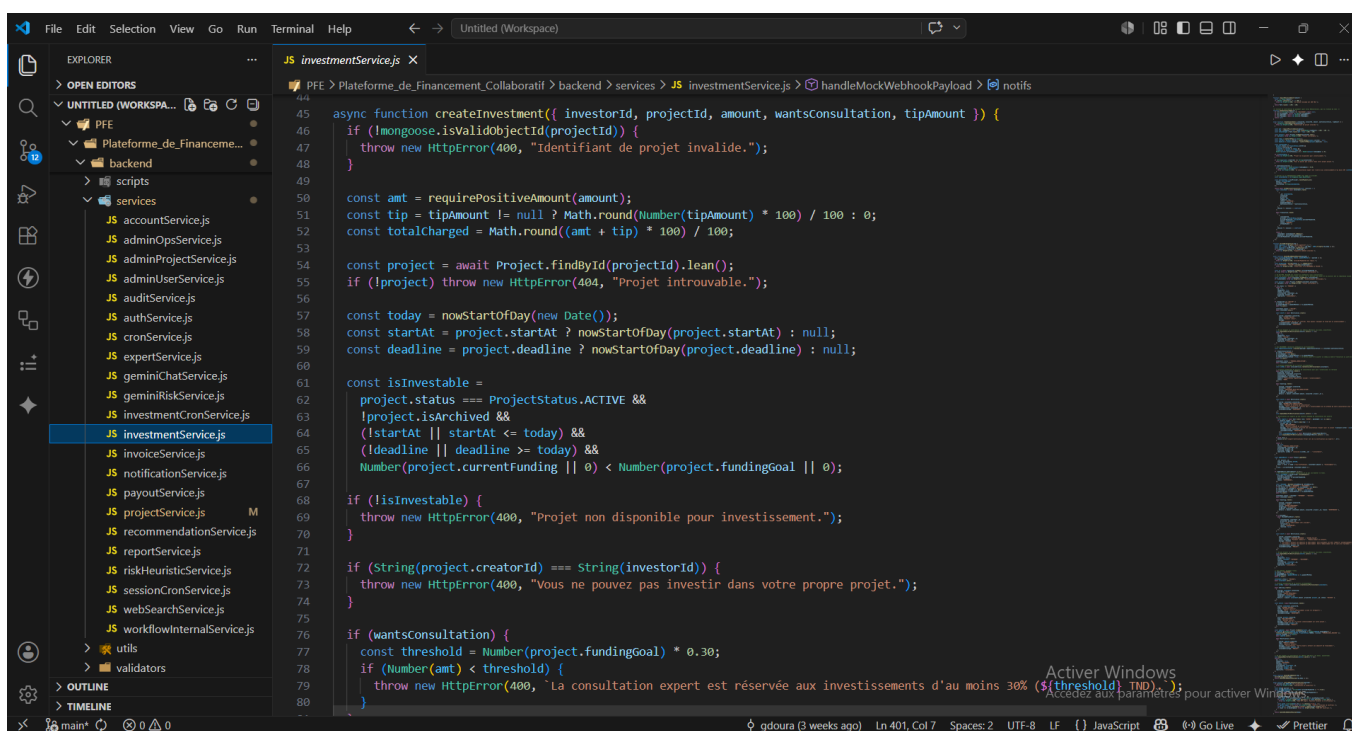


FIGURE 25 – Soutien — saisie du montant et du pourboire

FIGURE 26 – Soutien — confirmation du mode de paiement

FIGURE 27 – Confirmation OTP (d  mo paiement)

Extrait de code (VS Code — investmentService.js)



```

44
45 async function createInvestment({ investorId, projectId, amount, wantsConsultation, tipAmount }) {
46   if (!mongoose.isValidObjectId(projectId)) {
47     throw new HttpError(400, "Identifiant de projet invalide.");
48   }
49
50   const amt = requirePositiveAmount(amount);
51   const tip = tipAmount != null ? Math.round(Number(tipAmount) * 100) / 100 : 0;
52   const totalCharged = Math.round((amt + tip) * 100) / 100;
53
54   const project = await Project.findById(projectId).lean();
55   if (!project) throw new HttpError(404, "Projet introuvable.");
56
57   const today = nowStartOfDay(new Date());
58   const startAt = project.startAt ? nowStartOfDay(project.startAt) : null;
59   const deadline = project.deadline ? nowStartOfDay(project.deadline) : null;
60
61   const isInvestable =
62     project.status === ProjectStatus.ACTIVE &&
63     !project.isArchived &&
64     (!startAt || startAt <= today) &&
65     (!deadline || deadline >= today) &&
66     Number(project.currentFunding || 0) < Number(project.fundingGoal || 0);
67
68   if (!isInvestable) {
69     throw new HttpError(400, "Projet non disponible pour investissement.");
70   }
71
72   if (String(project.creatorId) === String(investorId)) {
73     throw new HttpError(400, "Vous ne pouvez pas investir dans votre propre projet.");
74   }
75
76   if (wantsConsultation) {
77     const threshold = Number(project.fundingGoal) * 0.30;
78     if (Number(amt) < threshold) {
79       throw new HttpError(400, "La consultation expert est réservée aux investissements d'au moins 30% (${{threshold}} TND).");
80     }
81   }

```

FIGURE 28 – Création d'investissement : contrôles métier

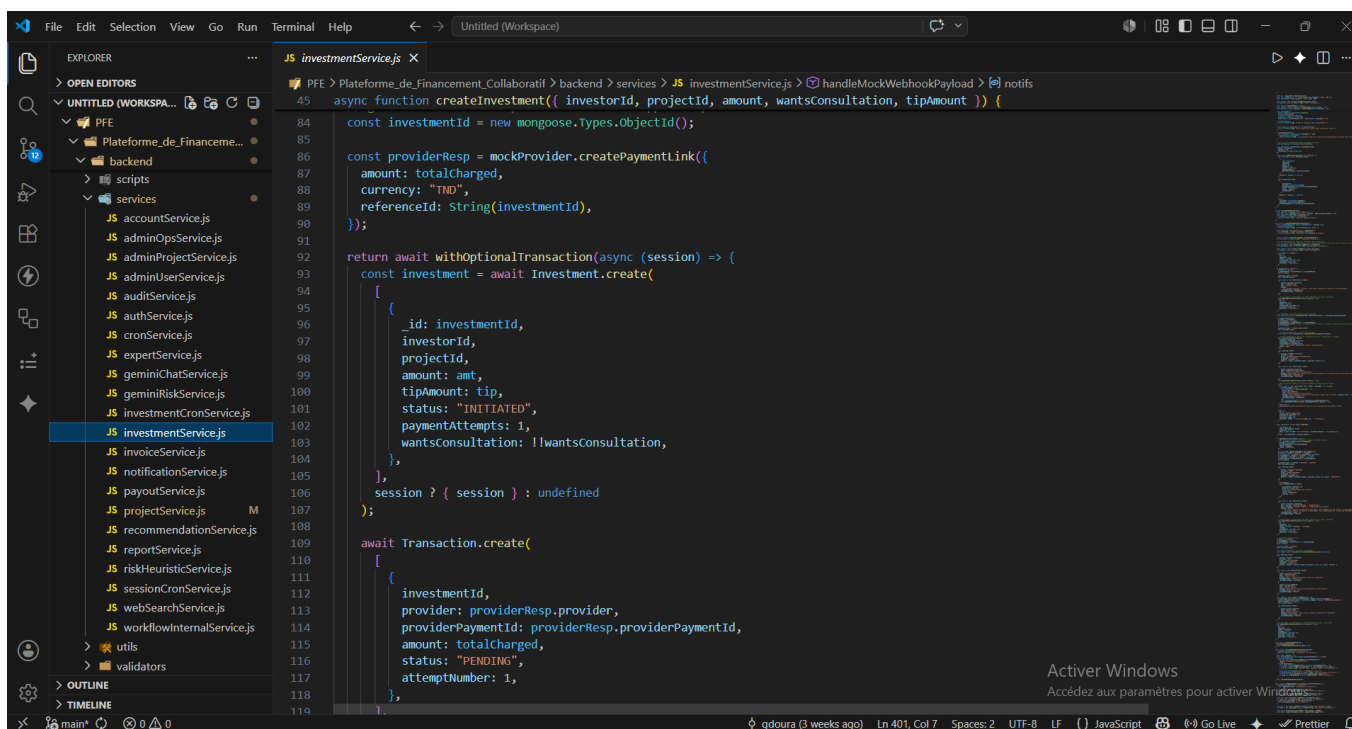


FIGURE 29 – Création d'investissement : seuil consultation 30 %

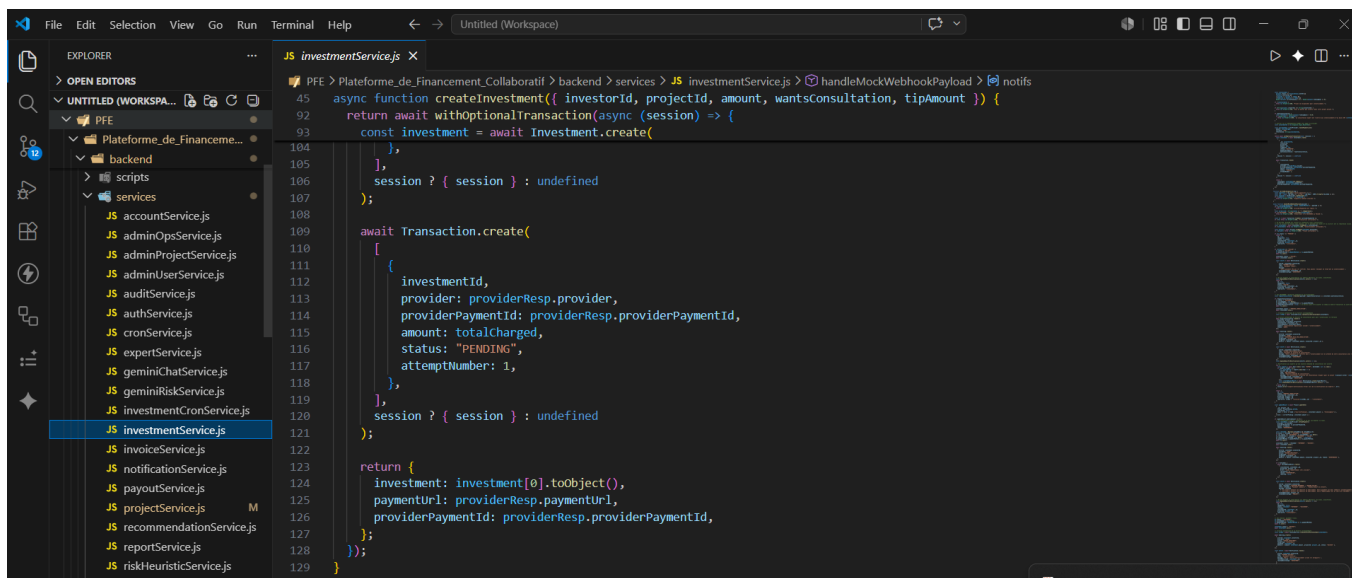


FIGURE 30 – Création d'investissement : enregistrement INITIATED / transaction PENDING

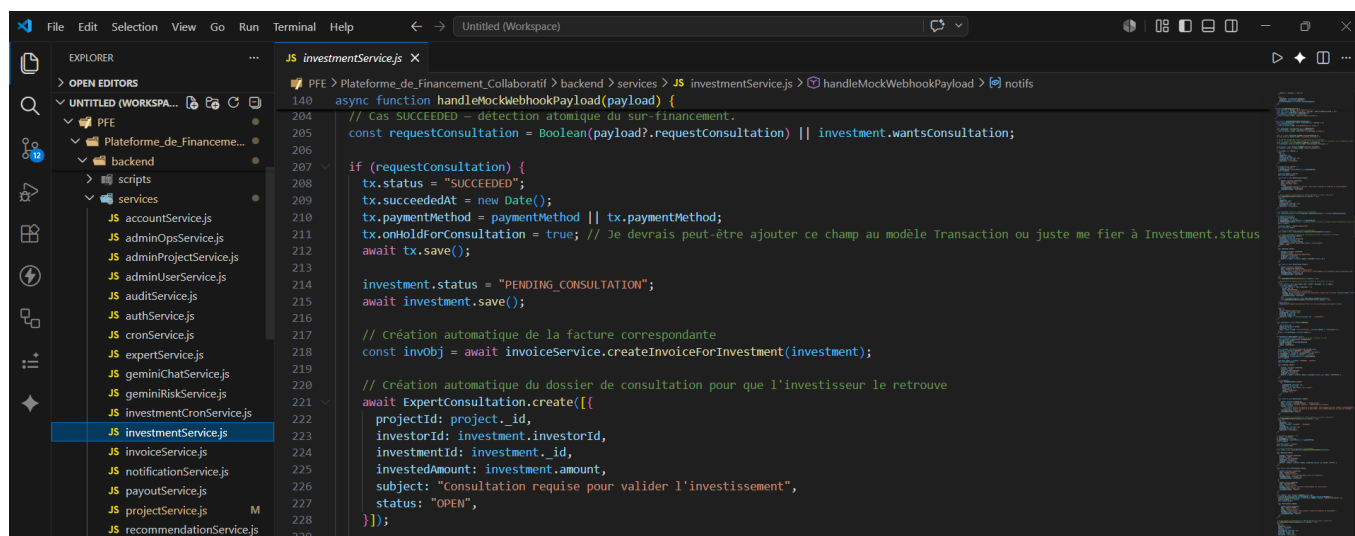


FIGURE 31 – Webhook : statut PENDING_CONSULTATION et dossier expert

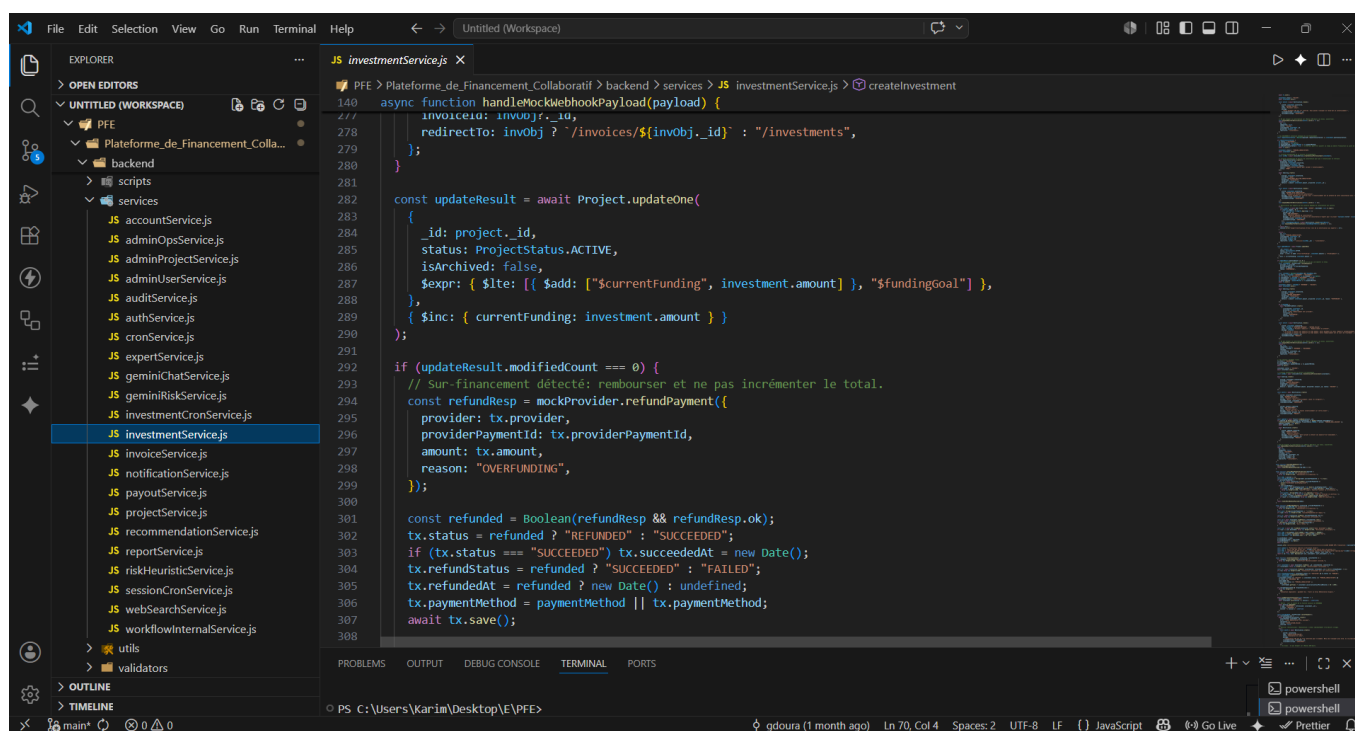


FIGURE 32 – Webhook : mise à jour atomique de currentFunding (anti-sur-financement)

Bibliothèques et composants

- mongoose, crypto (HMAC webhook), mock provider, invoiceService, file refund-queue, commission plateforme 5 % + pourboire.

4.3.4 Valider ou rejeter un projet (Expert et Administrateur)

Lien avec la conception

Ce cas correspond aux figures 9 et 10 (chapitre 3).

Description

L'expert statue sur un projet UNDER_REVIEW dont l'analyse IA est terminée (APPROVED ou REJECTED, figures 33–38). L'administrateur publie ensuite les projets APPROVED en ACTIVE depuis la console admin (figure 36, route POST /api/admin/projects/:id/publish).

Captures d'écran

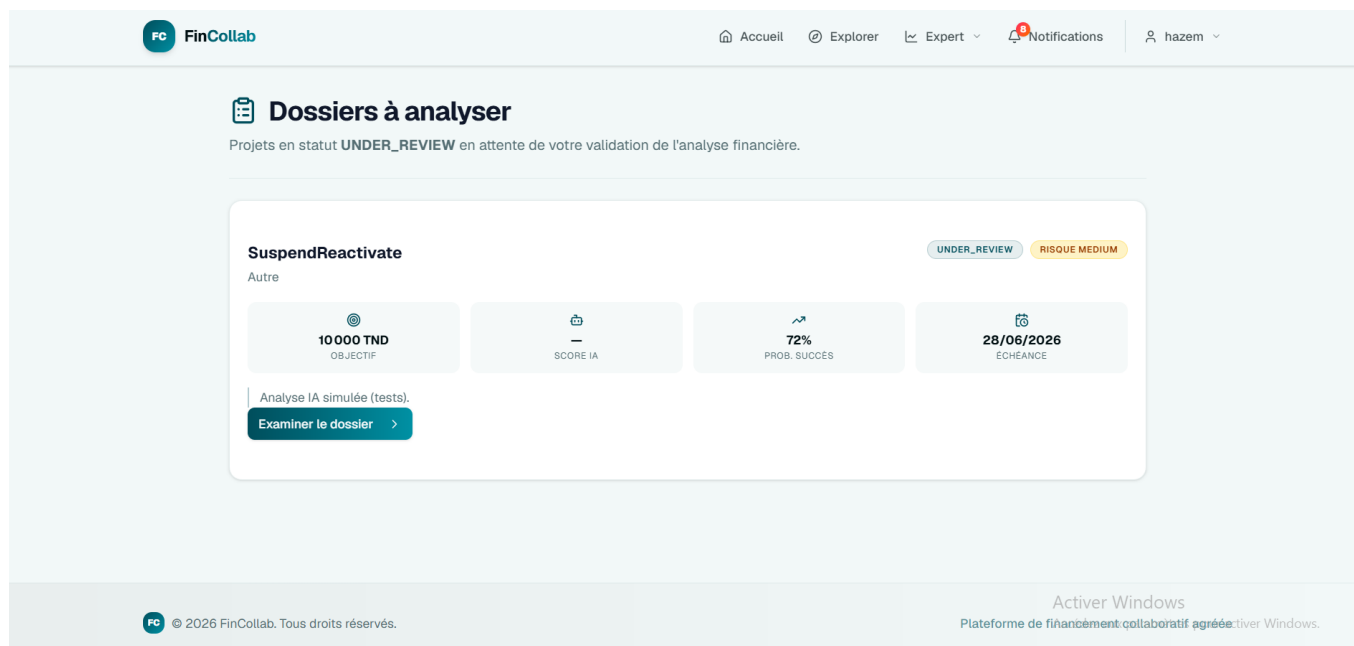


FIGURE 33 – Expert — liste des projets en revue

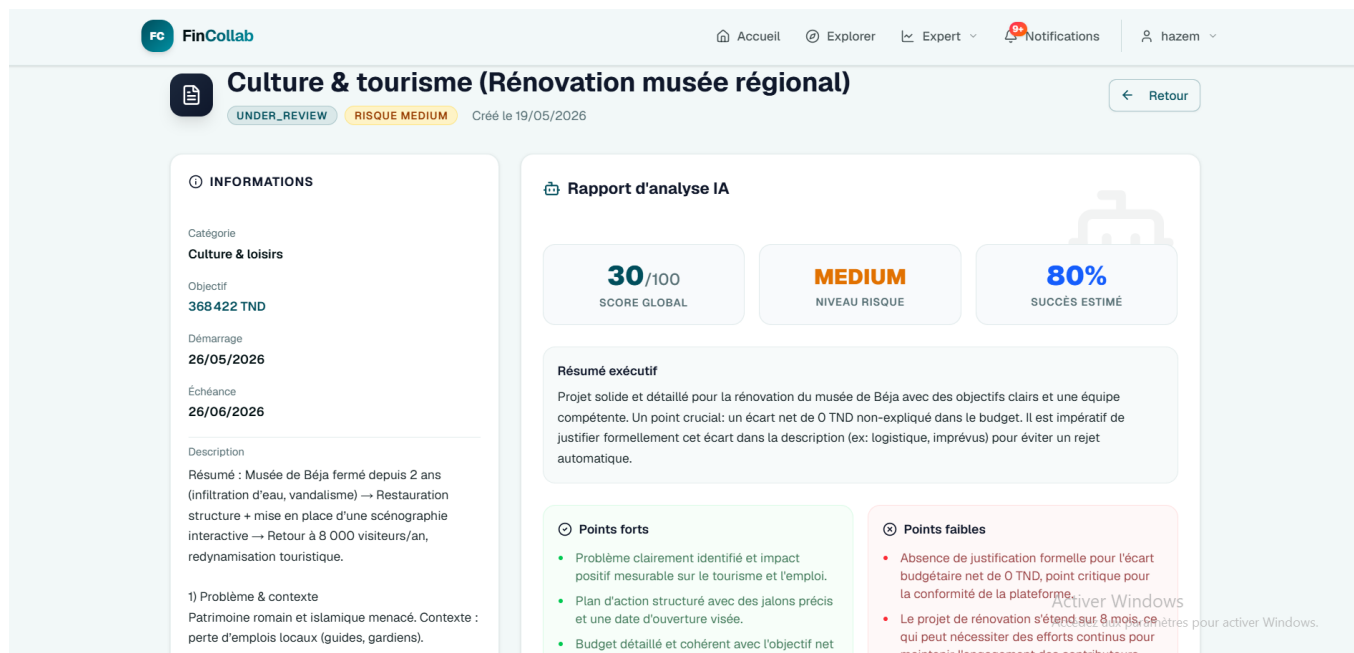


FIGURE 34 – Expert — détail de l'analyse IA avant décision

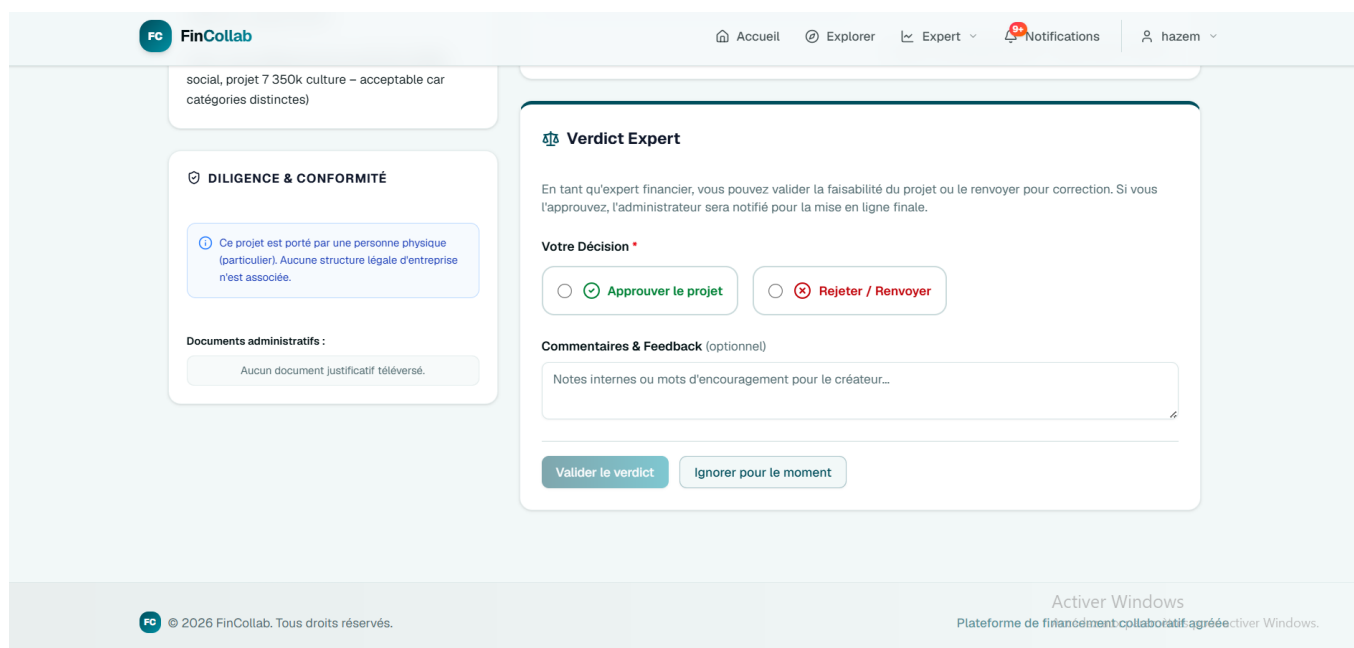


FIGURE 35 – Expert — décision de validation ou de rejet

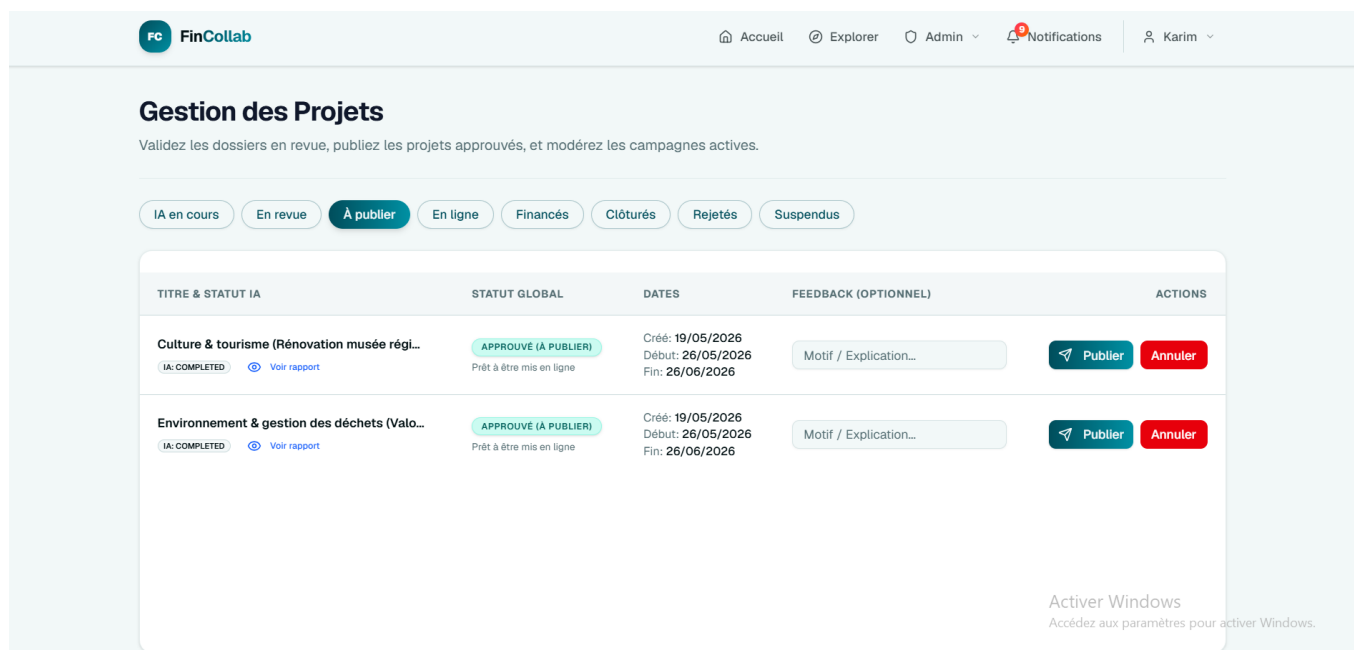


FIGURE 36 – Administrateur — publication finale

Extrait de code (VS Code — expertService.js)

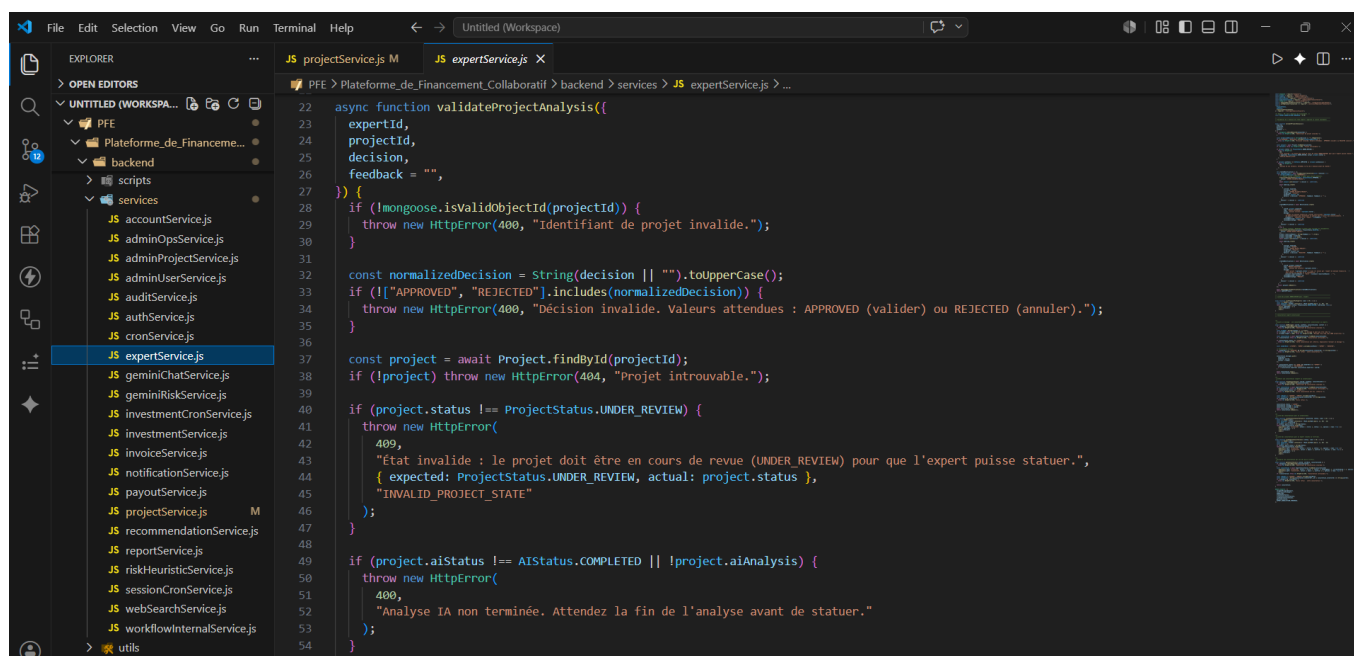


FIGURE 37 – Validation expert : contrôles et décision APPROVED

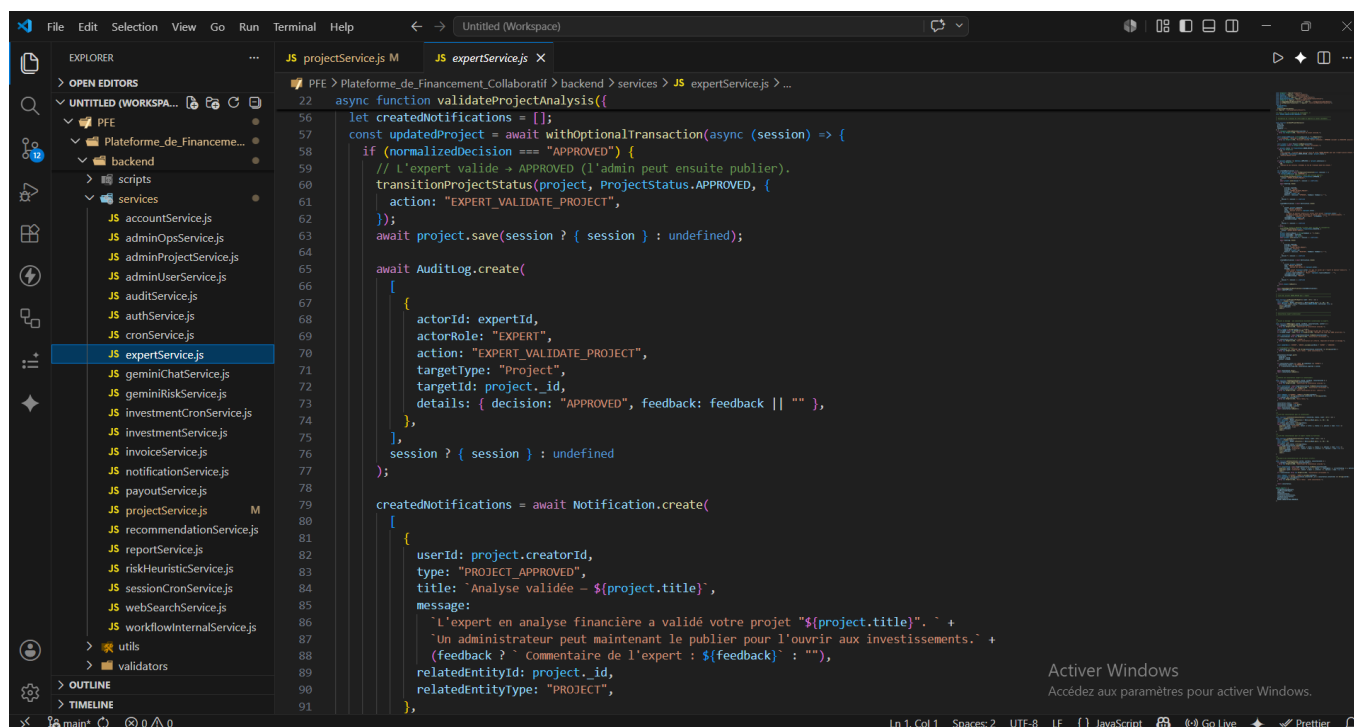


FIGURE 38 – Validation expert : notifications et branche REJECTED

Bibliothèques et composants

- mongoose (transactions optionnelles), AuditLog, Notification, routes `/api/expert/projects/:id/validate`, `/api/admin/projects/:id/publish`; voie de test de bout en bout `/api/admin/projects/:id/validate`.

4.3.5 Coordonnées bancaires et demande de retrait

Lien avec la conception

Ce cas correspond aux figures 11 et 12 (chapitre 3).

Description

Après clôture réussie d'un projet, le porteur initie un *payout* au statut PENDING, puis saisit son RIB sur l'écran dédié (figure 39). Le service `provideBankDetails` valide la saisie, scelle les données (AES-256-GCM) et fait passer le retrait en READY (figure 40); le RIB ne transite jamais en clair dans les listes admin.

Capture d'écran

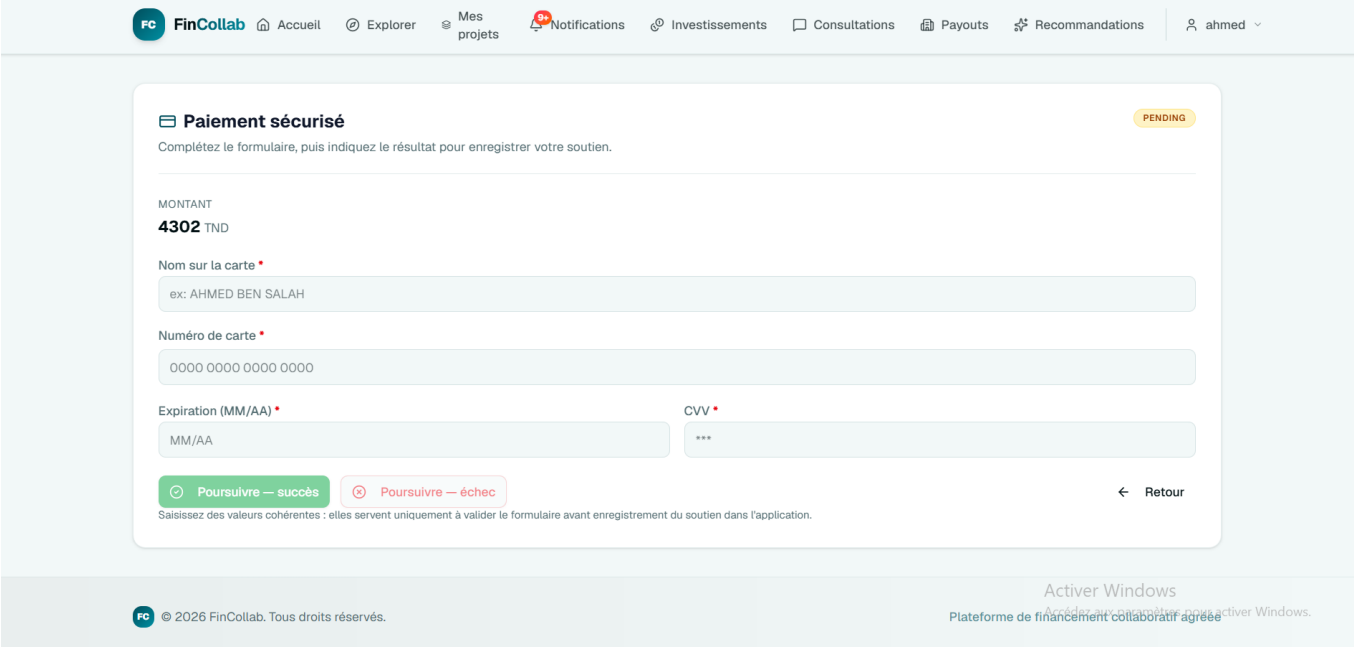
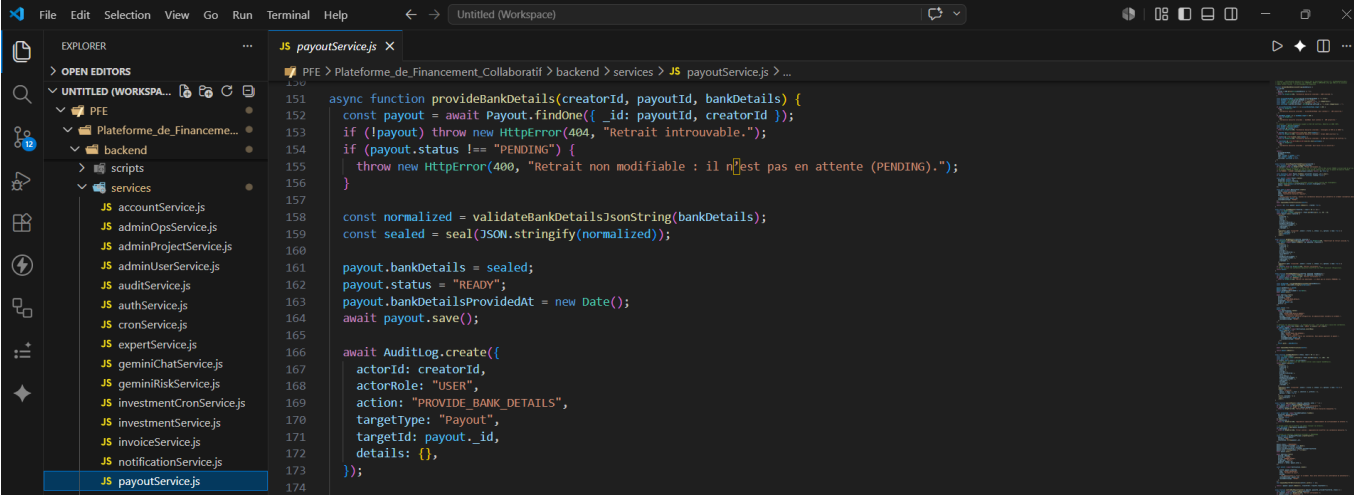


FIGURE 39 – Saisie des coordonnées bancaires

Extrait de code (VS Code — payoutService.js)



```

151 async function provideBankDetails(creatorId, payoutId, bankDetails) {
152   const payout = await Payout.findOne({ _id: payoutId, creatorId });
153   if (!payout) throw new HttpError(404, "Retrait introuvable.");
154   if (payout.status !== "PENDING") {
155     throw new HttpError(400, "Retrait non modifiable : il n'est pas en attente (PENDING).");
156   }
157
158   const normalized = validateBankDetailsJsonString(bankDetails);
159   const sealed = seal(JSON.stringify(normalized));
160
161   payout.bankDetails = sealed;
162   payout.status = "READY";
163   payout.bankDetailsProvidedAt = new Date();
164   await payout.save();
165
166   await Auditlog.create({
167     actorId: creatorId,
168     actorRole: "USER",
169     action: "PROVIDE BANK DETAILS",
170     targetType: "Payout",
171     targetId: payout._id,
172     details: {},
173   });
174 }

```

FIGURE 40 – Chiffrement du RIB (seal) et passage du retrait en READY

Bibliothèques et composants

- Module `seal/open` (AES-256-GCM), `zod`, `AuditLog`, route `PUT/api/payouts/:id/bank-details`; l'approbation admin et la facture de retrait sont traitées au § 4.3.6.

4.3.6 Approuver un retrait (Administrateur)

Lien avec la conception

Ce cas correspond aux figures 13 et 14 (chapitre 3). Il fait suite à la fourniture des coordonnées bancaires (§ 4.3.5) : l'administrateur n'approuve qu'un Payout au statut `READY` avec RIB chiffré présent.

Description

L'administrateur valide le virement simulé : déchiffrement interne du RIB (jamais renvoyé en clair), appel au prestataire mock, passage `READY` → `PROCESSING`, journal d'audit `APPROVE_PAYOUT`, notification `PAYOUT_PROCESSING` au porteur. La confirmation finale (`COMPLETED`) est traitée par `confirmMockPayoutTransfer` et crée la facture numérique associée (export PDF via l'interface), comme prévu sur la séquence du chapitre 3.

Captures d'écran

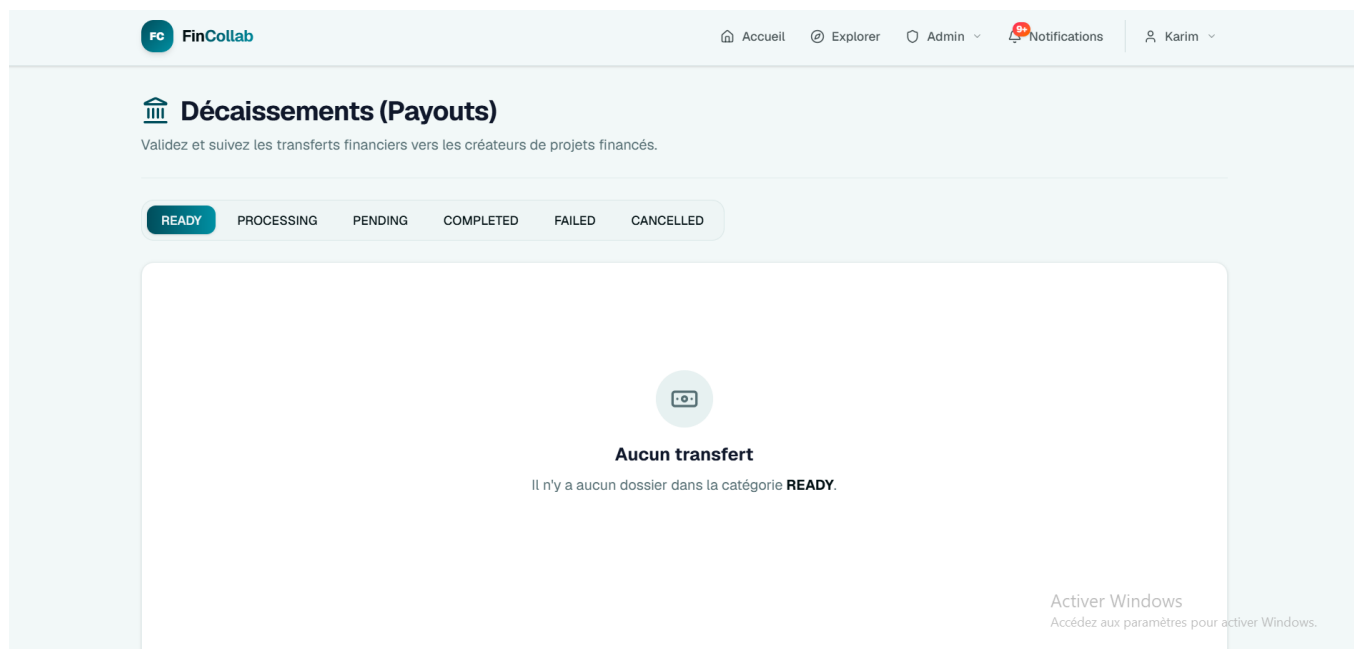


FIGURE 41 – Console administrateur — retrait en statut READY (approbation possible)

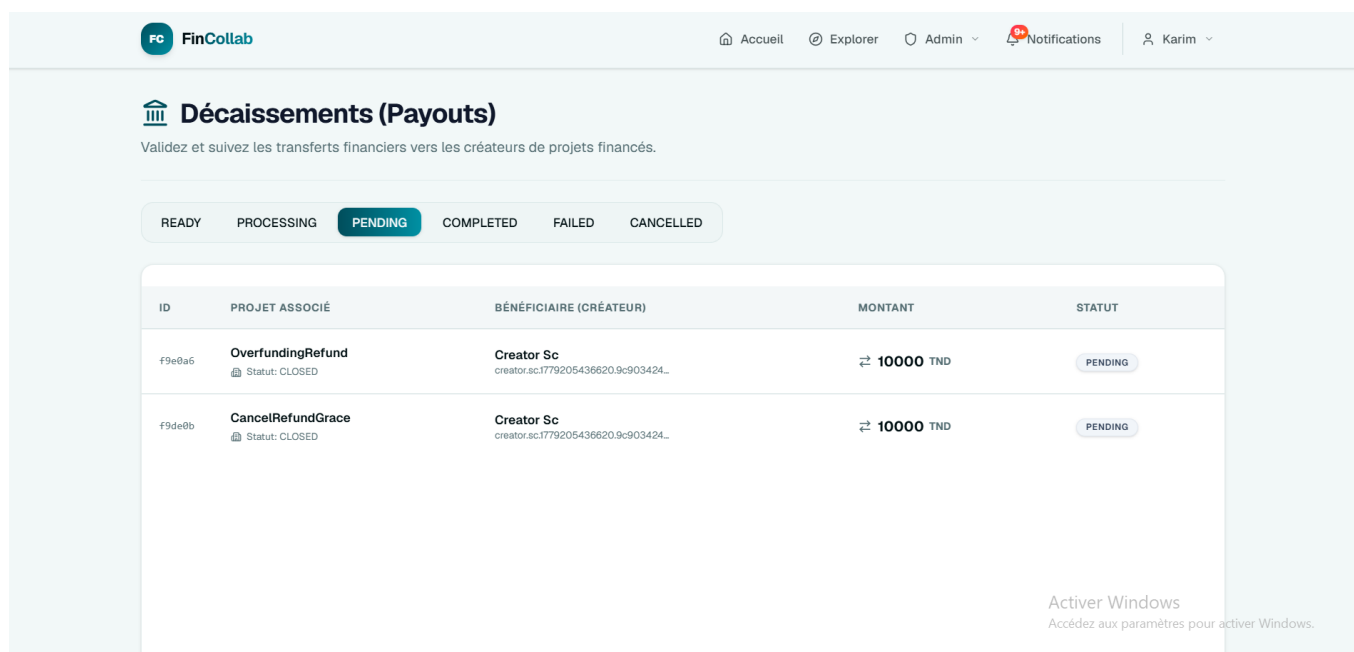
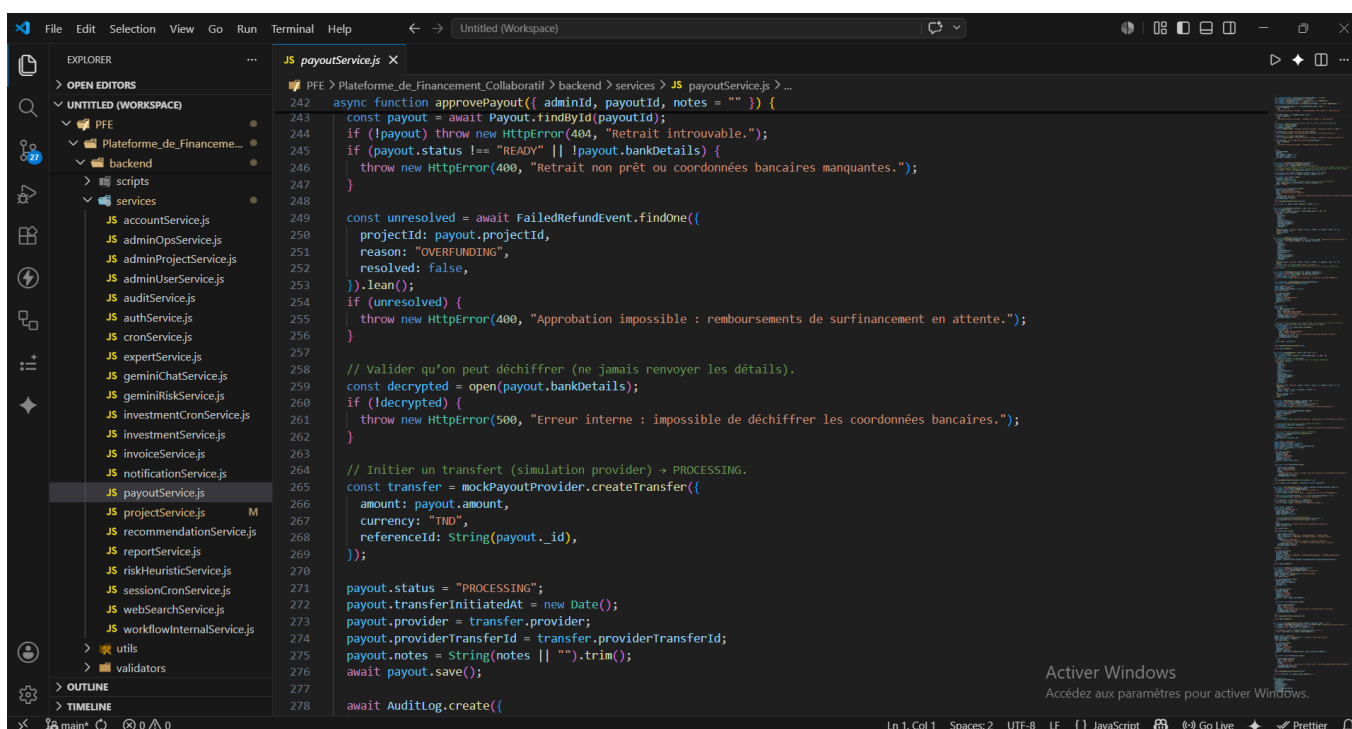


FIGURE 42 – Console administrateur — retrait après approbation (PROCESSING / COMPLETED)

Extrait de code (VS Code — payoutService.js)

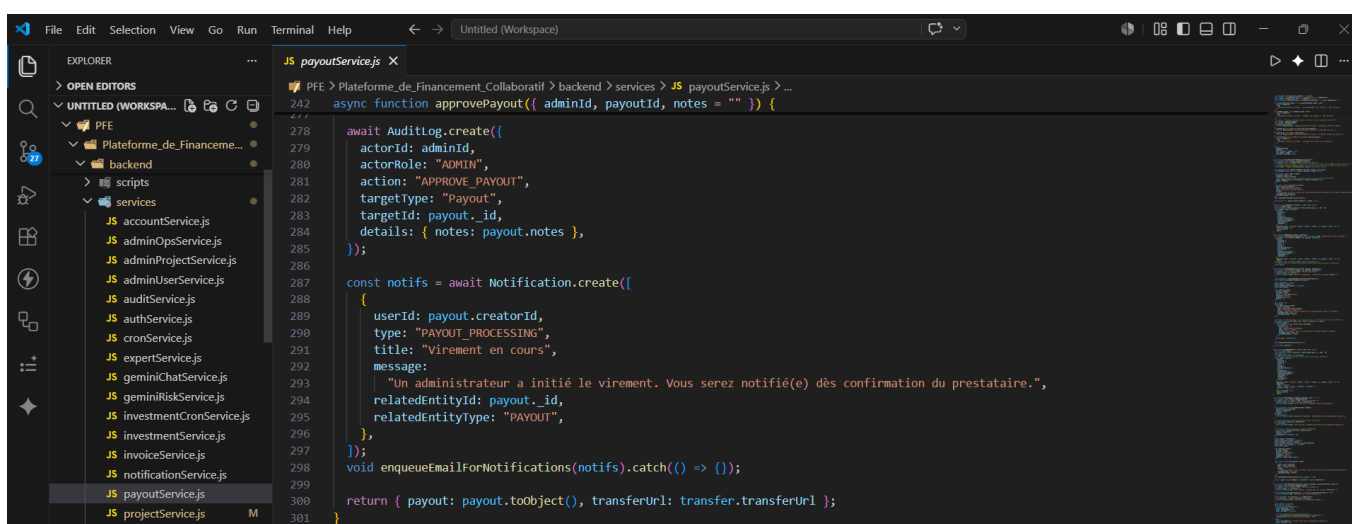


```

242 async function approvePayout({ adminId, payoutId, notes = "" }) {
243   const payout = await Payout.findById(payoutId);
244   if (!payout) throw new HttpError(404, "Retrait introuvable.");
245   if (payout.status !== "READY" || !payout.bankDetails) {
246     throw new HttpError(400, "Retrait non prêt ou coordonnées bancaires manquantes.");
247   }
248
249   const unresolved = await FailedRefundEvent.findOne({
250     projectId: payout.projectId,
251     reason: "OVERFUNDING",
252     resolved: false,
253   });
254   if (unresolved) {
255     throw new HttpError(400, "Approbation impossible : remboursements de surfinancement en attente.");
256   }
257
258   // Valider qu'on peut déchiffrer (ne jamais renvoyer les détails).
259   const decrypted = open(payout.bankDetails);
260   if (!decrypted) {
261     throw new HttpError(500, "Erreur interne : impossible de déchiffrer les coordonnées bancaires.");
262   }
263
264   // Initier un transfert (simulation provider) → PROCESSING.
265   const transfer = mockPayoutProvider.createTransfer({
266     amount: payout.amount,
267     currency: "TND",
268     referenceId: String(payout._id),
269   });
270
271   payout.status = "PROCESSING";
272   payout.transferInitiatedAt = new Date();
273   payout.provider = transfer.provider;
274   payout.providerTransferId = transfer.providerTransferId;
275   payout.notes = String(notes || "").trim();
276   await payout.save();
277
278   await Auditlog.create({

```

FIGURE 43 – Approbation admin : contrôles et initiation du transfert simulé



```

278   await Auditlog.create({
279     actorId: adminId,
280     actorRole: "ADMIN",
281     action: "APPROVE_PAYOUT",
282     targetType: "Payout",
283     targetId: payout._id,
284     details: { notes: payout.notes },
285   });
286
287   const notifs = await Notification.create([
288     {
289       userId: payout.creatorId,
290       type: "PAYOUT_PROCESSING",
291       title: "Virement en cours",
292       message:
293         "Un administrateur a initié le virement. Vous serez notifié(e) dès confirmation du prestataire.",
294       relatedEntityId: payout._id,
295       relatedEntityType: "PAYOUT",
296     },
297   ]);
298   void enqueueEmailForNotifications(notifs).catch(() => {});
299
300   return { payout: payout.toObject(), transferUrl: transfer.transferUrl };
301 }

```

FIGURE 44 – Approbation admin : statut PROCESSING et notification

Bibliothèques et composants

- `payoutService.approvePayout`, `mockPayoutProvider`, `seal/open`, `invoiceService`, route `POST/api/admin/payouts/:id/approve`, script de test `e2e-lifecycle-payout-api.js`.

4.3.7 Suspendre ou réactiver un projet (Administrateur)

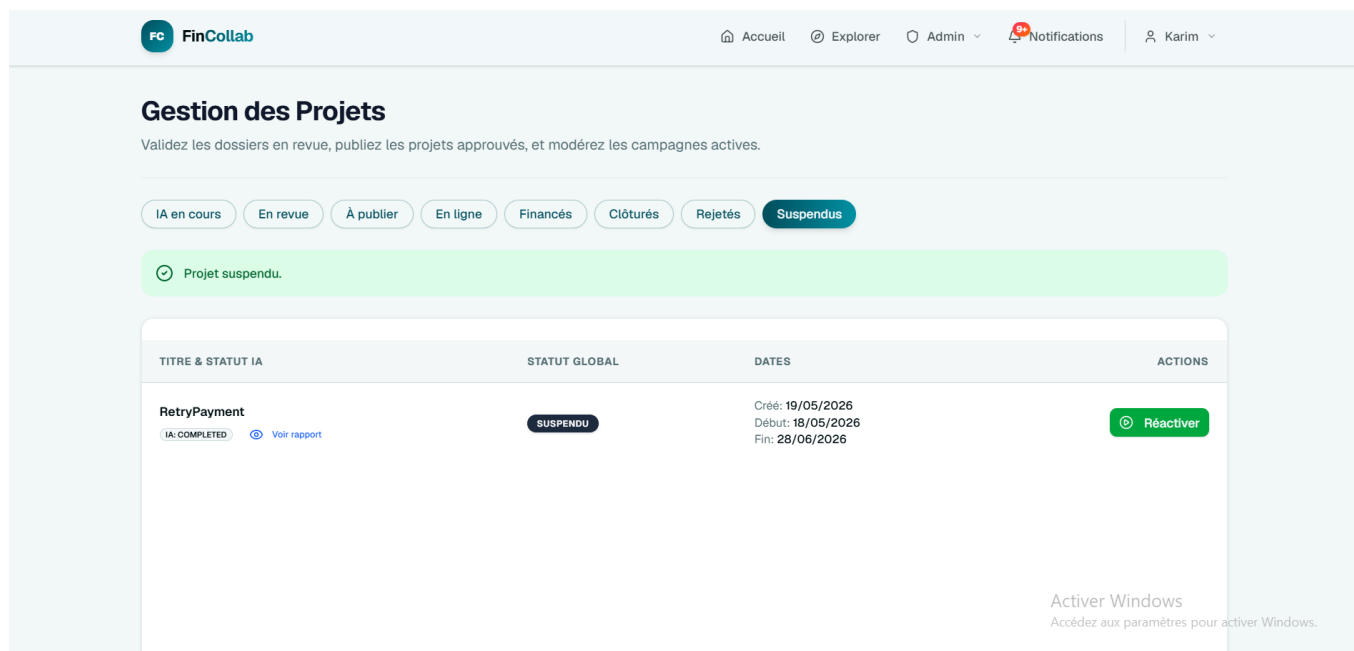
Lien avec la conception

Ce cas correspond aux figures 15 et 16 (chapitre 3).

Description

L'administrateur peut suspendre un projet actif (**ACTIVE** ou **FUNDED**) : transition vers **SUSPENDED**, motif enregistré, notification **PROJECT_SUSPENDED**, annulation au mieux d'un Payout ouvert. La réactivation n'est possible que depuis **SUSPENDED** ; selon la fraîcheur de l'analyse IA, le projet repasse en **UNDER_REVIEW** ou relance une analyse (**AWAITING_AI**).

Captures d'écran

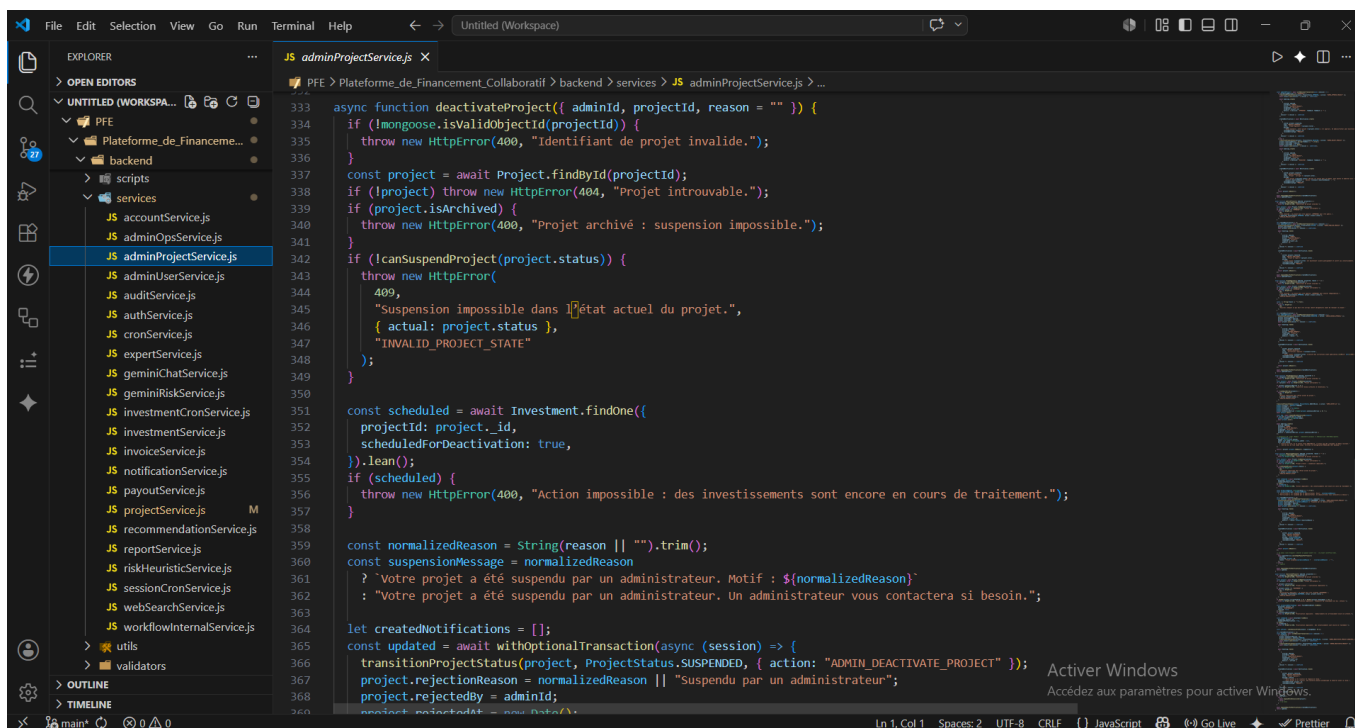


The screenshot shows the 'Gestion des Projets' (Project Management) section of the FinCollab administrator console. The user is logged in as 'Karim'. The interface includes a navigation bar with links to 'Accueil', 'Explorer', 'Admin', 'Notifications', and the user profile. Below the navigation bar, there's a section for 'Gestion des Projets' with a sub-header 'Validez les dossiers en revue, publiez les projets approuvés, et modérez les campagnes actives.' A row of filters allows selecting project status: 'IA en cours', 'En revue', 'À publier', 'En ligne', 'Financés', 'Clôturés', 'Rejetés', and 'Suspendus' (which is currently selected). A green notification bar states 'Projet suspendu.' Below this, a table lists projects. The table has columns for 'TITRE & STATUT IA', 'STATUT GLOBAL', 'DATES', and 'ACTIONS'. One project, 'RetryPayment', is shown with a status of 'SUSPENDU' and a 'Réactiver' button. The project details include 'Créé: 19/05/2026', 'Début: 18/05/2026', and 'Fin: 28/06/2026'. A watermark 'Activer Windows' is visible in the bottom right corner.

TITRE & STATUT IA	STATUT GLOBAL	DATES	ACTIONS
RetryPayment IA: COMPLETED Voir rapport	SUSPENDU	Créé: 19/05/2026 Début: 18/05/2026 Fin: 28/06/2026	Réactiver

FIGURE 45 – Console administrateur — suspension d'un projet (SUSPENDED)

Extrait de code (VS Code — adminProjectService.js)



```

333 async function deactivateProject({ adminId, projectId, reason = "" }) {
334   if (!mongoose.isValidObjectId(projectId)) {
335     throw new HttpError(400, "Identifiant de projet invalide.");
336   }
337   const project = await Project.findById(projectId);
338   if (!project) throw new HttpError(404, "Projet introuvable.");
339   if (project.isArchived) {
340     throw new HttpError(400, "Projet archivé : suspension impossible.");
341   }
342   if (!canSuspendProject(project.status)) {
343     throw new HttpError(
344       400,
345       "Suspension impossible dans l'état actuel du projet.",
346       { actual: project.status },
347       "INVALID_PROJECT_STATE"
348     );
349   }
350   const scheduled = await Investment.findOne({
351     projectId: project.id,
352     scheduledForDeactivation: true,
353   }).lean();
354   if (scheduled) {
355     throw new HttpError(400, "Action impossible : des investissements sont encore en cours de traitement.");
356   }
357   const normalizedReason = String(reason || "").trim();
358   const suspensionMessage = normalizedReason
359     ? `Votre projet a été suspendu par un administrateur. Motif : ${normalizedReason}`
360     : "Votre projet a été suspendu par un administrateur. Un administrateur vous contactera si besoin.";
361   let createdNotifications = [];
362   const updated = await withOptionalTransaction(async (session) => {
363     transitionProjectStatus(project, ProjectStatus.SUSPENDED, { action: "ADMIN_DEACTIVATE_PROJECT" });
364     project.rejectionReason = normalizedReason || "Suspendu par un administrateur";
365     project.rejectedBy = adminId;
366     project.rejectedAt = new Date();
367   });
368   project.rejectedAt = new Date();
  
```

FIGURE 46 – Suspension : contrôles et transition vers SUSPENDED (1/3)

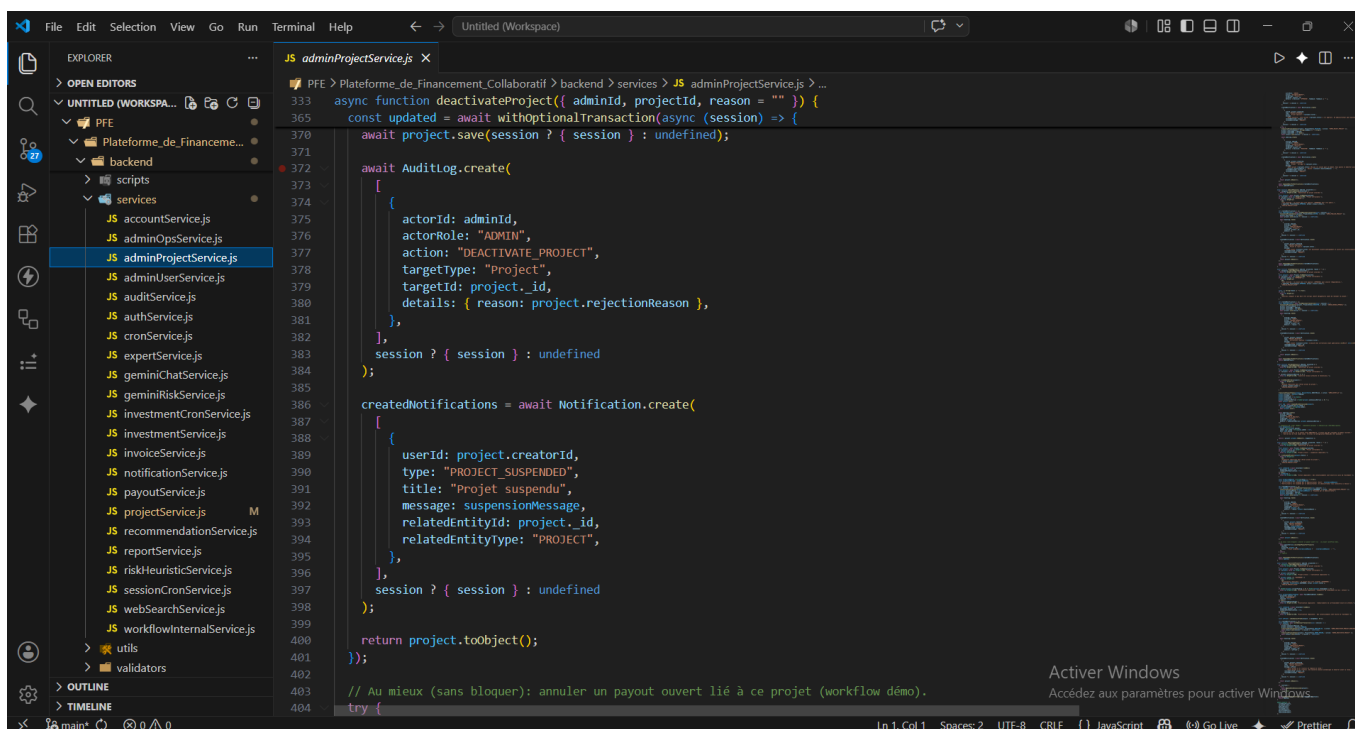


FIGURE 47 – Suspension : persistance et journal d’audit (2/3)

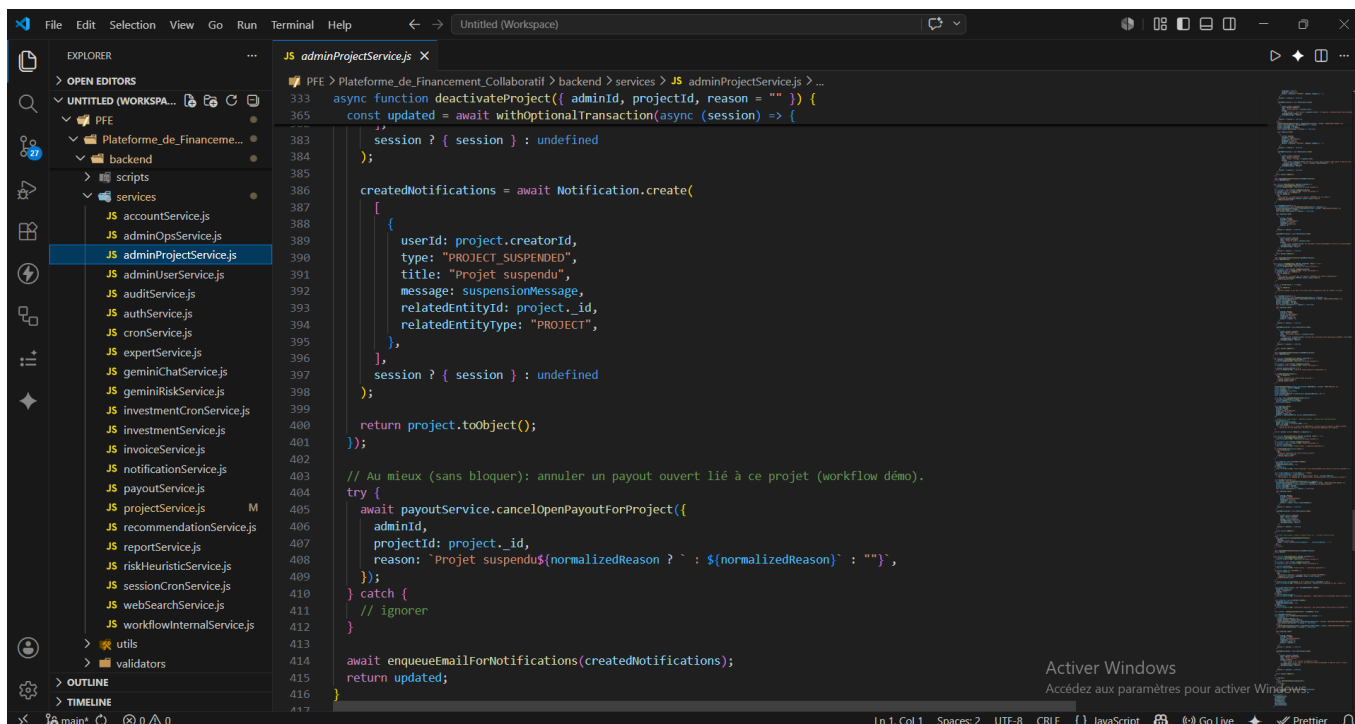


FIGURE 48 – Suspension : notification PROJECT_SUSPENDED (3/3)

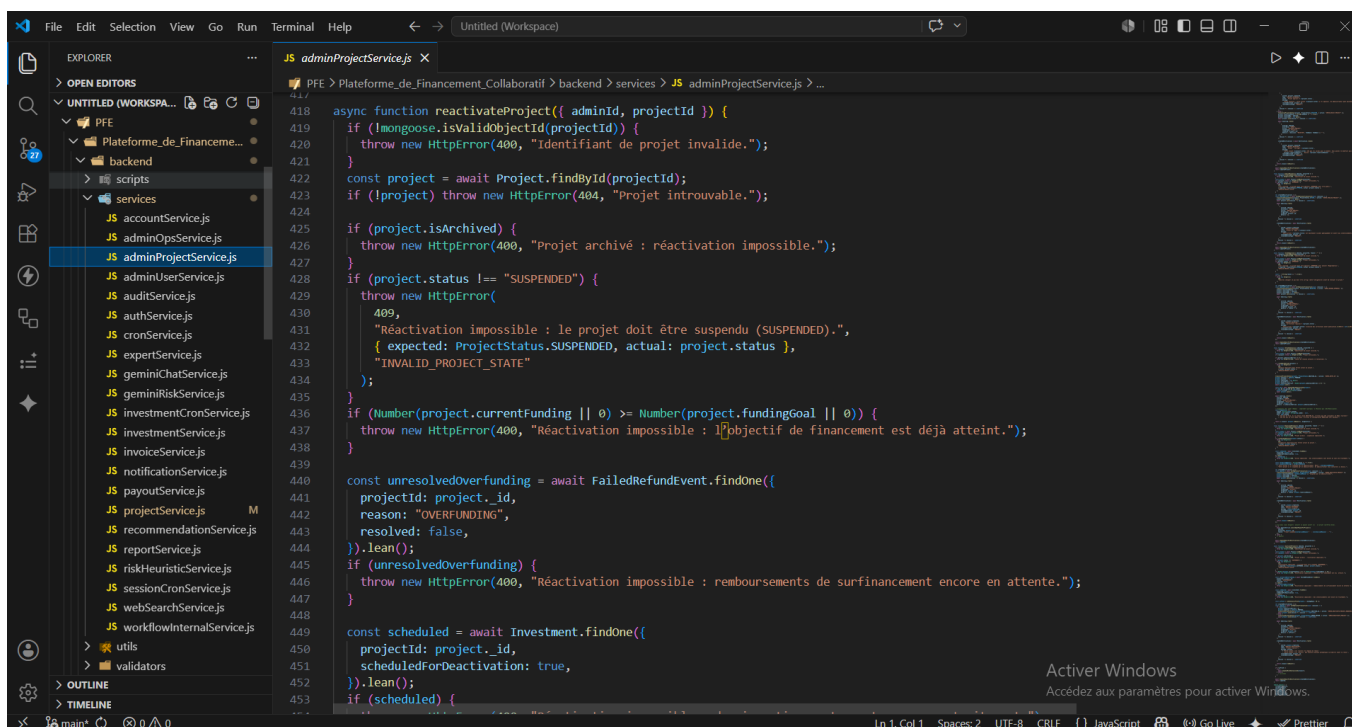


FIGURE 49 – Réactivation : contrôles préalables (1/3)

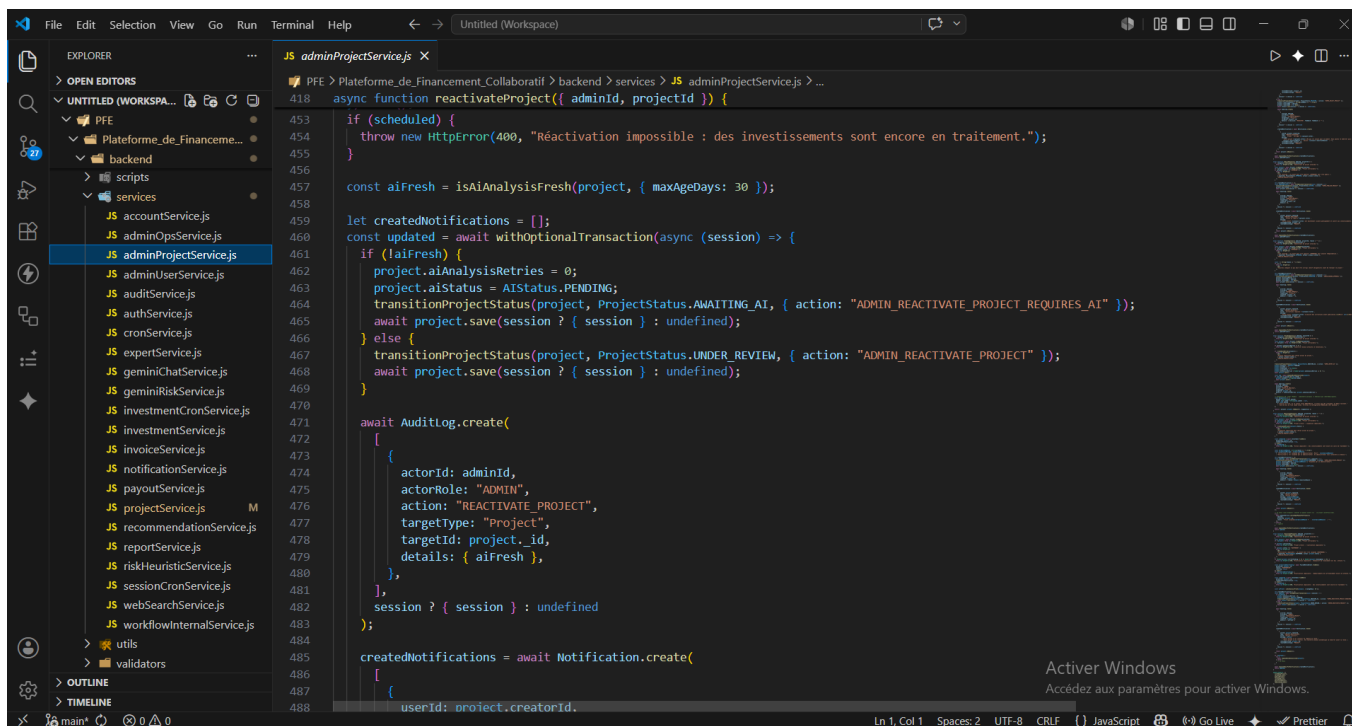


FIGURE 50 – Réactivation : transition UNDER_REVIEW ou AWAITING_AI (2/3)

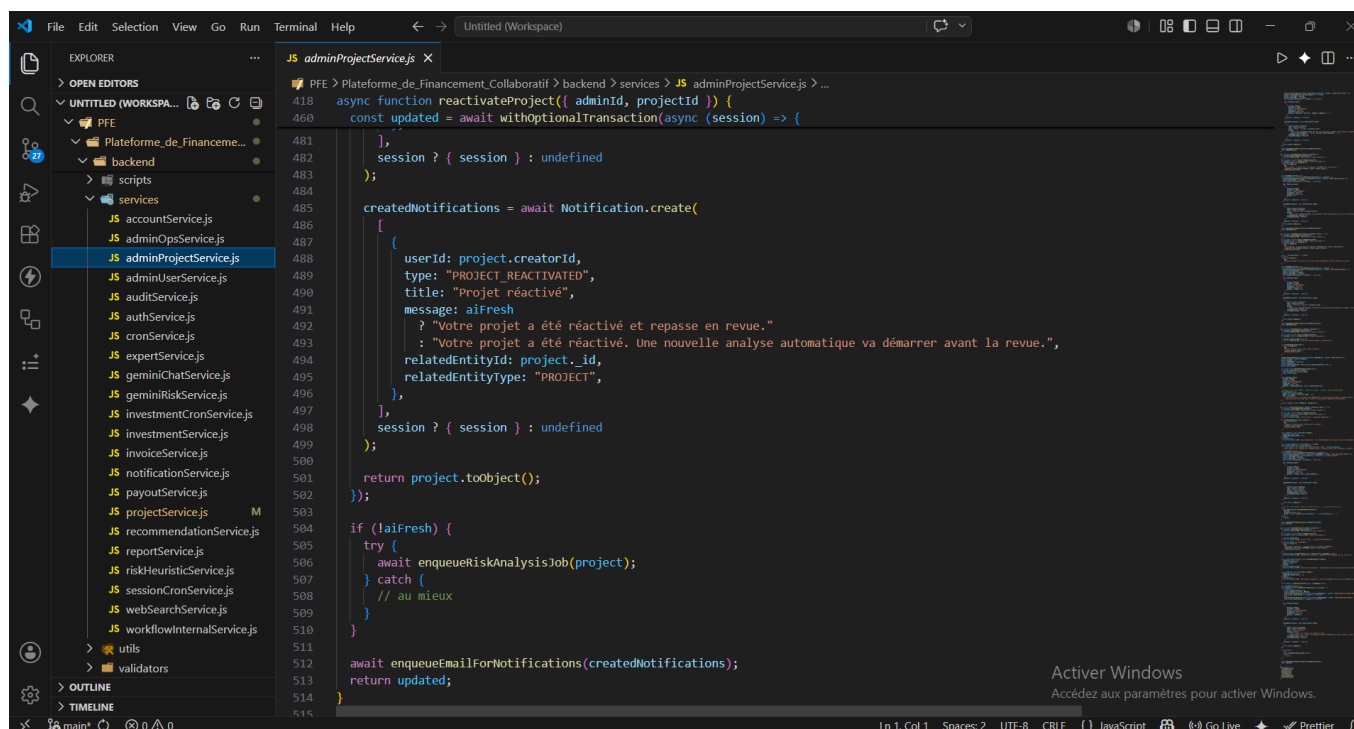


FIGURE 51 – Réactivation : notification PROJECT_REACTIVATED (3/3)

Bibliothèques et composants

- `adminProjectService.deactivateProject`, `reactivateProject`, `transitionProjectStatus`, `payoutService.cancelOpenPayoutForProject`; routes `/api/admin/projects/:id/deactivate` et `/api/admin/projects/:id/reactivate`.

4.4 Conclusion

Ce chapitre a exposé l'environnement de réalisation (section 4.2) puis la mise en œuvre technique de **sept cas d'utilisation** alignés sur le chapitre 3 (section 4.3 : captures d'écran, extraits de code, bibliothèques et composants, tableau 16). L'implémentation couvre les processus métier essentiels : cycle de vie des projets, contribution avec paiement simulé idempotent, analyse de risque orchestrée, gestion des échecs (DLQ) et administration. Le cycle de vie passe de DRAFT à ACTIVE, puis vers FUNDED, CLOSED ou SUSPENDED, avec une branche REJECTED. La cohérence a été vérifiée par des scripts E2E et des tests manuels (cf. § 4.2.6). La conclusion générale qui suit synthétise les apports, les limites et les perspectives d'évolution.

Conclusion générale

Travaux réalisés et solutions apportées

Le travail réalisé converge vers une plateforme web de financement participatif couvrant les processus métier essentiels : création et gestion des campagnes, contribution des investisseurs, gouvernance par validation expert puis publication administrative, traitement des retraits, complétés par des dispositifs visant la transparence (rapport d'analyse de risque, notifications, modération). Les principales réalisations se déclinent comme suit :

- Un **parcours complet** porteur de projet : brouillon, soumission à l'analyse, revue/validation, publication, suivi de la collecte, puis préparation du retrait.
- Un **parcours complet** investisseur : exploration, contribution, suivi des investissements et gestion des incidents (paiement échoué, annulation, remboursement).
- Des **aides à la décision** : analyse de risque produisant un rapport structuré et recommandations adaptées au profil.
- Une **gouvernance et conformité** renforcées : rôles, modération, signalements, journalisation des actions, notifications personnelles de suivi et relances contrôlées des files d'échec (e-mails, *payouts*).

Sur le plan technique, plusieurs solutions structurantes ont été conçues pour traiter les difficultés rencontrées : un *webhook* de paiement signé en HMAC et idempotent grâce à un index unique (`provider`, `providerPaymentId`) ; une mise à jour conditionnelle atomique de la collection `projects` (`currentFunding` + montant $\leq$ `fundingGoal`) qui préserve l'intégrité du financement sous charge concurrente ; un repli déterministe pour le moteur de recommandation et un cron `retryStuckAiAnalyses` pour les projets bloqués en `AWAITING_AI` en cas d'indisponibilité du LLM ; quatre files *dead-letter* côté collections `failedWorkflowEvents`, `failedRefundEvents`, `failedPayoutEvents` et `failedCancellationEvents`, qui rendent visible toute erreur définitive dans les chaînes asynchrones n8n / BullMQ et permettent une reprise manuelle ; enfin, le chiffrement AES-256-GCM des RIB avec un IV unique par enregistrement assure la confidentialité des coordonnées bancaires.

Indépendamment de la dimension strictement logicielle, le dispositif adopté illustre une

démarche articulée d'ingénierie des systèmes d'information, associant capture des besoins, modélisation UML, conception des données et validation par scénarios métier.

Apports personnels et compétences acquises

Ce projet de fin d'études a constitué une expérience formatrice à plusieurs niveaux :

- **Développement web complet** : conception d'une SPA React et d'API REST Express, avec séparation claire entre logique métier et accès aux données.
- **Sécurité applicative** : authentification par cookies HttpOnly, contrôle d'accès par rôles, limitation du débit des requêtes, signatures *webhook* et chiffrement applicatif.
- **Données et intégrité** : modélisation MongoDB, indexation, transactions multi-documents et gestion des cas critiques (sur-financement, remboursements, retraits).
- **IA appliquée au métier** : analyse de risque déclenchée par n8n mais exécutée par Express/Gemini (`/internal/run-risk-analysis`); assistant conversationnel synchrone (`/api/projects/:id/chat`); gestion des quotas et modes de repli.
- **Analyse et modélisation** : formalisation des besoins, cas d'utilisation, diagrammes de séquence et de classes pour structurer le système d'information.
- **Validation** : conception de scénarios de tests de bout en bout et contrôles de cohérence systématiques sur les statuts, les transitions et les messages affichés à l'utilisateur.

La période consacrée à ce stage a permis d'opérer une mise en pratique systématique des enseignements de la licence tout en renforçant l'autonomie nécessaire à la conduite d'un projet d'envergure. Les échanges réguliers avec l'encadrant se sont révélés déterminants pour consolider les arbitrages techniques et pour lever les blocages ponctuels rencontrés. L'intégration de briques d'intelligence artificielle au sein d'un système d'information métier a en outre offert un terrain privilégié d'appréhender, sur le plan empirique, les exigences attachées à ces technologies (performance, exactitude, gestion des erreurs), au-delà du seul cadre conceptuel.

Critiques et limites

Malgré les résultats obtenus, certaines limites subsistent :

- Le moteur de recommandation est principalement fondé sur les préférences et des règles de compatibilité (catégories/risque); l'absence de données historiques limite l'évaluation empirique.

- L'analyse de risque reste dépendante de la qualité de la description du projet ; une revue humaine demeure nécessaire, surtout pour les projets sensibles.
- La plateforme n'inclut pas de version mobile native, bien que le site soit *responsive*.
- Les validations ont été réalisées par scénarios automatisés côté API et tests manuels ; une automatisation IHM (tests UI) renforcerait encore la qualité.
- L'intégration d'un prestataire de paiement réel implique des exigences supplémentaires (KYC, *sandbox*, certificats, supervision) qui dépassent le périmètre PFE.
- La plateforme n'a pas été mise en ligne sur un serveur d'exploitation distant : développement, exécution et validation ont porté sur un environnement local (`localhost`).

Perspectives

Plusieurs améliorations peuvent être envisagées pour enrichir le système :

- **Industrialiser le paiement** : intégrer un prestataire réel (*sandbox*), renforcer l'observabilité des *webhooks* et formaliser les contrôles de conformité.
- **Améliorer la décision** : affiner l'analyse de risque (meilleure explication, calibration) et enrichir les recommandations par apprentissage dès qu'un historique existe.
- **Pilotage SI** : ajouter des tableaux de bord (KPIs) orientés métier (taux de succès, délais de validation, volumes de contribution, incidents).
- **Qualité** : automatiser des tests UI et renforcer les tests de non-régression sur le cycle de vie projet/paiement/*payout*.

Ce travail illustre par ailleurs l'intérêt d'intégrer l'IA dans un système d'information métier pour renforcer la transparence (rapport d'analyse lisible, assistant contextuel), complétée par des règles métier explicites (recommandations, heuristiques budgétaires) et une gouvernance humaine (validation, modération).

Webographie

- Banque Centrale de Tunisie (BCT) — Cadre légal du financement participatif (Tunisie).
<https://www.bct.gov.tn/>
- React — Documentation officielle. <https://react.dev/>
- Node.js — Documentation officielle. <https://nodejs.org/>
- MongoDB — Documentation officielle. <https://www.mongodb.com/docs/>
- n8n — Documentation officielle. <https://n8n.io/>

Bibliographie

- [1] Banque Centrale de Tunisie. (2020). *Loi n° 2020-37 du 2 juillet 2020 relative au financement participatif*. Journal Officiel de la République Tunisienne, n° 76.
- [2] OWASP Foundation. (2021). *OWASP Top Ten Web Application Security Risks*. Consulté le 3 mai 2026. <https://owasp.org/Top10/>
- [3] Jones, M., Bradley, J., & Sakimura, N. (2015). *JSON Web Token (JWT) - RFC 7519*. Internet Engineering Task Force (IETF).

Annexe

Format type d'une description textuelle de cas d'utilisation

Le tableau 17 reprend le modèle utilisé au chapitre 2 pour les fiches détaillées (section 2.5.2) et, le cas échéant, pour les autres cas listés aux tableaux 2 à 6.

TABLEAU 17 – Modèle de description textuelle d'un cas d'utilisation

Rubrique	Contenu attendu
Cas d'utilisation	Nom métier.
Acteur(s)	Rôles métier déclencheurs et participants.
Précondition	État du système requis avant l'exécution.
Postcondition	État du système garanti après une exécution réussie.
Scénario nominal	1. Initiatives de l'acteur principal et sollicitations éventuelles du système (ordre chronologique).
	2. Enchaînements et validations côté application, services internes ou tiers.
	3. Conclusion nominale : résultat métier, persistance des données, effets annexes (notifications, etc.).
Scénarios alternatifs	1. Première variante (succès partiel, autre issue, branche optionnelle).
	2. Autres variantes le cas échéant (une ligne numérotée par variante dans cette colonne).
Exceptions	1. Erreurs de saisie, règles métier ou validations refusées.
	2. Défaillances techniques ou refus d'accès ; comportement observable (code retour, message).